Inatel

Tópicos Avançados em IoT e Machine Learning

Mestrado e Doutorado em Telecomunicações

Profª. Victoria Dala Pegorara Souto

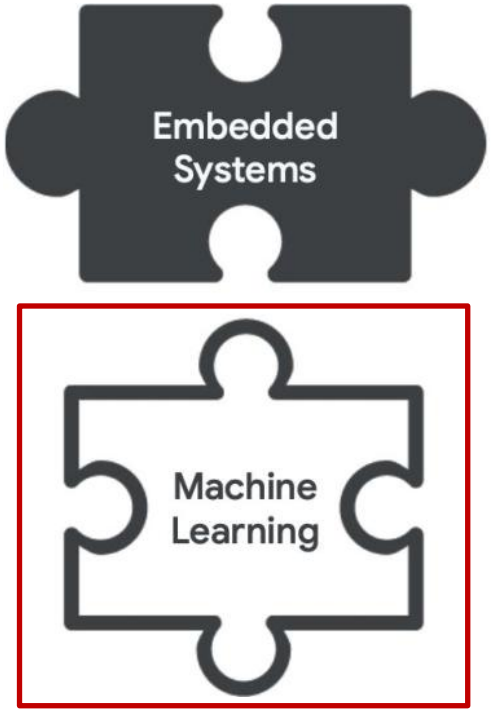
Desafios do TinyML

Machine Learning



Inatel

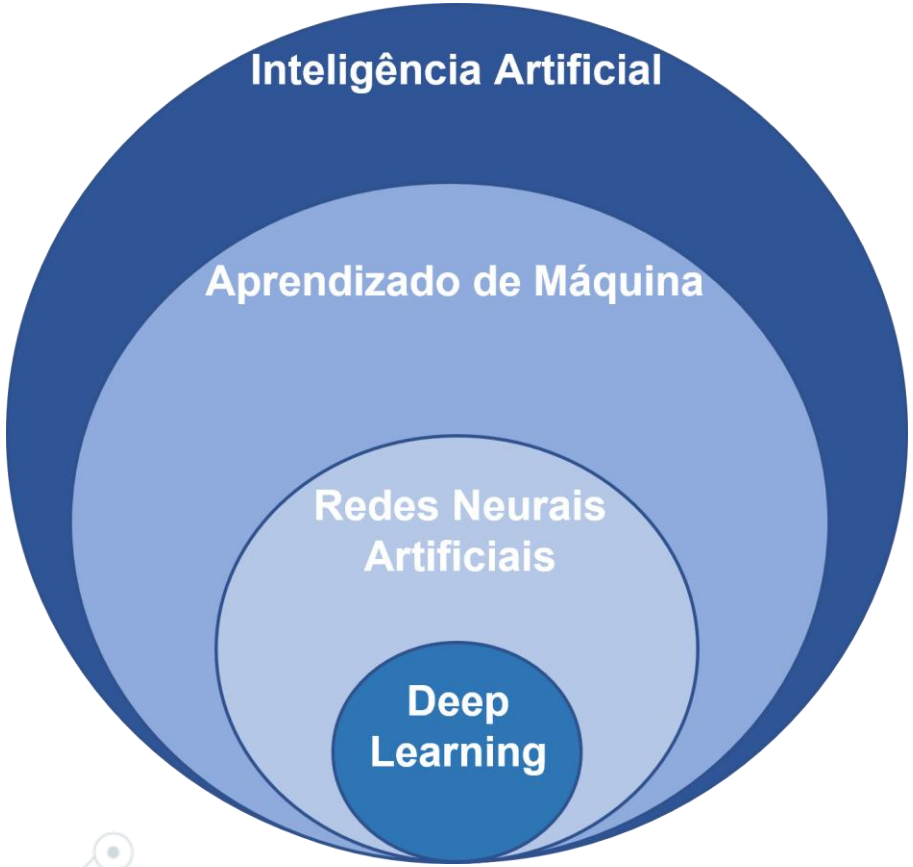
Desafios da Execução de ML em Sistemas Embarcados



EdgeML

- Anteriormente, falamos das **limitações de HW e SW quando usamos sistemas embarcados**.
- Agora, veremos os **desafios para a execução de algoritmos de ML**, mais especificamente de *Deep Learning*, nestes dispositivos
  - **Pequenos e com restrições de custo, de recursos computacionais e consumo**.

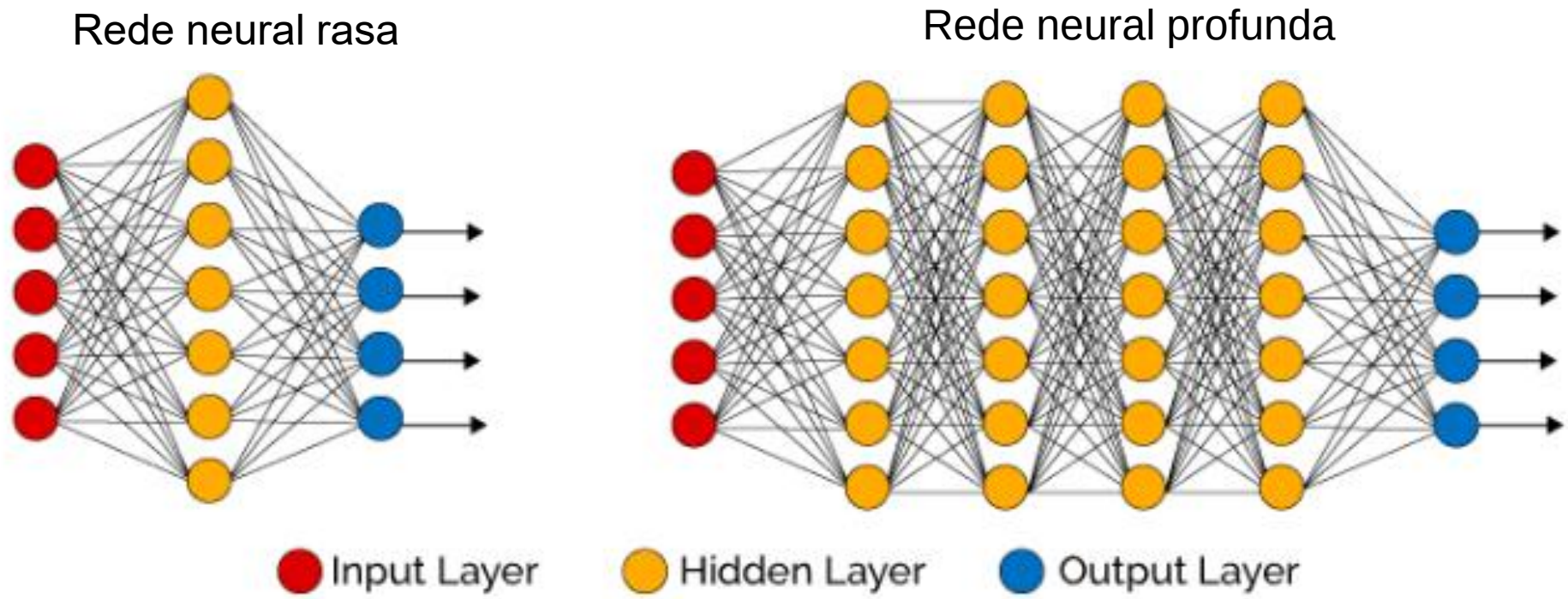
Deep Learning (Aprendizado Profundo)



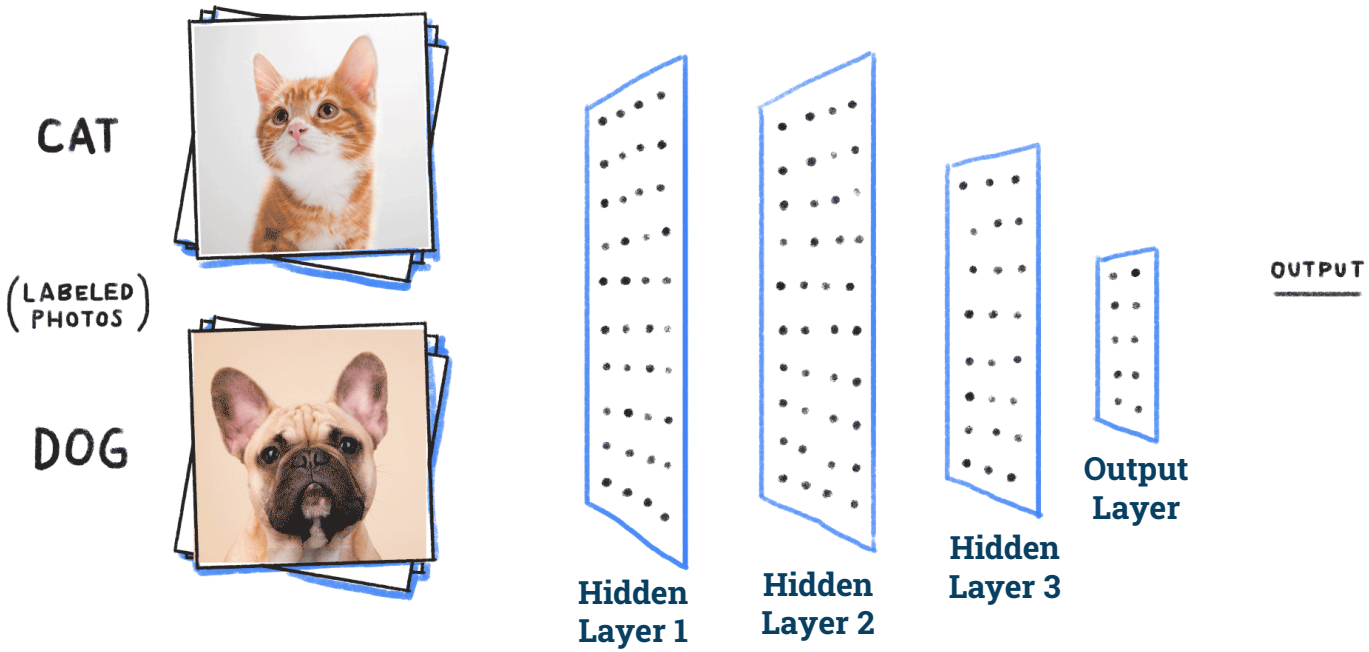
Subárea do aprendizado de máquina que usa **Redes Neurais Artificiais com muitas camadas ocultas** para aprender com dados.

Deep Learning (Aprendizado Profundo)

- Por terem uma **grande capacidade de aprendizado**, em geral, precisam de uma **grande quantidade de dados** para aprenderem (i.e., encontrar uma solução geral).



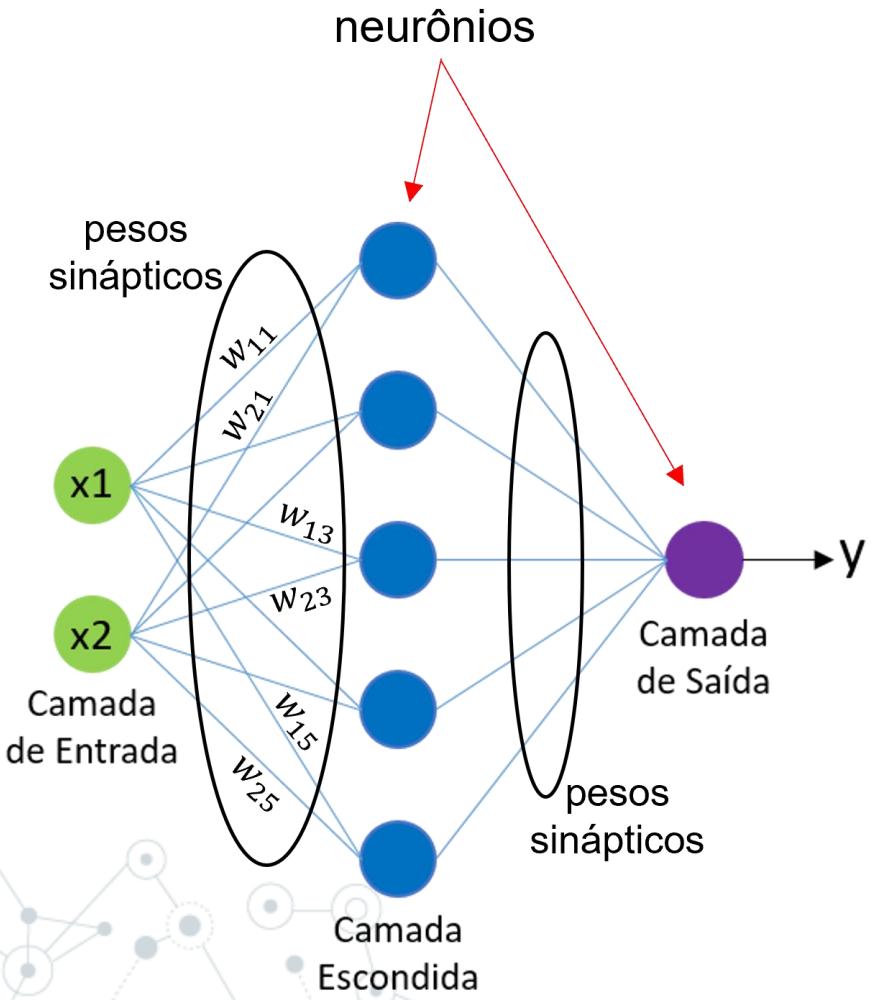
Deep Learning (Aprendizado Profundo)



- Um exemplo muito comum de uso do *Deep Learning* é a **Classificação de Imagens**.
- Se tivermos uma base de dados com imagens de gatos e cachorros, podemos **treinar uma rede neural profunda para identificá-los** em imagens.
- O **Objetivo** é obter um **modelo que generalize**, ou seja, que identifique gatos ou cachorros não vistos durante o treinamento.

Deep Learning (Aprendizado Profundo)

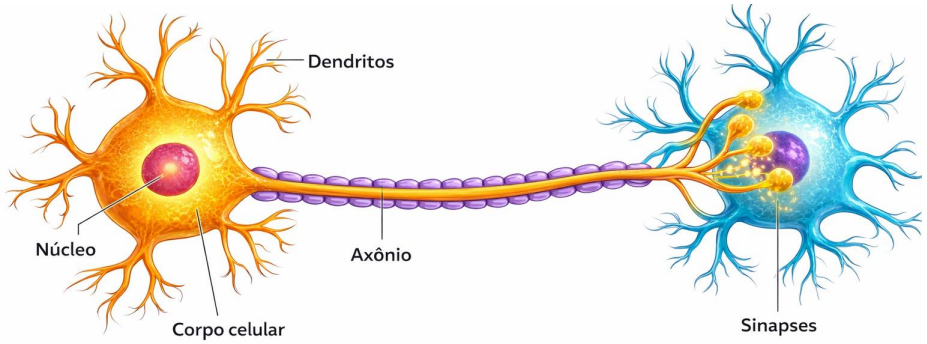
O que é uma Rede Neural?



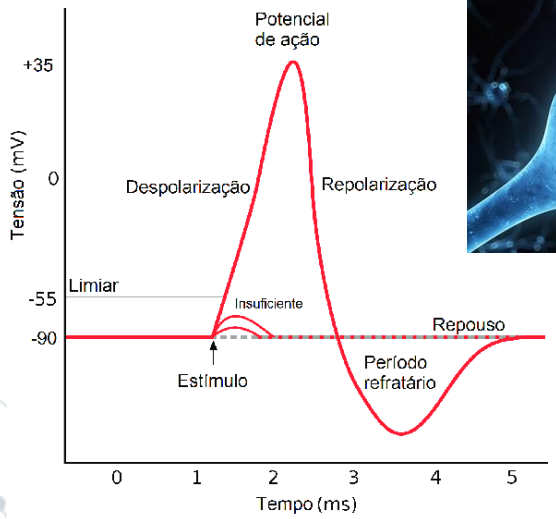
- É uma **conexão de camadas de neurônios artificiais**, ou também chamados de nós.
- Os **Neurônios Artificiais** são **Modelos Matemáticos inspirados no funcionamento de um Neurônio Biológico**.
- Os **neurônios** das diferentes camadas **são conectados através dos pesos sinápticos**, que determinam a **força daquela conexão**.

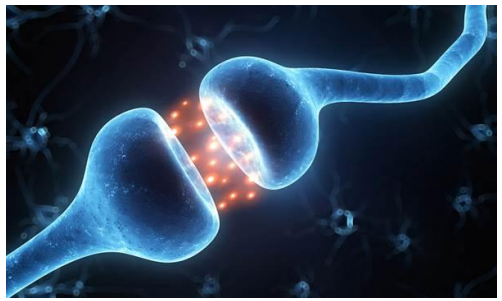
Deep Learning (Aprendizado Profundo)

Neurônio Biológico



- **NEURÔNIOS:**
  - São **células** que possuem **mecanismos eletroquímicos** para realizar a **transmissão de sinais elétricos** (i.e., informações) **ao longo do Sistema Nervoso**.

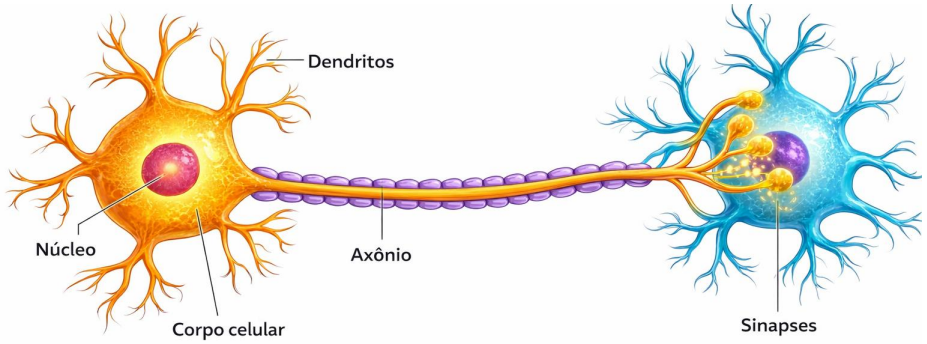


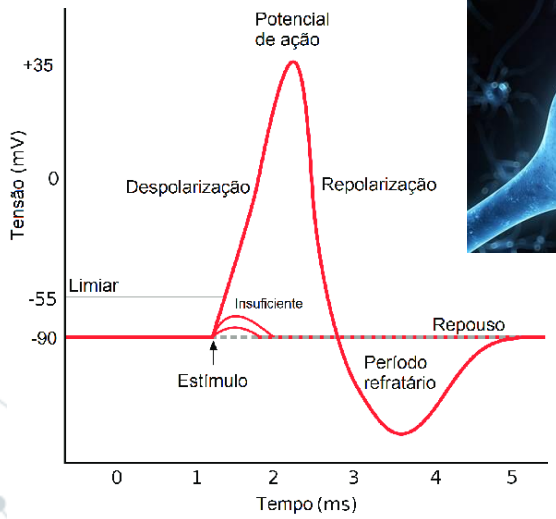


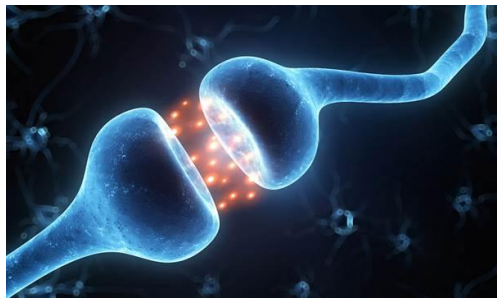
→ Têm três partes fundamentais: os **DENDRITOS**, o **AXÔNIO** e o **CORPO CELULAR**, também chamado de **SOMA**.

Deep Learning (Aprendizado Profundo)

Neurônio Biológico



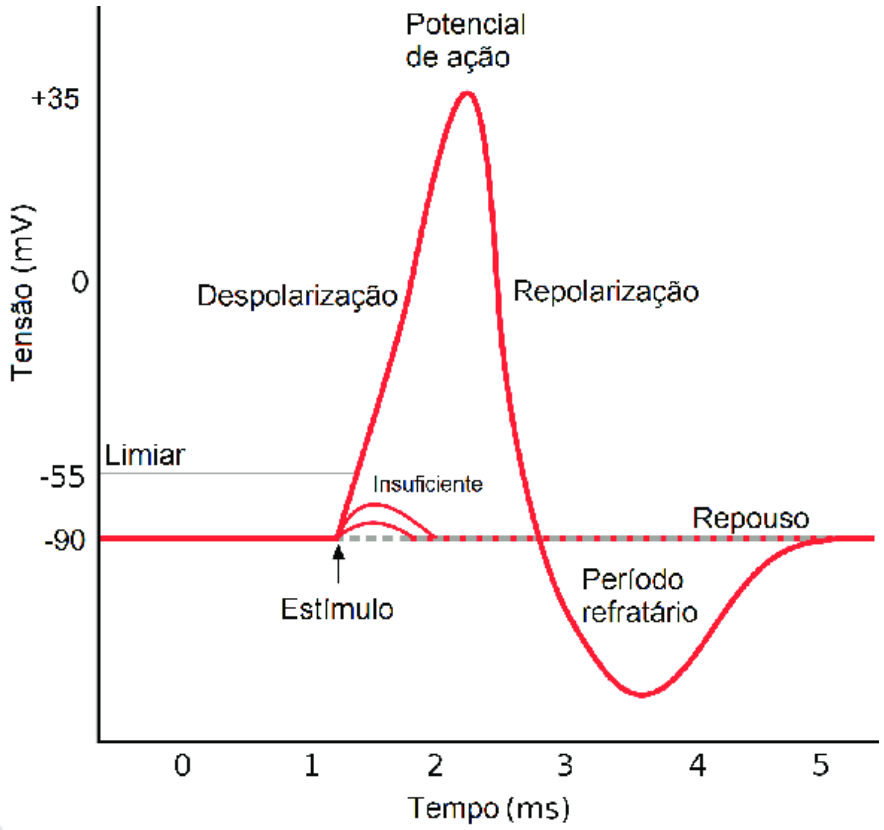




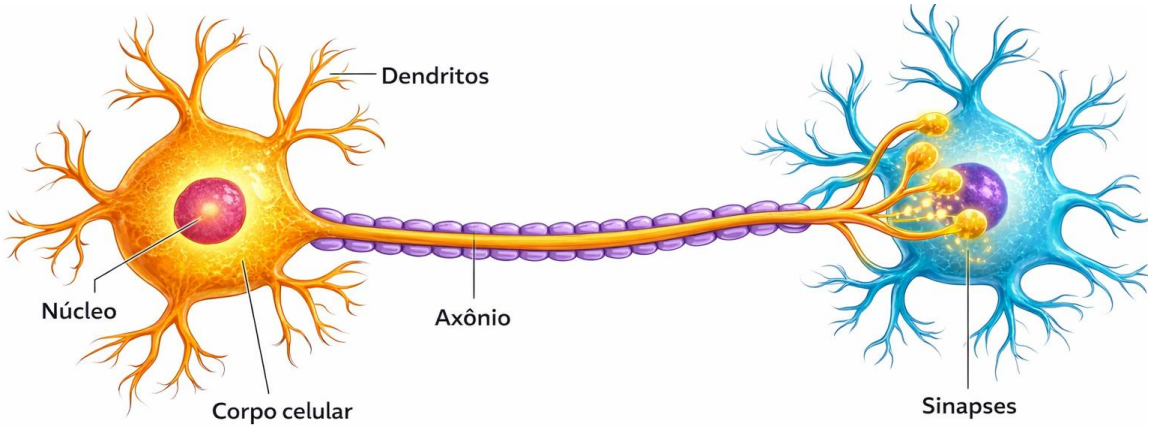
- Os **DENDRITOS** **recebem estímulos** vindos **de outros neurônios** e os levam em direção ao **corpo celular**.
- O **CORPO CELULAR** realiza a **integração** dos **estímulos e gera impulsos** caso um **limiar** seja ultrapassado.
- O **AXÔNIO** envia **impulsos** a outros neurônios através de seus **terminais**.
- Os neurônios se comunicam uns com os outros através das **SINAPSES**.
- **Sinapses** são os **pontos de contato** entre os **DENDRITOS** de um neurônio e os **terminais do AXÔNIO** de outros neurônios.

Deep Learning (Aprendizado Profundo)

Neurônio Biológico



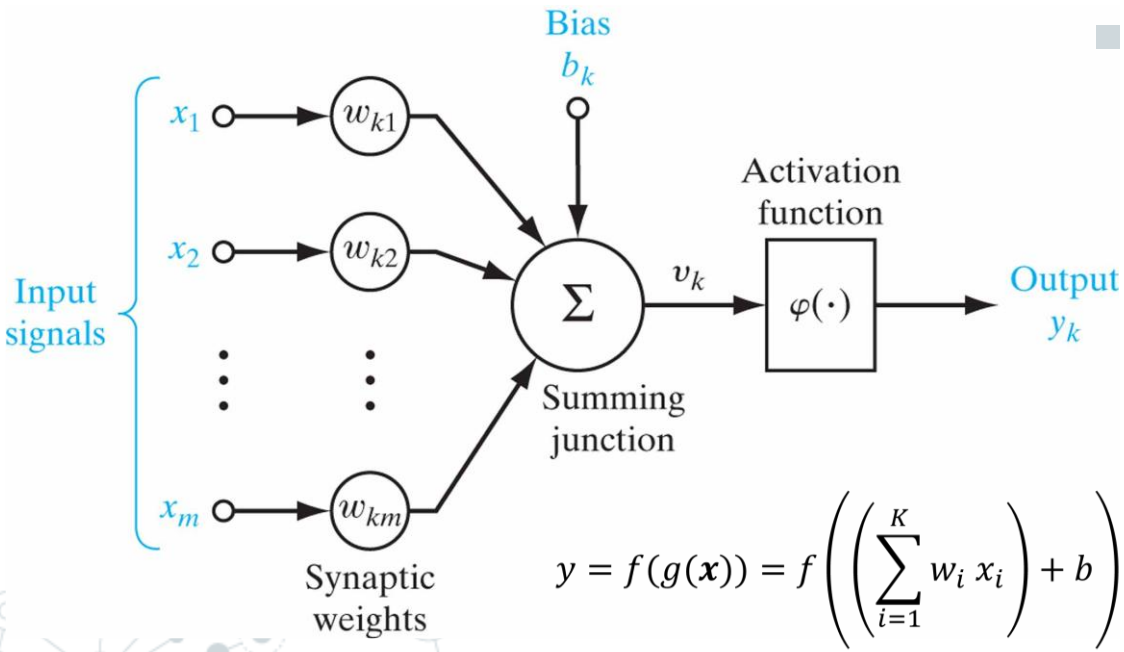
- Em termos bem simples, o funcionamento do **NEURÔNIO** pode ser explicado da seguinte forma:
  - Ele **recebe estímulos elétricos** através dos DENDRITOS.
  - Esses **estímulos são integrados** no CORPO CELULAR (soma).
  - Se a **integração dos estímulos** exceder um certo **limiar de ativação**, o **neurônio** gera um **impulso** (ou **potencial de ação/ativação**) que é enviado pelos terminais do AXÔNIO a outros neurônios através das **SINAPSES**.



Deep Learning (Aprendizado Profundo)

Neurônio Artificial

- Baseado no entendimento do funcionamento do neurônio biológico, a partir de meados da década de 40, pesquisadores propuseram o modelo matemático ao lado.
  - É uma **simplificação e abstração** do funcionamento do neurônio biológico.



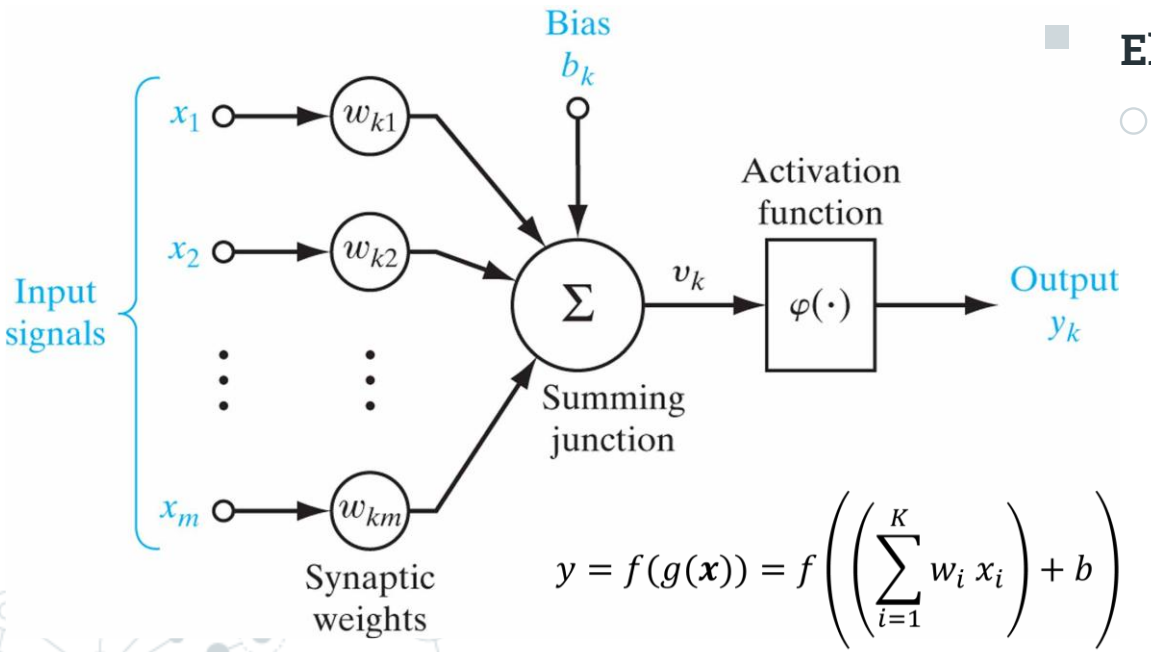
Elementos Básicos de um Neurônio Artificial:

- **Sinais de entrada** $\{x_1, x_2, \ldots, x_m\}$: sinais ou medidas do meio externo que representam os valores das variáveis;
- **Pesos sinápticos** $\{w_{k1}, w_{k2}, \ldots, w_{km}\}$: valores que servirão para ponderar cada uma das variáveis de entrada (indicam a relevância de cada entrada na rede);
- **Combinador linear** $\{\sum\}$: sua função é agregar todos os sinais de entrada que foram ponderados pelos respectivos pesos sinápticos a fim de gerar um valor de potencial de ativação;

Deep Learning (Aprendizado Profundo)

Neurônio Artificial

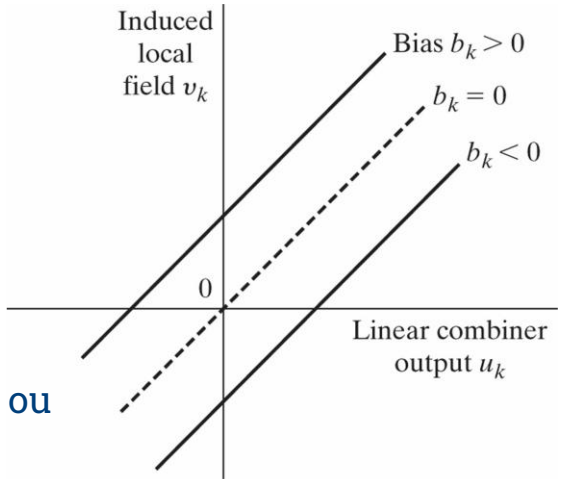
- Baseado no entendimento do funcionamento do neurônio biológico, a partir de meados da década de 40, pesquisadores propuseram o modelo matemático ao lado.
  - É uma **simplificação e abstração** do funcionamento do neurônio biológico.



$$y = f(g(\boldsymbol{x})) = f\left(\left(\sum_{i=1}^{K} w_i\, x_i\right) + b\right)$$

Elementos Básicos de um Neurônio Artificial:

- **Limiar de ação/ativação $\{\theta\}$ ou $\{b_k\}$:** variável que especifica qual será o patamar apropriado para que o resultado produzido pelo combinador linear possa gerar um valor de disparo em direção a saída do neurônio;

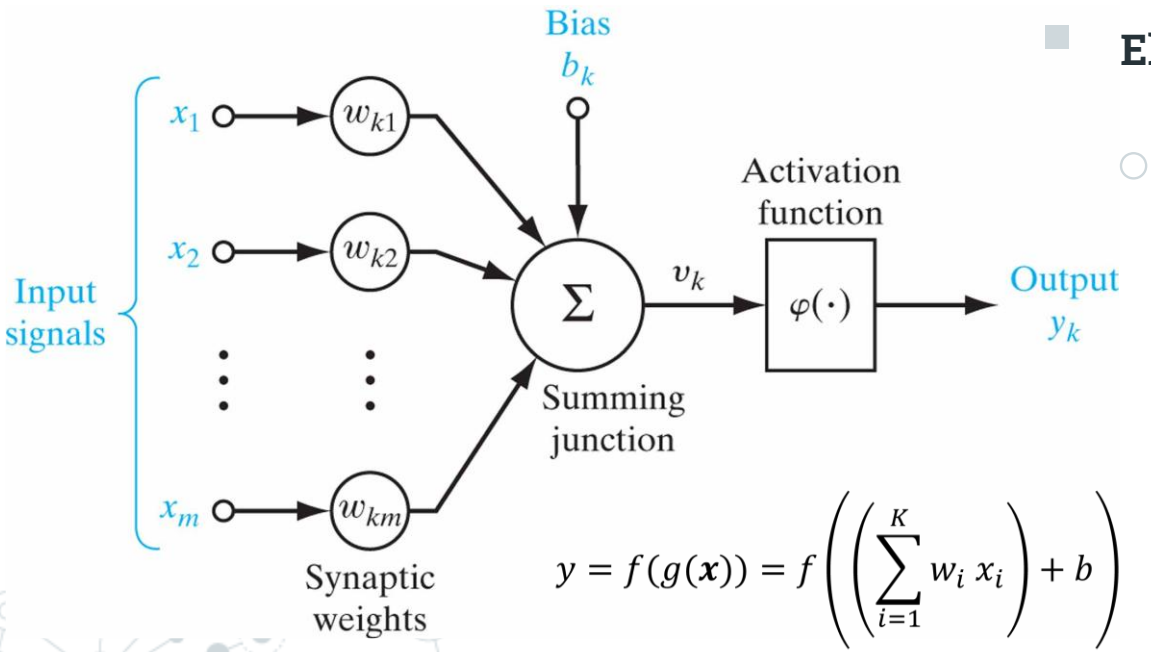


Campo local induzido ou potencial de ativação

Deep Learning (Aprendizado Profundo)

Neurônio Artificial

- Baseado no entendimento do funcionamento do neurônio biológico, a partir de meados da década de 40, pesquisadores propuseram o modelo matemático ao lado.
  - É uma **simplificação e abstração** do funcionamento do neurônio biológico.



$$y = f(g(\boldsymbol{x})) = f\left(\left(\sum_{i=1}^{K} w_i\, x_i\right) + b\right)$$

Elementos Básicos de um Neurônio Artificial:

- **Potencial de ativação $\{v_k\}$:** resultado produzido pela diferença do valor produzido entre o combinador linear e o limiar de ativação. Se tal valor é positivo, ou seja, **se $u_k + b_k \geq 0$ então o neurônio produz um potencial excitatório; caso contrário, o potencial será inibitório;**

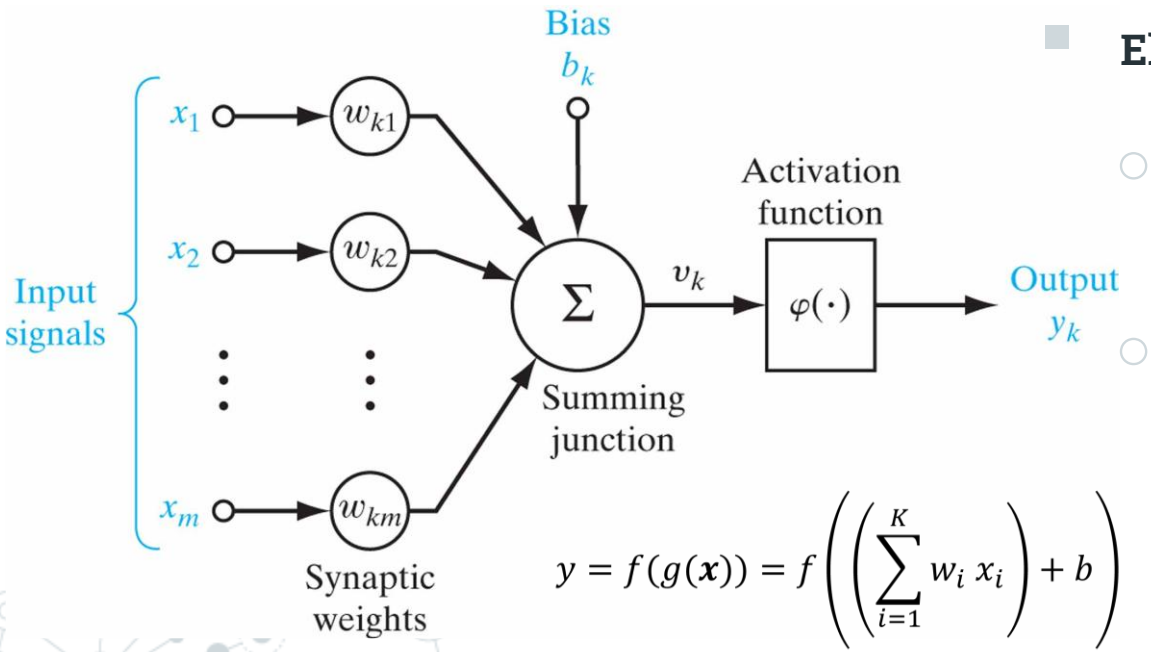
$$u_k = \sum_{j=1}^{m} w_{kj}.\, x_j$$

$$v_k = u_k + b_k$$

Deep Learning (Aprendizado Profundo)

Neurônio Artificial

- Baseado no entendimento do funcionamento do neurônio biológico, a partir de meados da década de 40, pesquisadores propuseram o modelo matemático ao lado.
  - É uma **simplificação e abstração** do funcionamento do neurônio biológico.



$$y = f(g(\boldsymbol{x})) = f\left(\left(\sum_{i=1}^{K} w_i\, x_i\right) + b\right)$$

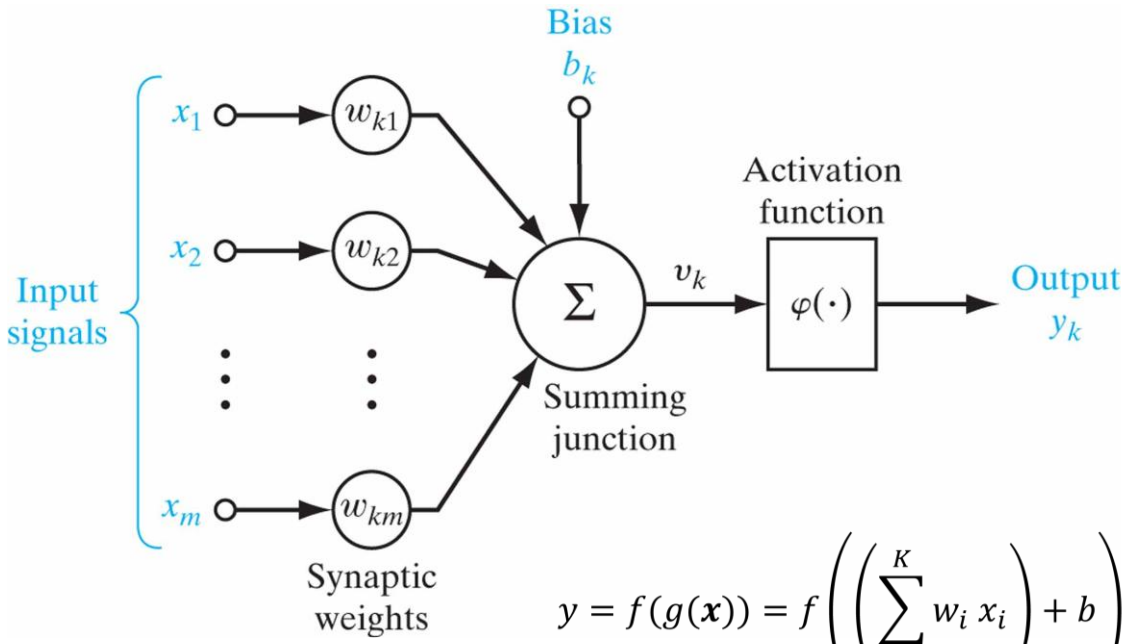
Elementos Básicos de um Neurônio Artificial:

- **Função de ativação** $\{\varphi\}$: seu objetivo é limitar a saída do neurônio dentro de um intervalo de valores razoáveis a serem assumidos pela sua própria imagem funcional;
- **Sinal de saída** $\{y_k\}$: valor final produzido pelo neurônio em relação a um determinado conjunto de sinais de entrada, podendo ser também utilizado por outros neurônios que estão sequencialmente interligados;

$$y_k = \varphi(u_k + b_k) = \varphi(v_k)$$

Deep Learning (Aprendizado Profundo)

Neurônio Artificial



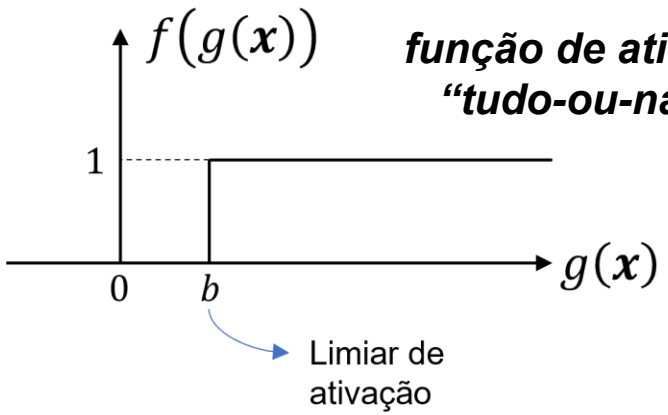
$$y = f(g(\boldsymbol{x})) = f\left(\left(\sum_{i=1}^{K} w_i\, x_i\right) + b\right)$$

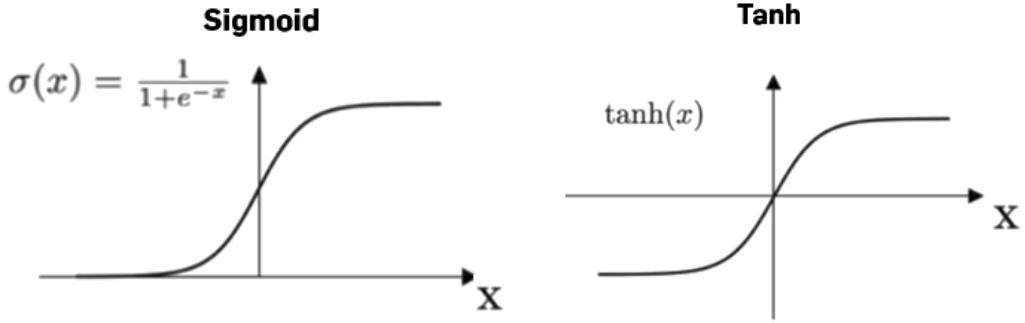
→ Resumo:

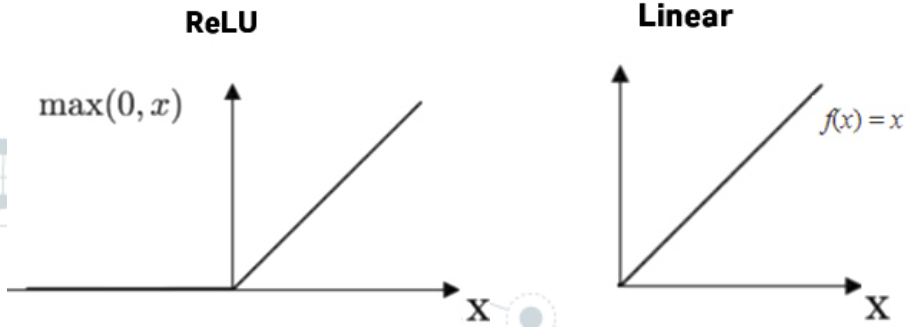
- Os **estímulos**, $x_k, k = 1, \ldots, K$, são multiplicados pelos **pesos sinápticos**, $w_k, k = 1, \ldots, K$, somados com o **peso de bias**, b, e o resultado é passado por uma **função de ativação**, $f(.)$.
- No contexto de redes neurais os estímulos são chamados de **atributos**.
- O **peso de bias**, b, controla o **ponto de ativação**.
- Os **pesos sinápticos** w_k dão a **força de cada conexão** e se ela é **excitatória ou inibitória**.

Deep Learning (Aprendizado Profundo)

Neurônio Artificial



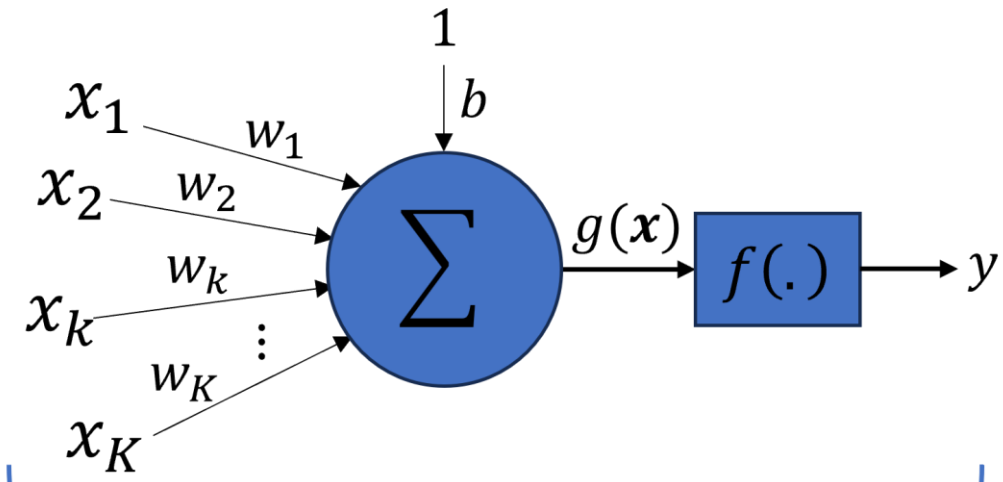




- O **peso de bias**, b, permite **ajustar o limiar de ativação** (ou ponto de disparo) da função de ativação.
- No início dos estudos dos neurônios, a **função de ativação** era do tipo **tudo-ou-nada**, ou seja, ativava ou não (degrau unitário).
- Com o decorrer do tempo, outras funções com características diferentes foram propostas, tais como as funções **Sigmoide, Tangente Hiperbólica, Relu, Linear, etc.**

Deep Learning (Aprendizado Profundo)

Neurônio Artificial



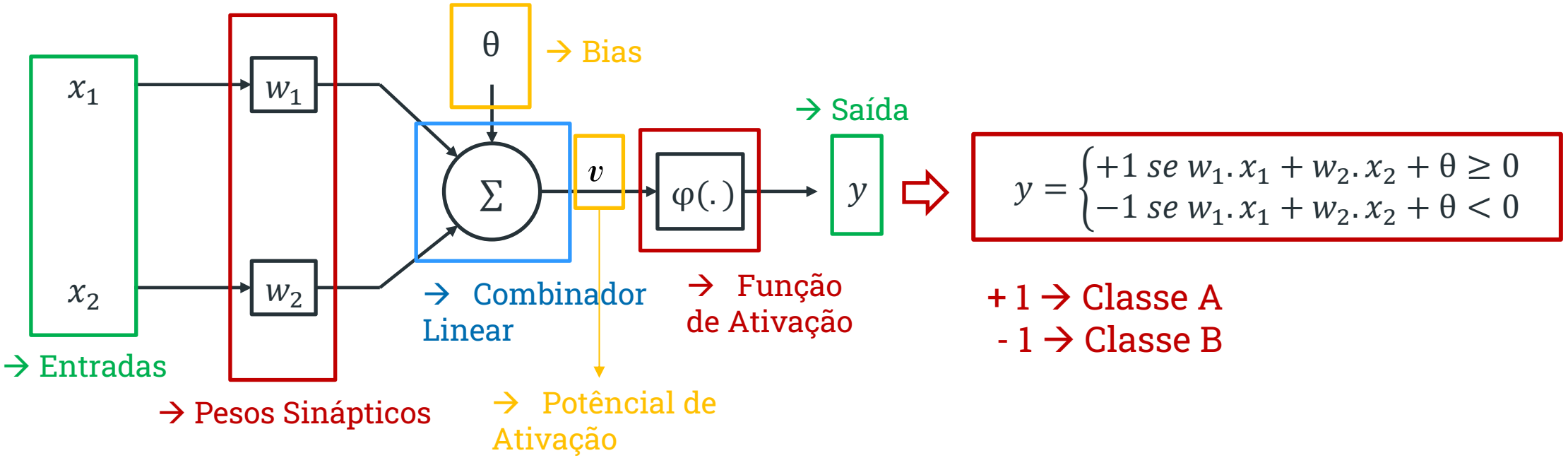
$$y = f\left(\left(\sum_{i=1}^{K} w_i x_i\right) + b\right)$$

Pesos ou parâmetros

- O **OBJETIVO** do **treinamento** de uma rede neural artificial é **encontrar os valores ideais dos pesos** (bias e sinápticos) **de todos os neurônios** de forma que a rede **resolva uma tarefa específica**, como, por exemplo, a classificação de imagens.
- Se os atributos são os pixels das imagens e gatos e cachorros são representados por saídas iguais a 1 e 0 (*labels*), respectivamente, **quais devem ser os valores dos pesos para classificar corretamente as imagens?**
- Os pesos são ajustados através de um **processo de otimização iterativo** chamado de **Retropropagação do Erro**, onde apresenta-se ao modelo as **entradas** e **saídas esperadas** e o processo **minimiza o erro** entre a **saída da rede** e os **valores de saída esperados, chamados de rótulos ou *labels***.

Deep Learning (Aprendizado Profundo)

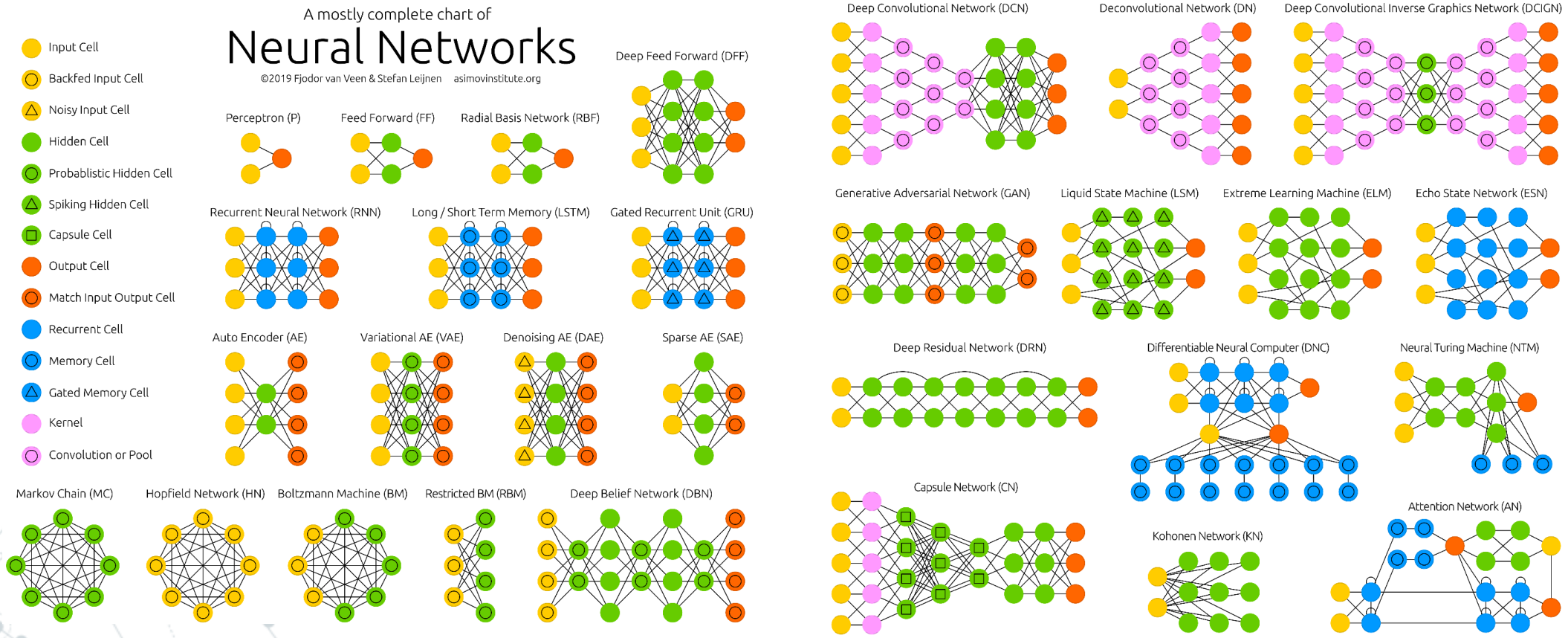
Neurônio Artificial



Deep Learning (Aprendizado Profundo)

Arquiteturas de Redes Neurais Artificiais

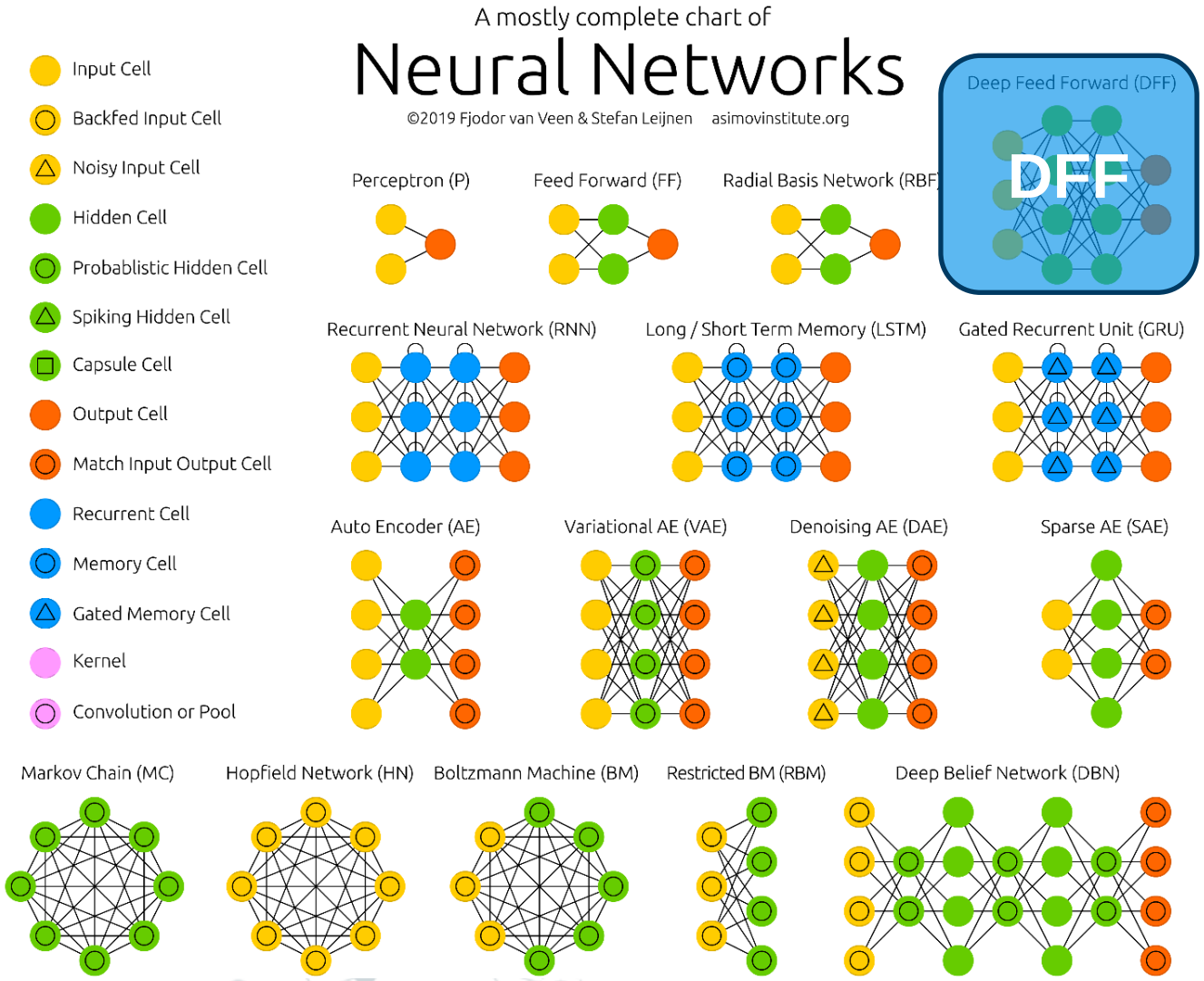
- Diferentes **tipos de neurônios**, **quantidades de camadas** e de **neurônios** em cada uma delas e a **forma como eles são conectados**, resultam em **arquiteturas** diferentes.



Fonte: https://www.asimovinstitute.org/neural-network-zoo/

Deep Learning (Aprendizado Profundo)

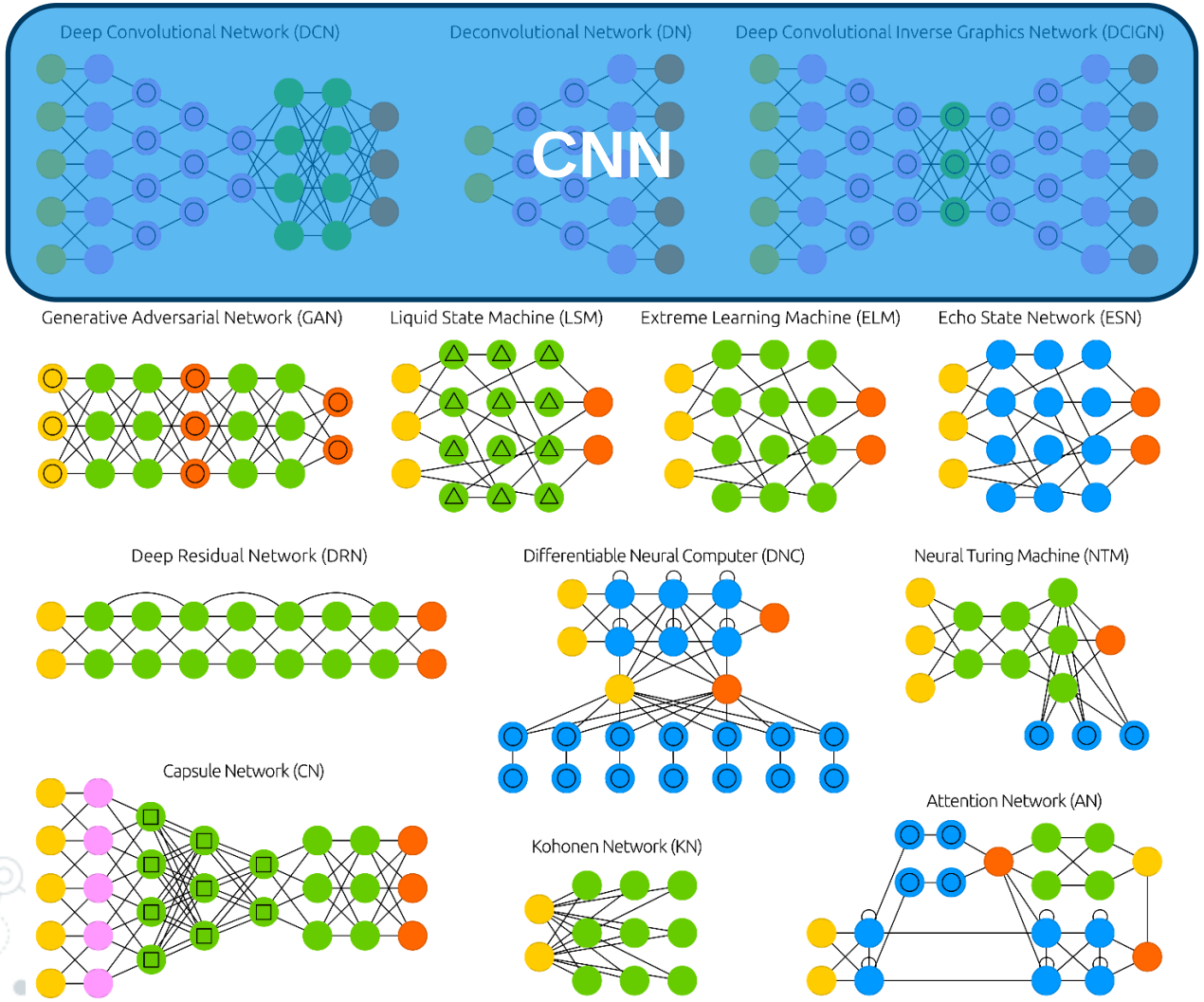
Arquiteturas de Redes Neurais Artificiais



- **Deep Feed Forward (DFF) Networks** -- *Dense Neural Networks (DNNs):*
  - São redes **densamente** conectadas: todas as saídas de uma camada se conectam a todos os nós da próxima camada.
- Para serem consideradas profundas (***Deep***), devem ter 2 ou mais camadas ocultas.
- O termo **FEED FOWARD** vem do fato de que as conexões são sempre no sentido da **entrada para a saída**, ou seja, **não há realimentação**.
- São usadas em tarefas de **aproximação de funções** (i.e., regressão e classificação).

Deep Learning (Aprendizado Profundo)

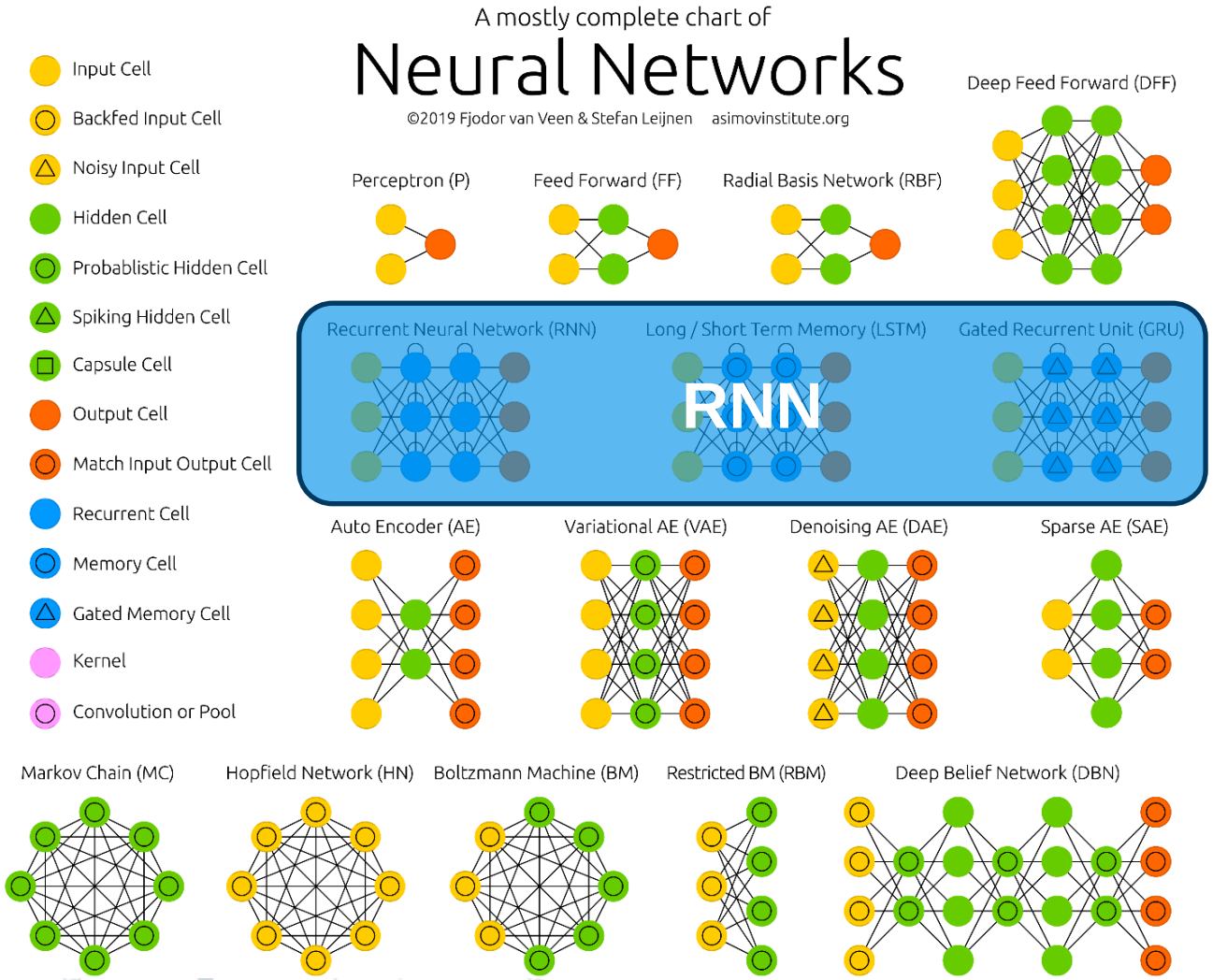
Arquiteturas de Redes Neurais Artificiais



- **Convolutional Neural Networks (CNNs):**
  - São redes que utilizam **camadas de convolução**, que **aplicam filtros para extrair características relevantes** dos dados de entrada, que, em geral, são imagens.
- São usadas em aplicações de **Visão Computacional**, como:
  - Classificação de imagens,
  - Processamento de vídeos,
  - Detecção e segmentação de objetos em imagens ou vídeos.

Deep Learning (Aprendizado Profundo)

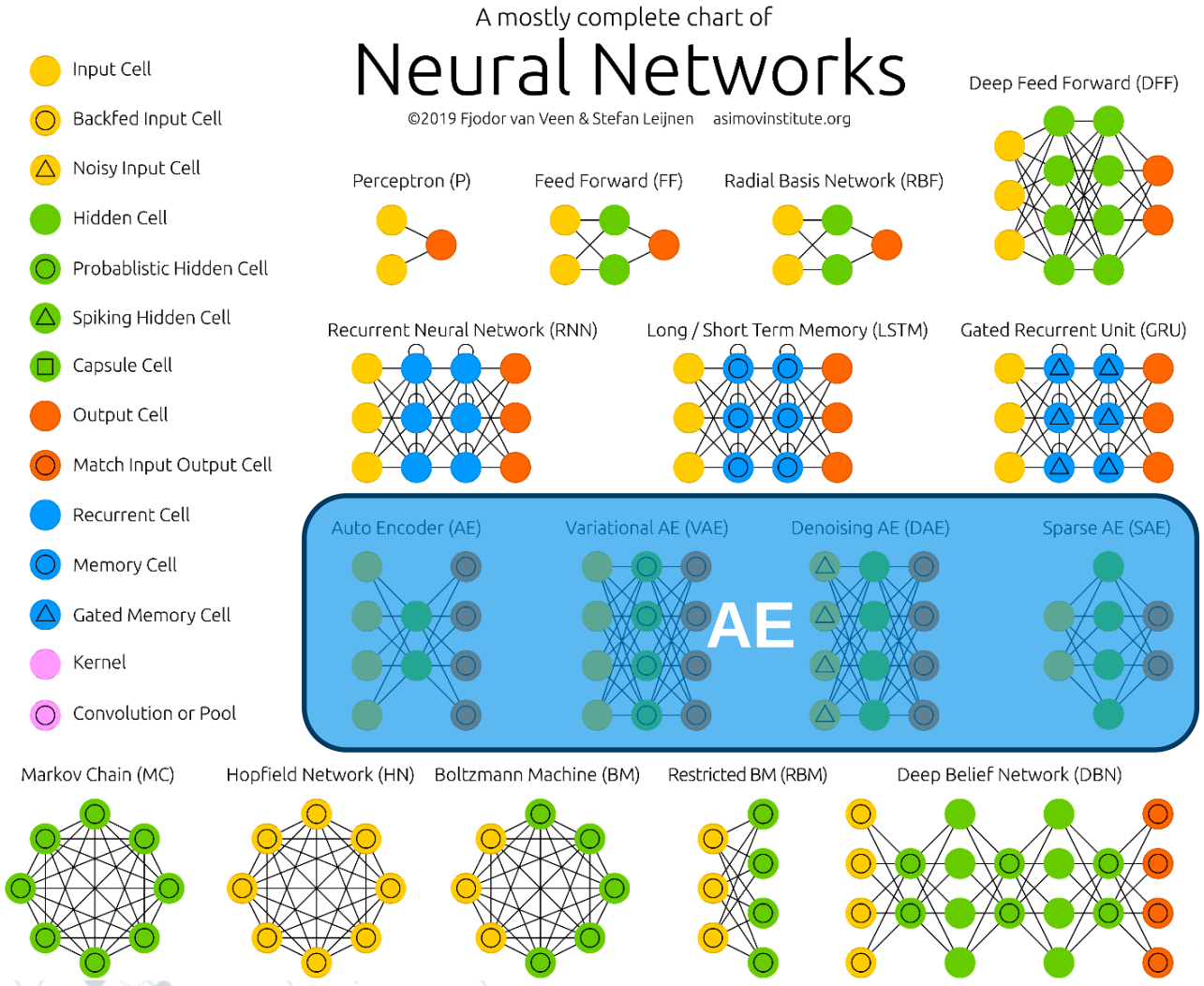
Arquiteturas de Redes Neurais Artificiais



- **Recurrent Neural Networks (RNNs)** ou **Redes Neurais Recorrentes.**
  - Possuem **conexões que formam loops**, permitindo que **informações anteriores sejam armazenadas e influenciem as entradas futuras**, ou seja, elas possuem **memória**.
- Elas consideram o **contexto anterior** ao fazer predições.
- Essa capacidade as torna especialmente úteis em **tarefas que envolvem dados sequenciais ou temporais**, como
  - **Análise de texto**,
  - **Previsão de séries temporais** e
  - **Processamento de fala e áudio**.

Deep Learning (Aprendizado Profundo)

Arquiteturas de Redes Neurais Artificiais



Autoencoders (AEs) têm como objetivo aprender uma **representação latente** dos dados de entrada.

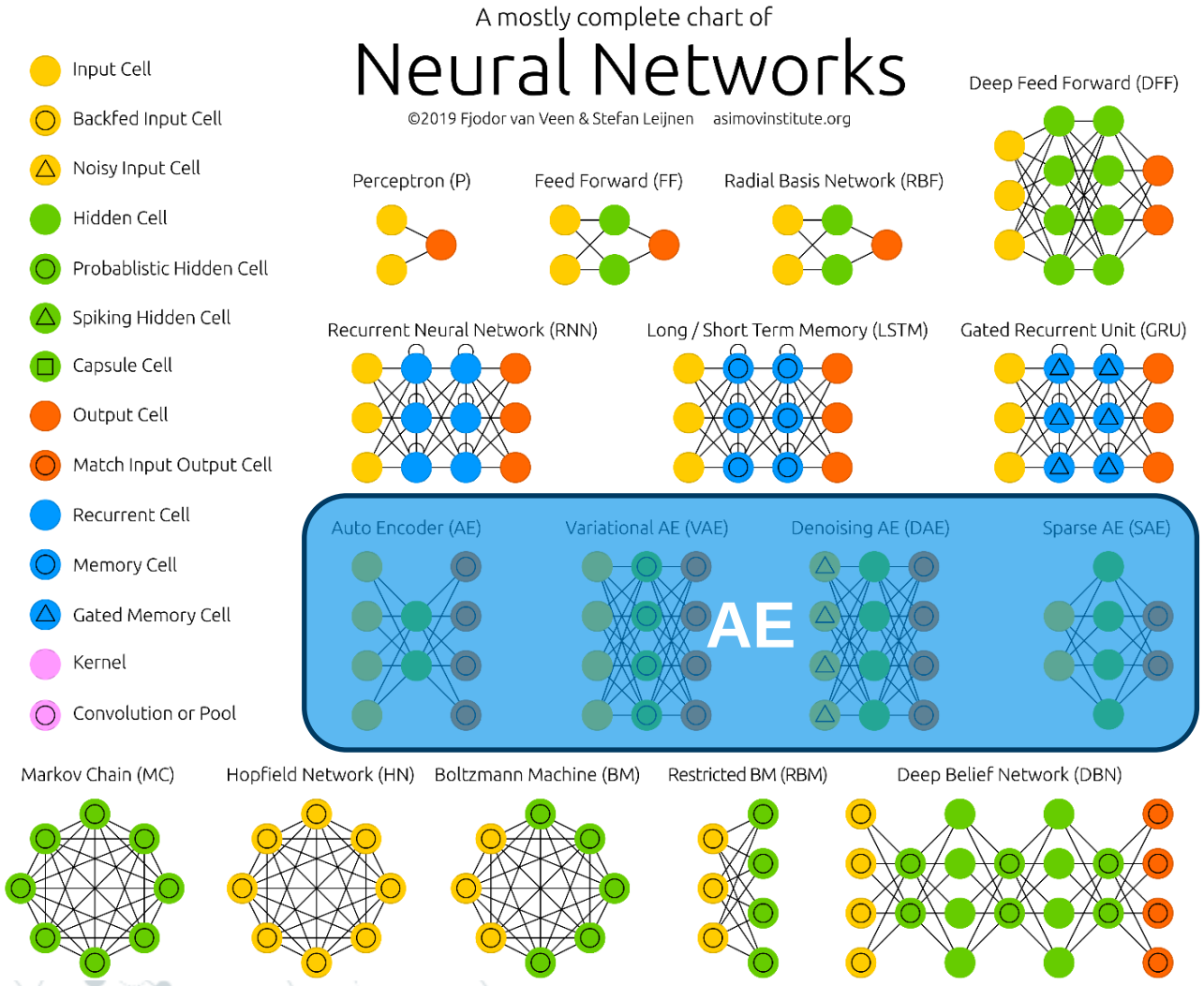
Eles aprendem características importantes, mas ocultas, nos dados.

Consistem em duas partes: **codificador** e **decodificador**.

- O **codificador mapeia** os dados de entrada em uma **representação latente** (de maior ou menor dimensão).
- O **decodificador reconstrói** os **dados originais** a partir da representação com maior ou menor dimensionalidade.

Deep Learning (Aprendizado Profundo)

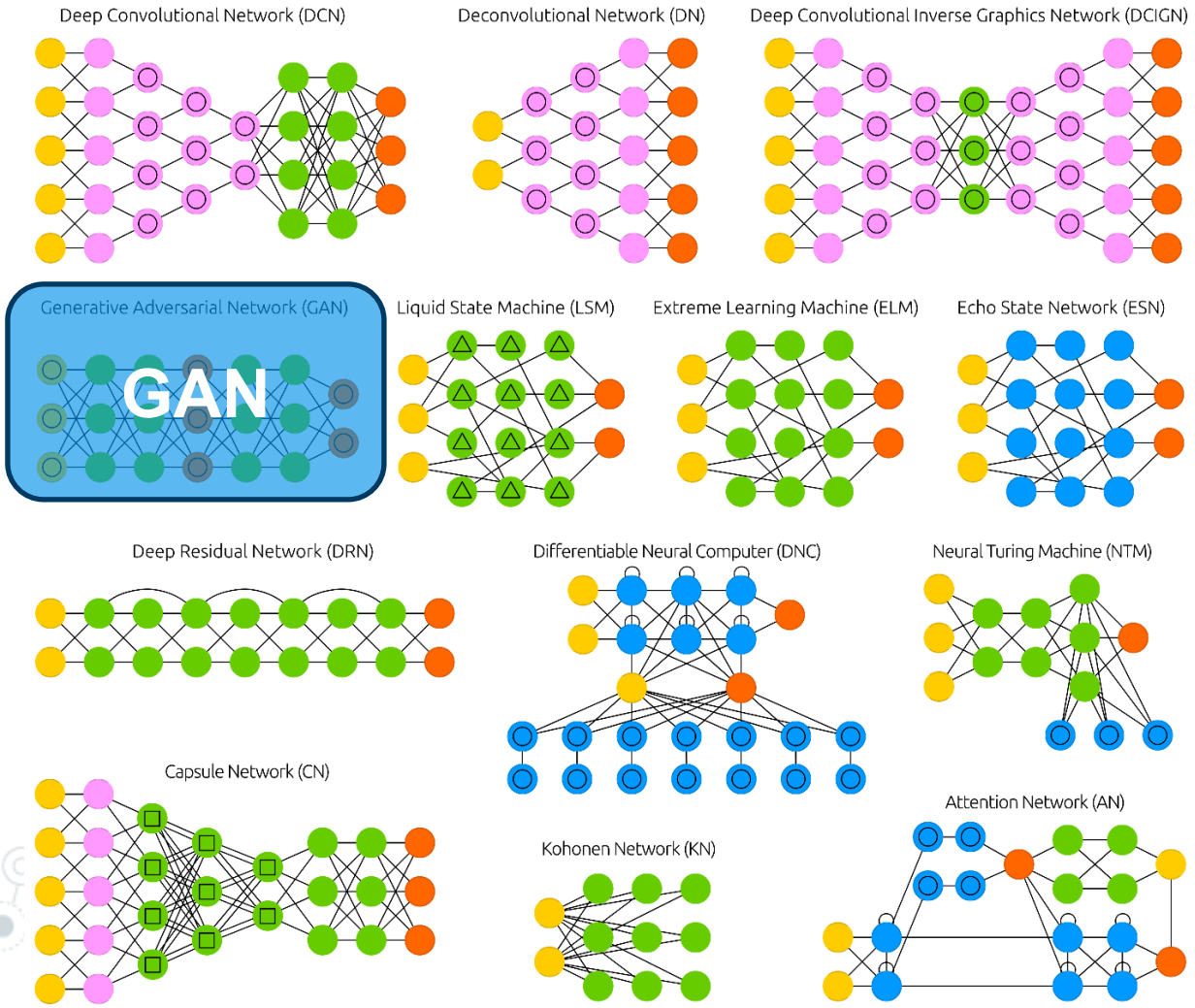
Arquiteturas de Redes Neurais Artificiais



- **Autoencoders (AEs)** têm como objetivo aprender uma **representação latente** dos dados de entrada.
- Eles aprendem características importantes, mas ocultas, nos dados.
- São usados em tarefas de
  - ***Redução de dimensionalidade***,
  - ***Remoção de ruído***,
  - ***Compressão de dados***,
  - ***Geração de dados sintéticos*** e
  - ***Adição de redundância***.

Deep Learning (Aprendizado Profundo)

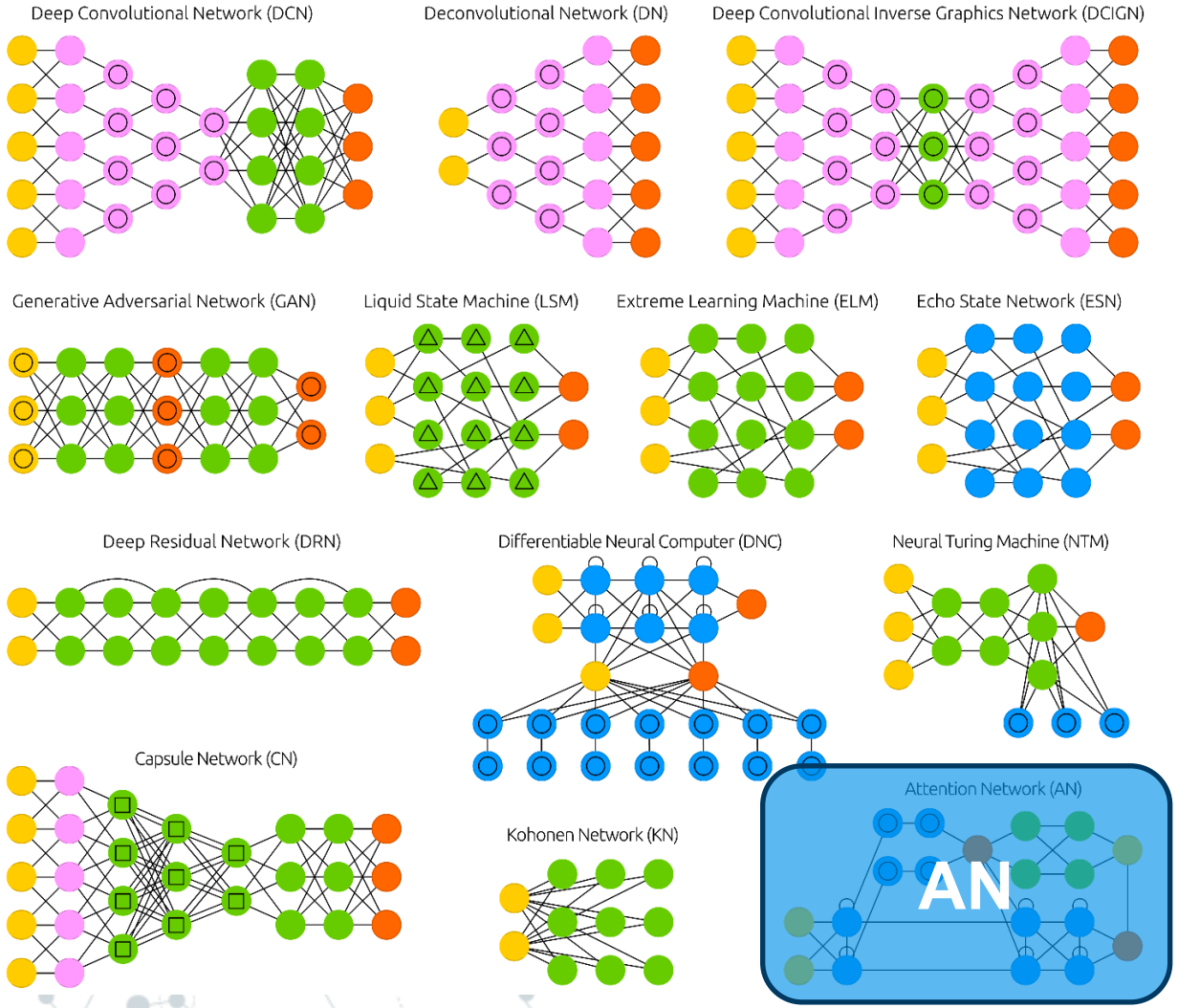
Arquiteturas de Redes Neurais Artificiais



- **Generative Adversarial Networks (GANs)** ou **Redes Adversárias Generativas**.
- São um tipo especial de rede neural que consiste em **duas redes, uma competindo com a outra.**
- Estas duas redes são chamadas de **Gerador e Discriminador.**
  - O **gerador** **cria dados sintéticos** que se assemelham aos dados reais do conjunto de treinamento.
  - O **discriminador** tenta **distinguir entre dados reais e sintéticos**.
  - O **objetivo** do treinamento é fazer com que o **gerador engane o discriminador**.
- São usadas para **geração de imagens**, **vídeos** e **sons** realistas.

Deep Learning (Aprendizado Profundo)

Arquiteturas de Redes Neurais Artificiais

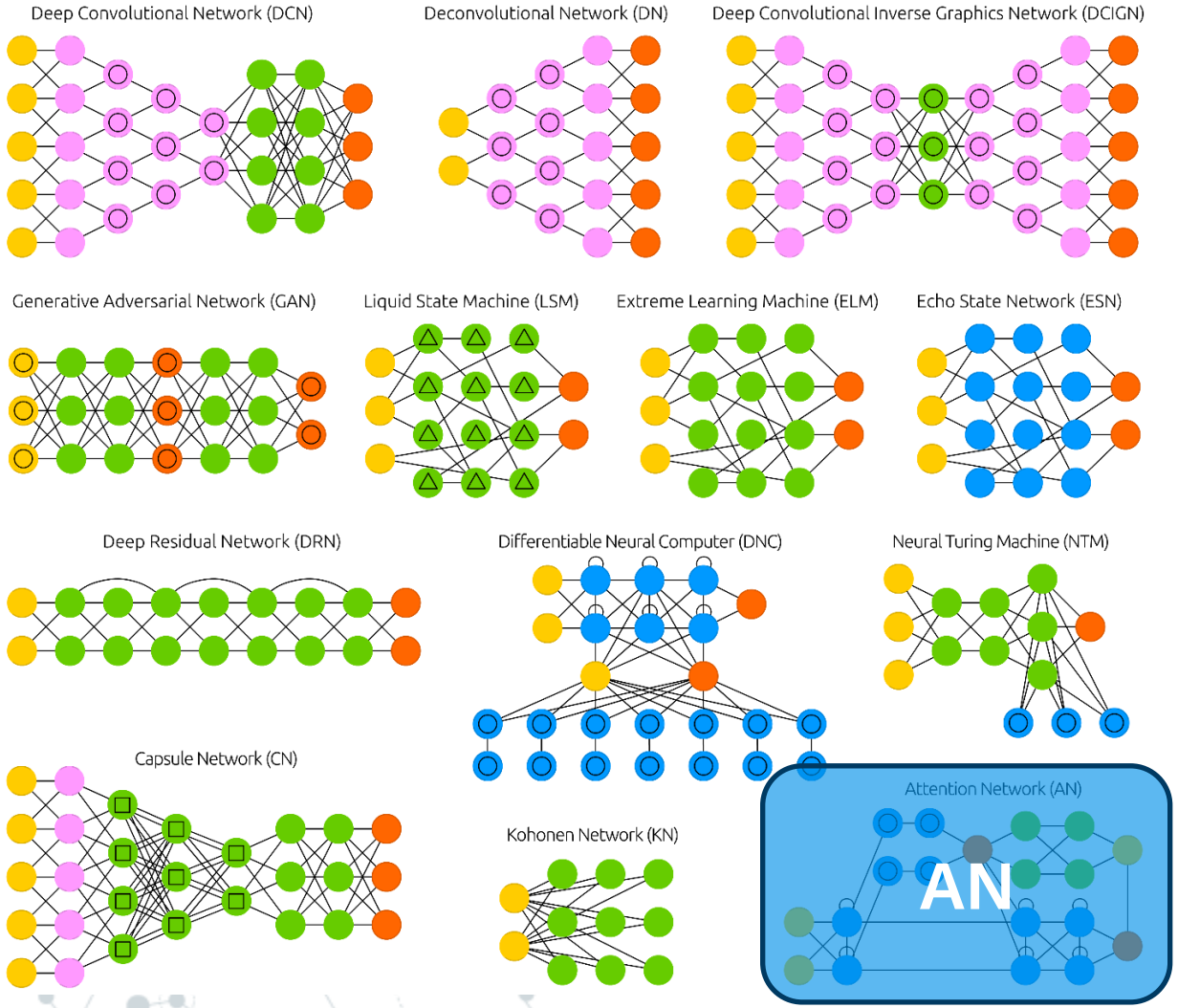


https://www.asimovinstitute.org/neural-network-

- **Attention Networks (ANs)** ou **Redes de Atenção**.
  - São redes que **focam em partes importantes** dos dados de entrada, dando-lhes **maior peso durante** (i.e., **atenção**) o processamento.
  - Por exemplo, ao ler um texto ou visualizar uma imagem, seu **cérebro foca nas partes mais relevantes para entender o contexto**.
  - O mecanismo de atenção faz algo similar.
- Elas **atribuem pesos** às diferentes **partes da entrada** (palavras ou regiões de uma imagem) e **dão mais importância às partes que ajudam a resolver** a tarefa em questão.
- Propostas para lidar com a **dificuldade de RNNs** de lidar com **sequências longas** ou **entradas complexas**.

Deep Learning (Aprendizado Profundo)

Arquiteturas de Redes Neurais Artificiais



- São usadas em tarefas que envolvem sequências de dados, como
  - ***Tradução automática***,
  - ***Processamento de linguagem natural***,
  - ***Reconhecimento de fala*** e
  - ***Visão computacional***.

https://www.asimovinstitute.org/neural-network-

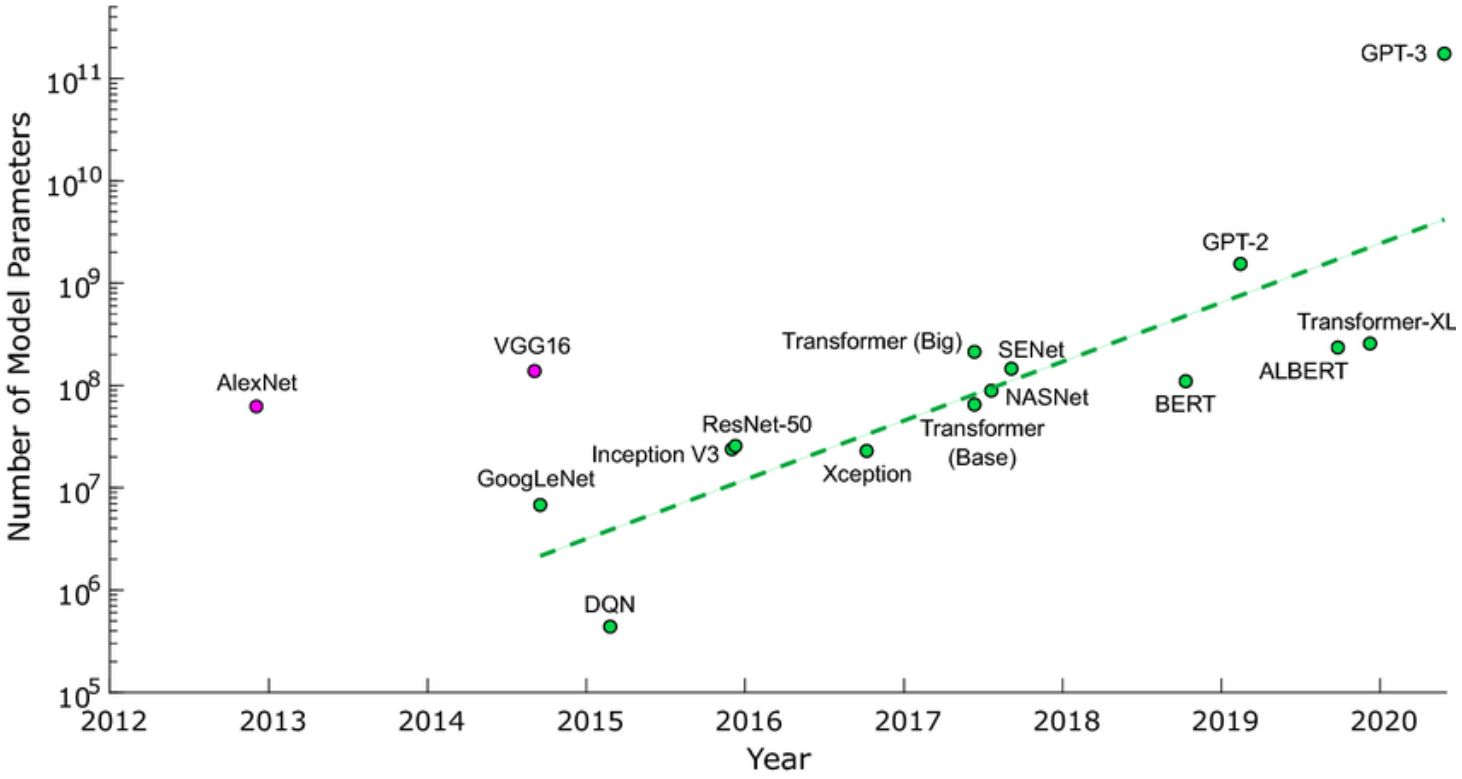
Crescimento do Tamanho dos Modelos





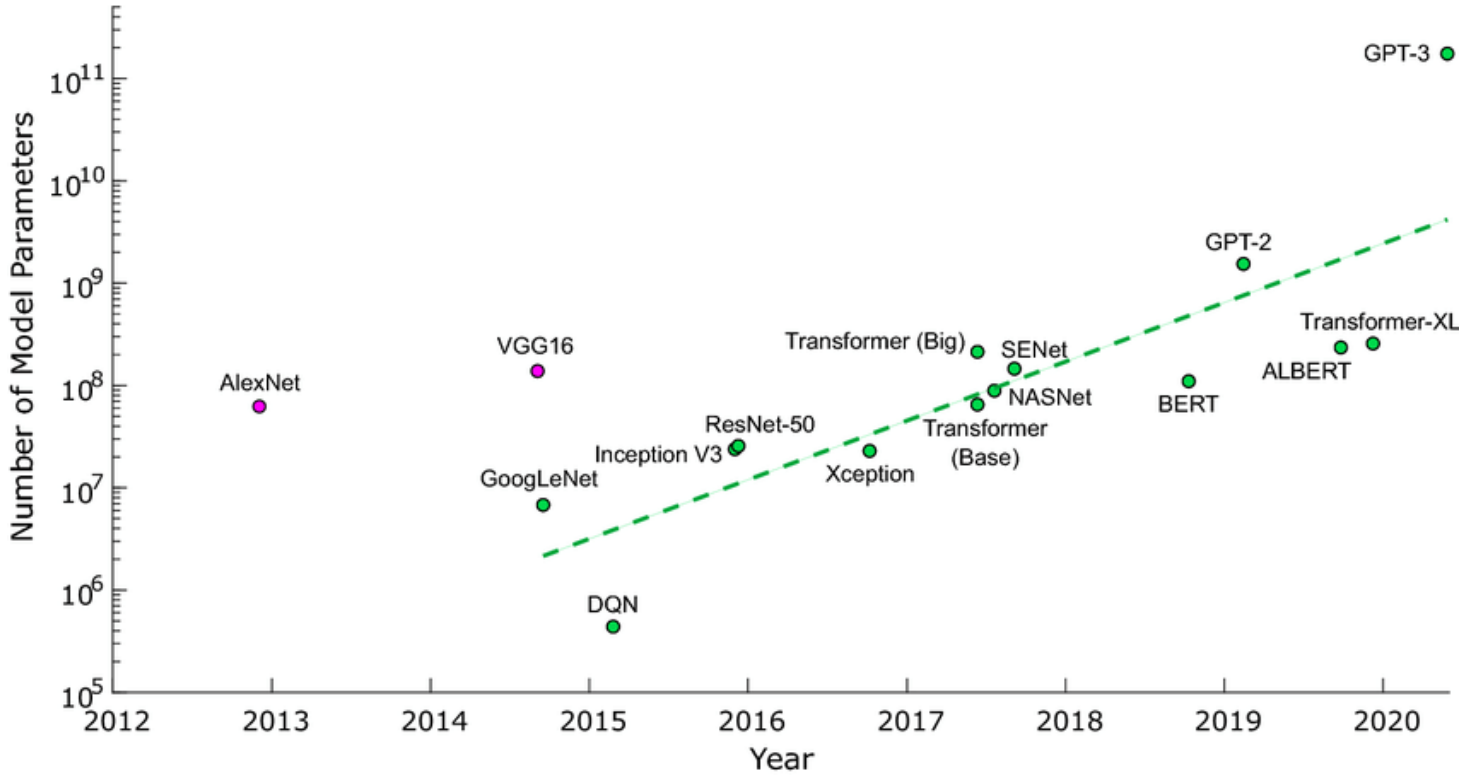
- Ao longo dos anos, para resolver **problemas mais e mais complexos** com **ótimos resultados**, as **arquiteturas** das redes neurais têm se **tornado maiores e mais complexas**.
- **OBS**.: A **capacidade** de uma rede neural está **relacionada** com sua **quantidade de camadas e nós** (i.e., **conexões**).
- A **complexidade computacional** de uma rede neural é **diretamente proporcional ao seu número de conexões.**
- Portanto, **quanto mais camadas e nós**, **mais memória e cálculos matemáticos** são necessários e, **consequentemente**, **maior será o consumo de energia**.
- Até recentemente, **não havia** uma **preocupação com as eficiências computacional e energética** de soluções envolvendo IA.

Crescimento do Tamanho dos Modelos



- **Os Modelos Não Param de Crescer!**
- Isso se deve ao
  - Aumento da **disponibilidade de dados,**
  - Aumento do **poder computacional**
  - Redução do **preço do hardware** e
  - Desenvolvimento de **novas técnicas de aprendizado** (principalmente de aprendizado profundo).

Crescimento do Tamanho dos Modelos

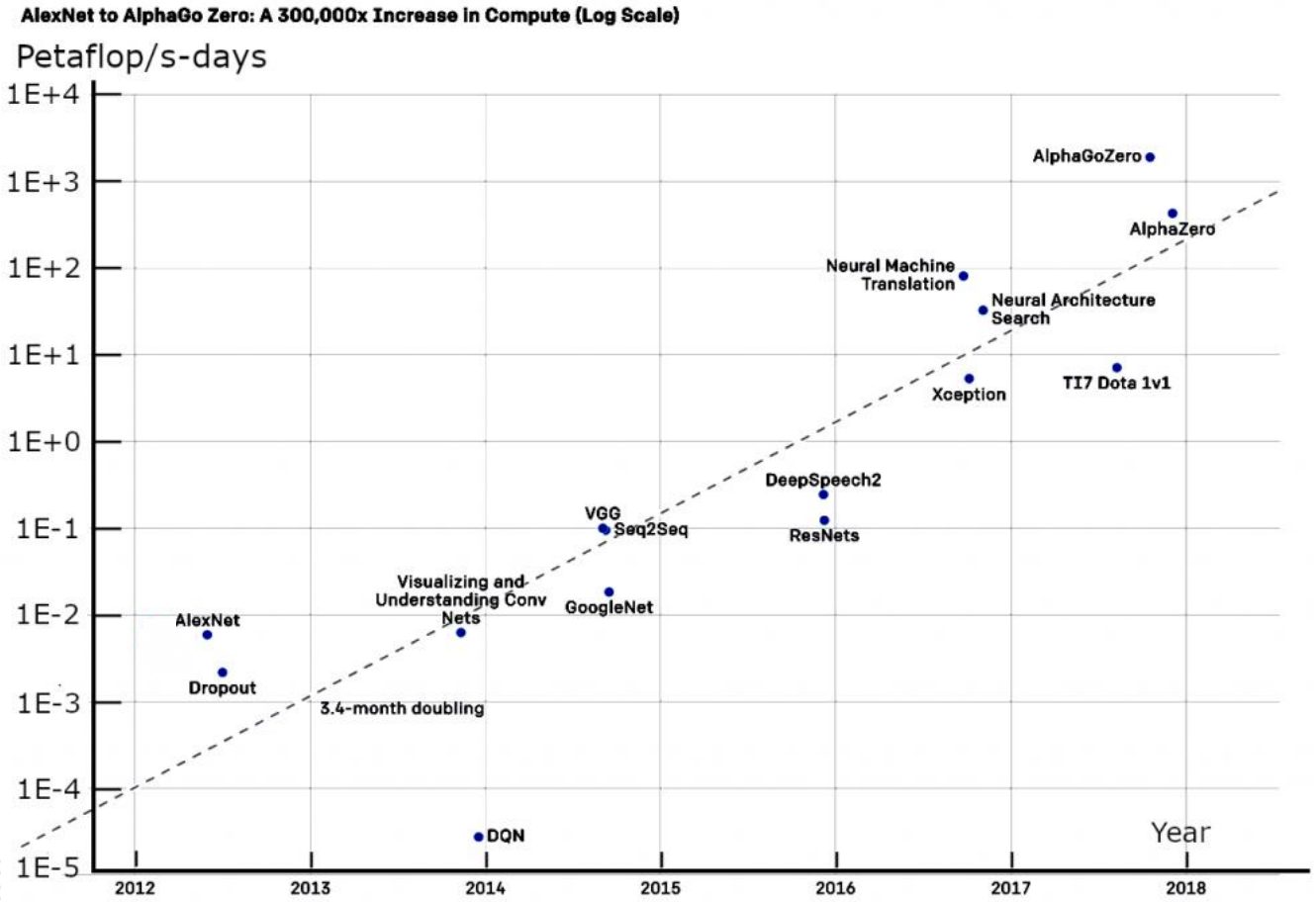


- Vejamos o modelo de linguagem GPT:
  - O GPT-2 tinha aproximadamente **1.5 bilhão de parâmetros** (i.e., pesos sinápticos e de bias).
  - Já os GPT-3/3.5 têm aproximadamente **175 bilhões de parâmetros**.
  - E estima-se que o GPT-4 tenha **1.76 trilhão de parâmetros.**

Se cada parâmetro corresponde a uma variável do tipo *float* (4 bytes), precisamos de $1.76 \times 10^{12} \times 4 \sim 7 \times 10^{12}$ bytes, ou seja, **7 Terabytes** para armazenar o modelo GPT-4!

Crescimento do Tamanho dos Modelos

Necessidades Computacionais (2012 - Atual)

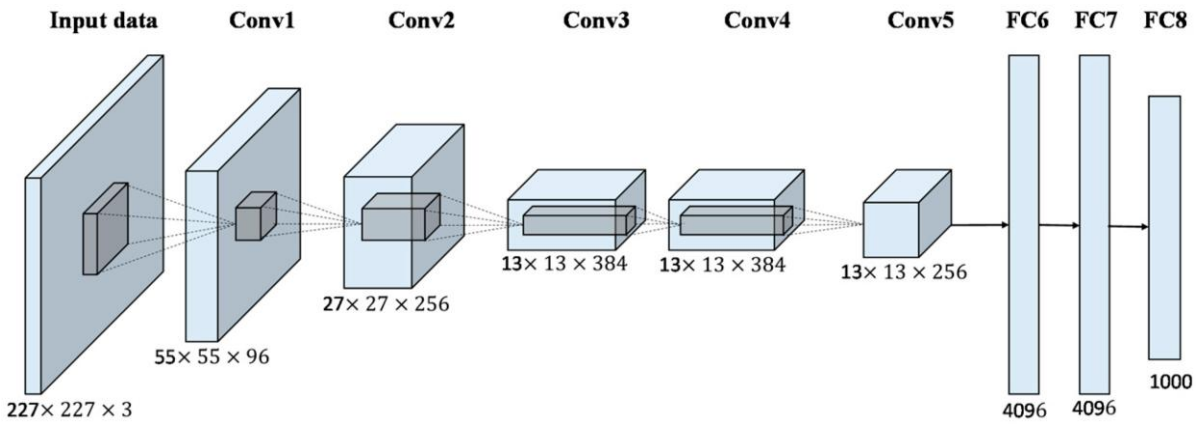


Flops/s-day: quantidade de operações em ponto flutuante por segundo por dia necessárias para treinar o modelo.

- Nos últimos anos, o **poder computacional** necessário para treinar os modelos de ML amplamente usados hoje **teve que crescer 300.000 vezes**.
- Essa tendência de redes imensas começou em 2012 com a AlexNet.
  - Rede Neural Convolucional desenvolvida na universidade de Toronto no Canadá para classificação de imagens.

Crescimento do Tamanho dos Modelos

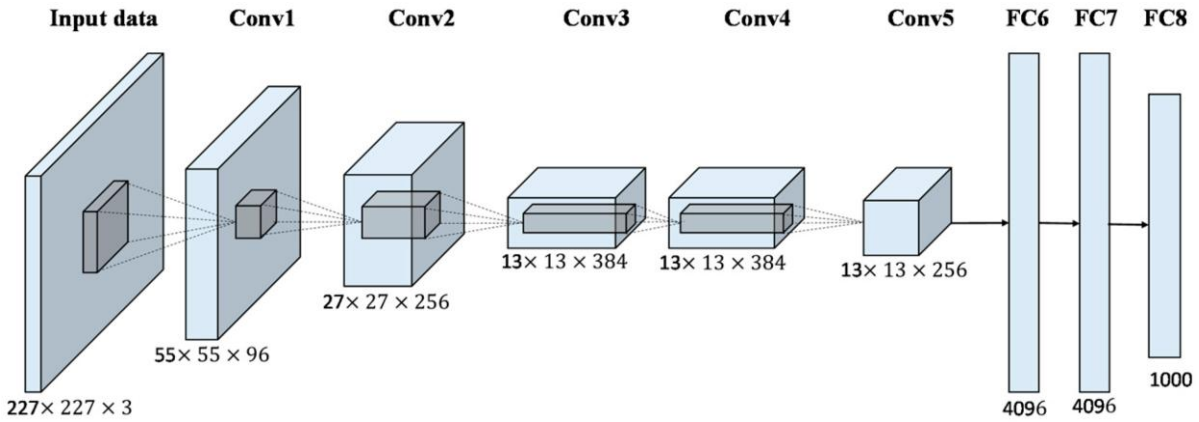
Necessidades Computacionais (2012 - Atual)



- **CNN** que **bateu** com folga todos os **recordes anteriores** do desafio ImageNet Large Scale Visual Recognition de 2012.
- Tem **60 milhões de parâmetros**, o que na época a tornou uma das **maiores e mais complexas redes neurais**.
- **Popularizou** o uso de **camadas convolucionais**, impulsionando a revolução do aprendizado profundo.

Crescimento do Tamanho dos Modelos

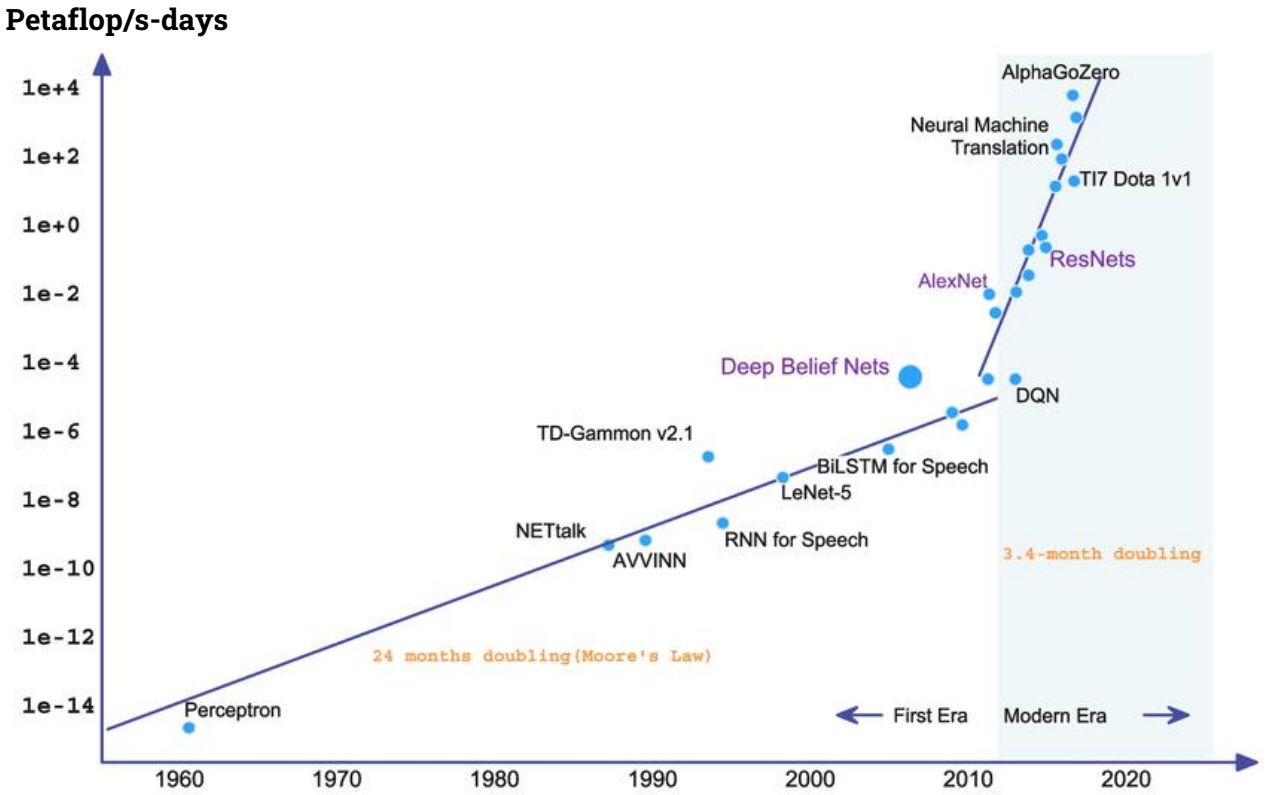
Necessidades Computacionais (2012 - Atual)



- Introduziu o uso da **função de ativação ReLU**, que mitiga o problema do desaparecimento do gradiente.
- Foi uma das **primeiras redes neurais a usar GPUs** para reduzir o tempo de treinamento.
- Estabeleceu a **base para muitas arquiteturas de CNNs** subsequentes.

Crescimento do Tamanho dos Modelos

Necessidades Computacionais (Desde 1958)



- Nos últimos 10 anos, a quantidade de cálculos necessários cresceu extraordinariamente rápido.
- Na "primeira era" do ML, a quantidade de cálculos e, consequentemente, o **tamanho dos modelos**, **dobrava a cada dois anos** aproximadamente.
- Na "era moderna", os requisitos de computação praticamente dobram a cada **3/4 meses**.

Crescimento do Tamanho dos Modelos

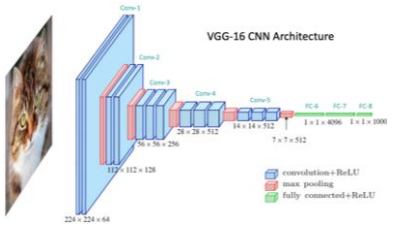
Necessidades Computacionais (Desde 1958)

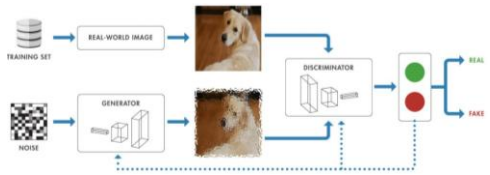












TensorFlow

PyTorch

Google Cloud

aws

Microsoft Azure

→ O que aconteceu nesses últimos 10 anos que explica esse Boom?

Crescimento do Tamanho dos Modelos

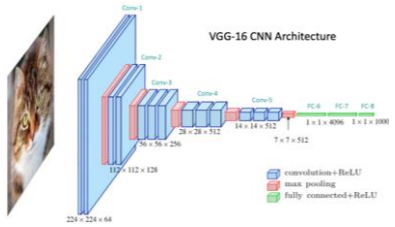
Necessidades Computacionais (Desde 1958)

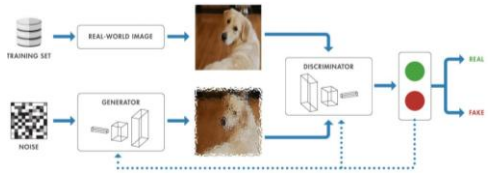












TensorFlow

PyTorch

Google Cloud

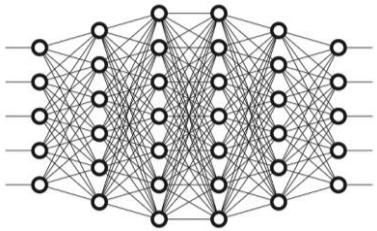
aws

Microsoft Azure

- Disponibilidade de **grandes volumes de dados** devido à internet (> 100 TB/dia).
- **Aumento do poder de computacional** através de GPUs, FPGAs, ASICs especializdos e CPUs com múltiplos cores.
- **Surgimento de novos algoritmos de ML**, como redes neurais profundas, redes adversárias generativas (GANs), redes de atenção, transformers, deep reinforment-learning, etc.
- Disponibilidade de **frameworks e bibliotecas amigáveis**, como TensorFlow e PyTorch, que facilitam o desenvolvimento de soluções com ML.
- O surgimento de **serviços de computação em nuvem** ofereceu **acesso econômico e escalável** a recursos de computação.

Crescimento do Tamanho dos Modelos

Consequências do Aumento da Capacidade







Cloud TPU



TinyML



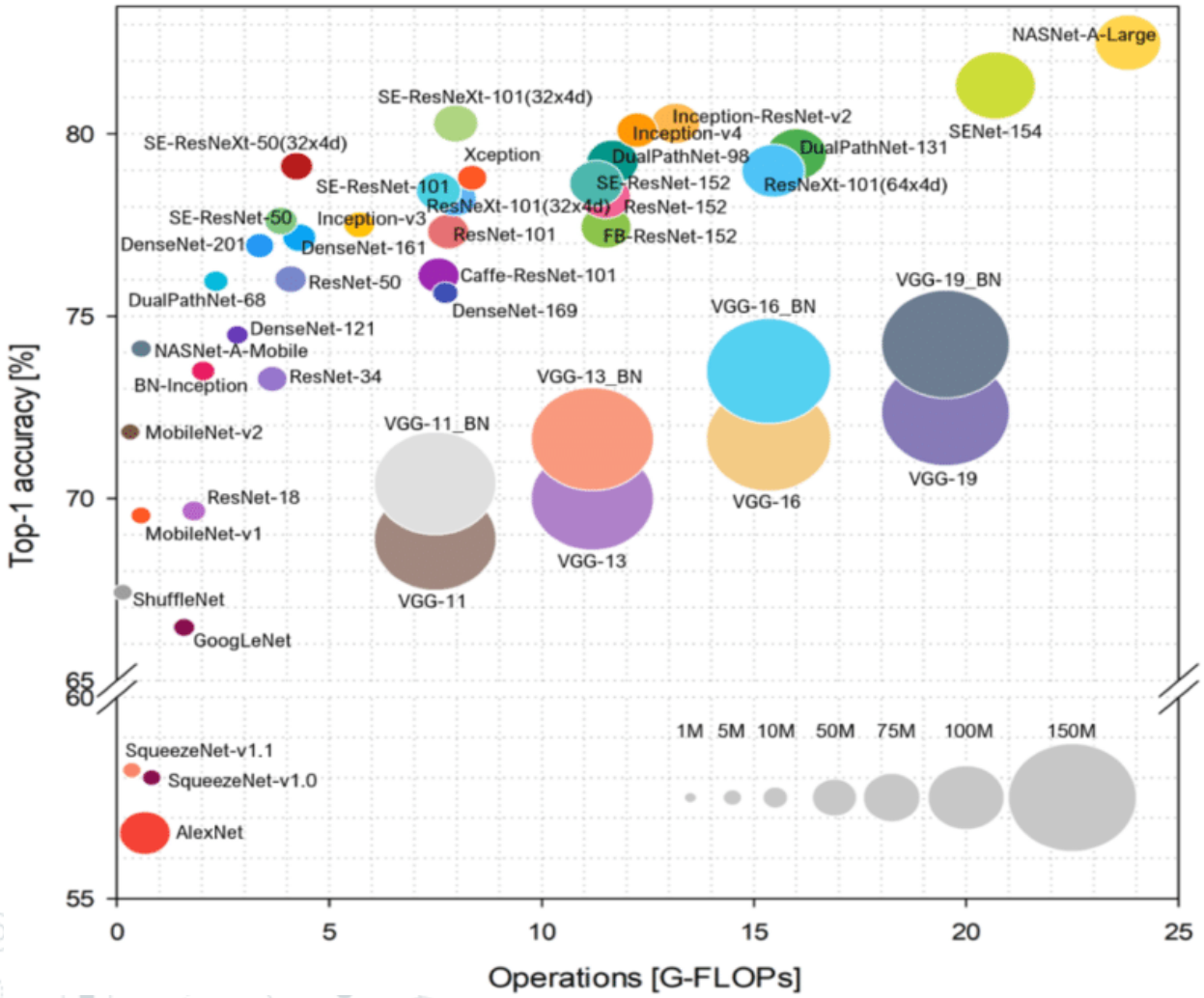
?

- Entretanto, esses **modelos mais e mais complexos**, **necessitam**, consequentemente, de mais
  - **capacidade de armazenamento**,
  - **energia**,
  - **poder de processamento e**
- que acabam sendo **muito caros** e **ocupando grandes espaços**.

TPU: Tensor Processing Unit, é um ASIC **altamente especializado para realizar operações tensoriais (ou arrays)**, o que acelera o treinamento e inferência de modelos de aprendizado de máquina, especialmente aqueles que usam o TensorFlow.

Como Colocar Tudo em Dispositivos EdgeML?

Evolução dos Modelos de ML

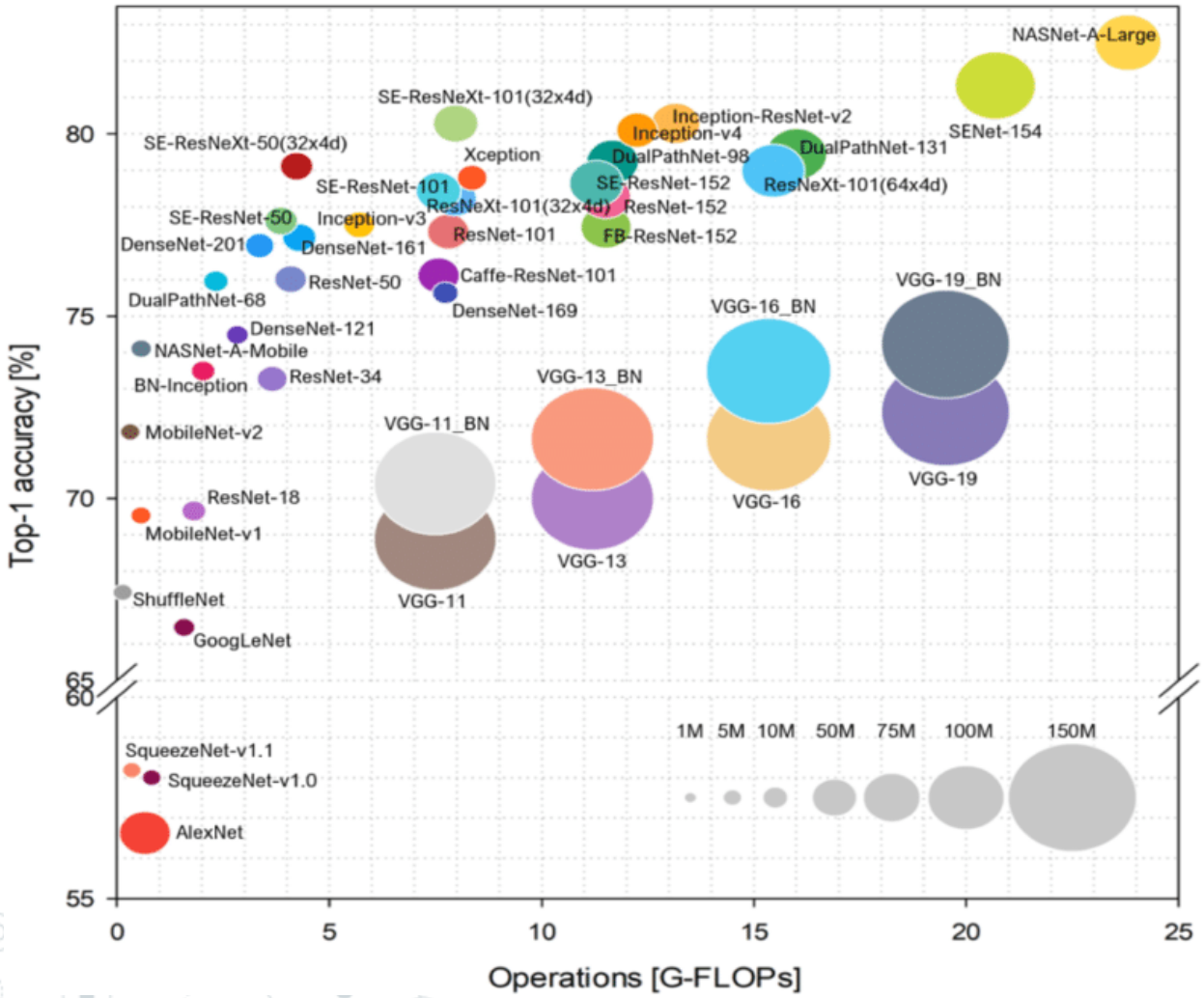


- A figura ao lado mostra a **evolução dos modelos** de ML para a tarefa de **classificação de imagens** usando o conjunto de dados ImageNet-1K (1000 classes) como benchmark.
- O eixo x mostra o **custo computacional** do modelo (i.e., quantidade de flops necessárias para uma classificação).
- O eixo y mostra a **acurácia** (taxa de acertos).
- O tamanho de cada círculo corresponde à **complexidade** (i.e., quantidade de parâmetros) do modelo.

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML

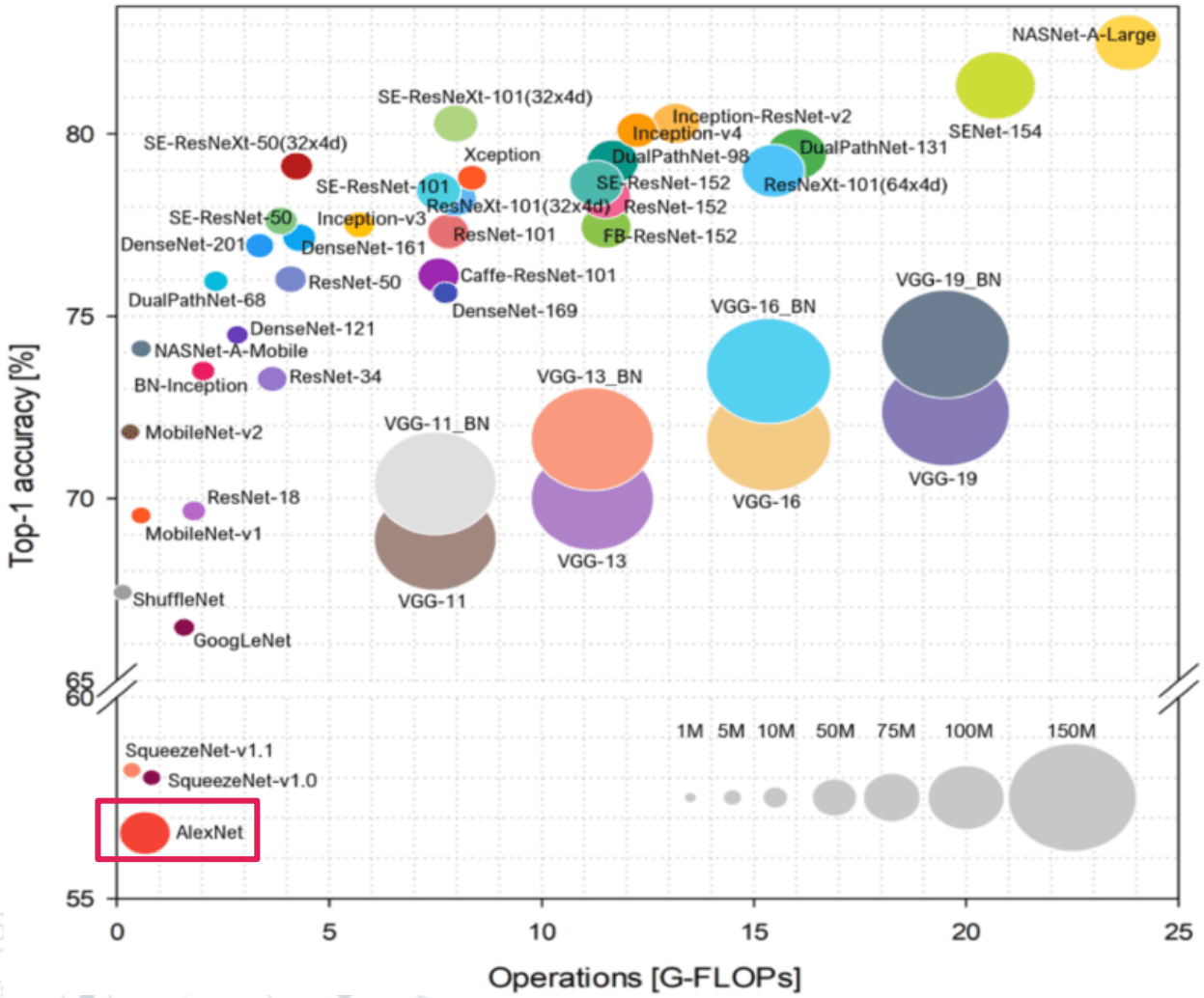


- Para execução em dispositivos **EdgeML**, devemos procurar por modelos com:
  - **Poucos parâmetros** (i.e., círculos pequenos), pois a quantidade de parâmetros está associada à **quantidade de memória demandada pelo modelo**.
  - **Quantidade necessária de flops baixa**, pois a quantidade de operações está diretamente **relacionada ao consumo de energia**.
  - **Alta acurácia**.

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



AlexNet (Universidade de Toronto, Canadá - 2012)

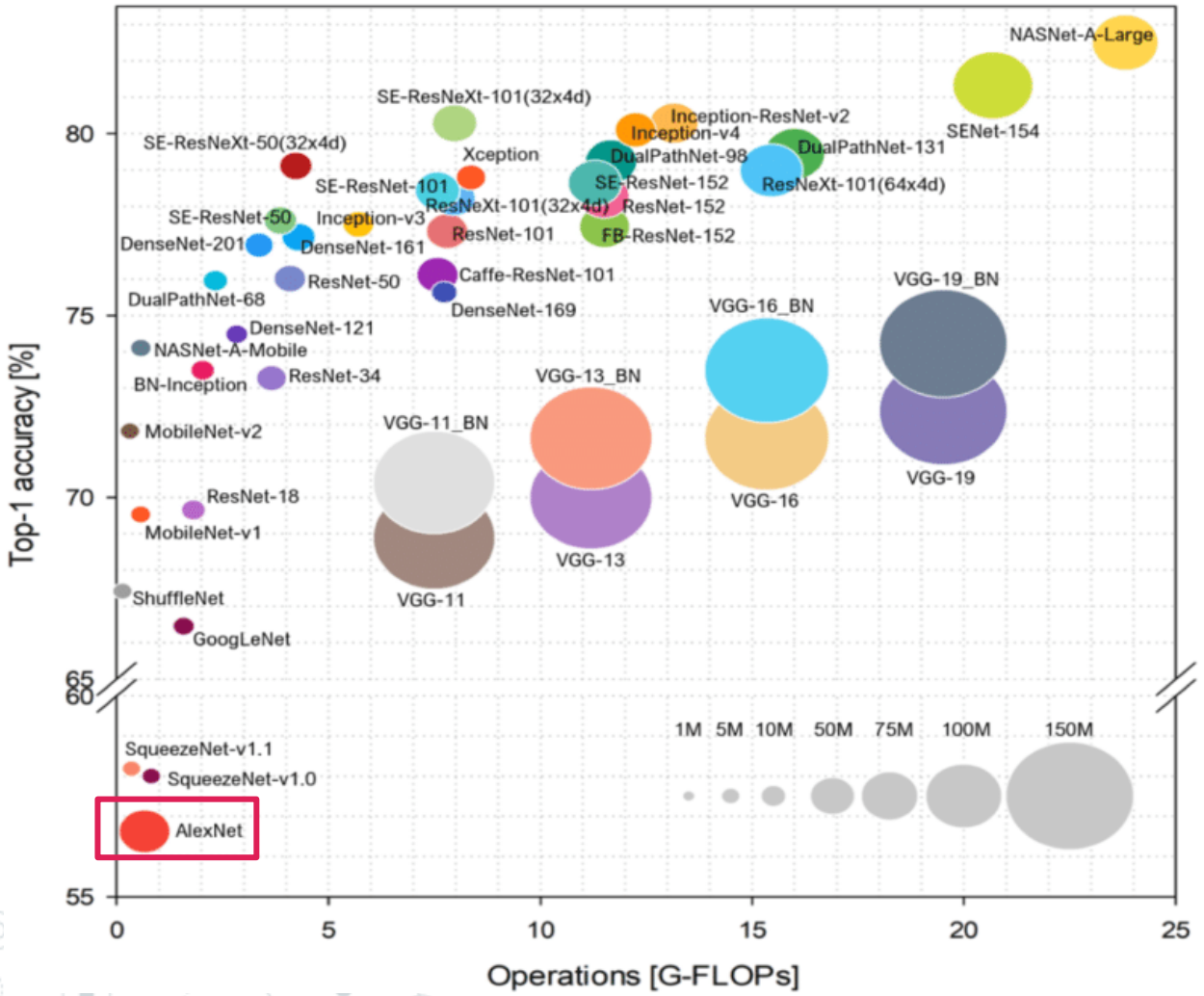
- **Acurácia:** 57.1%
- **Tamanho:** 61 MB

→ **Camadas**: 5 convolucionais e 3 densas.

FLOPs: operações em ponto flutuante necessárias para inferência.

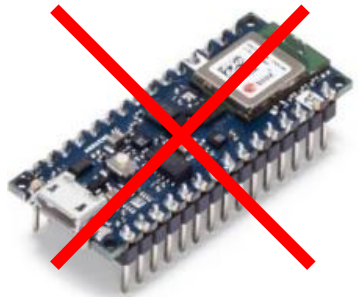
Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



AlexNet (Universidade de Toronto, Canadá - 2012)

- **Acurácia:** 57.1%
- **Tamanho:** 61 MB



RAM: 256 KB
Flash: 1 MB

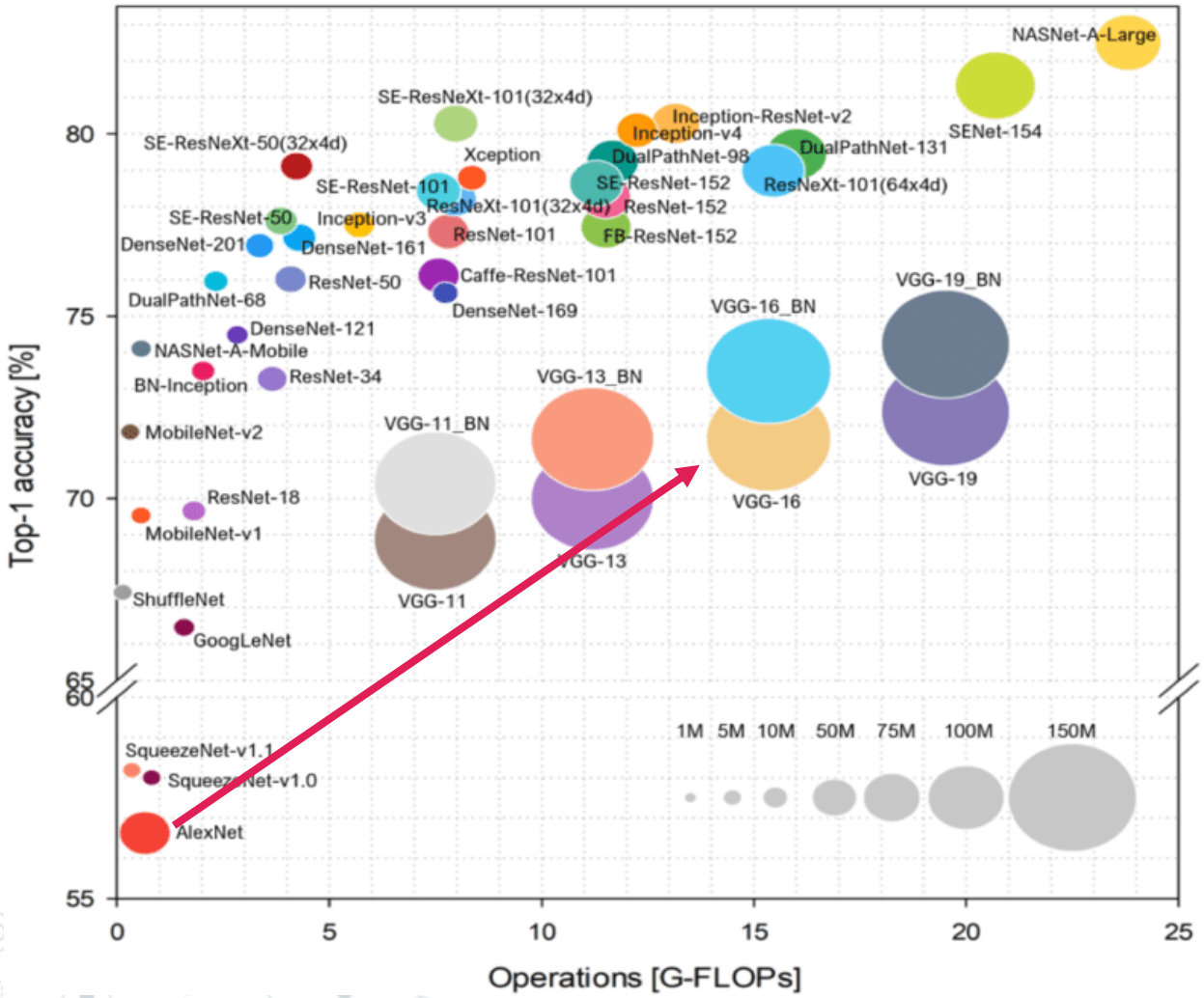


RAM: até 16 GB
Flash: 64 a 256 GB

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



VGG-16 (Universidade de Oxford, Reino Unido - 2014)

- **Acurácia:** 71.5%
- **Tamanho:** 528 MB

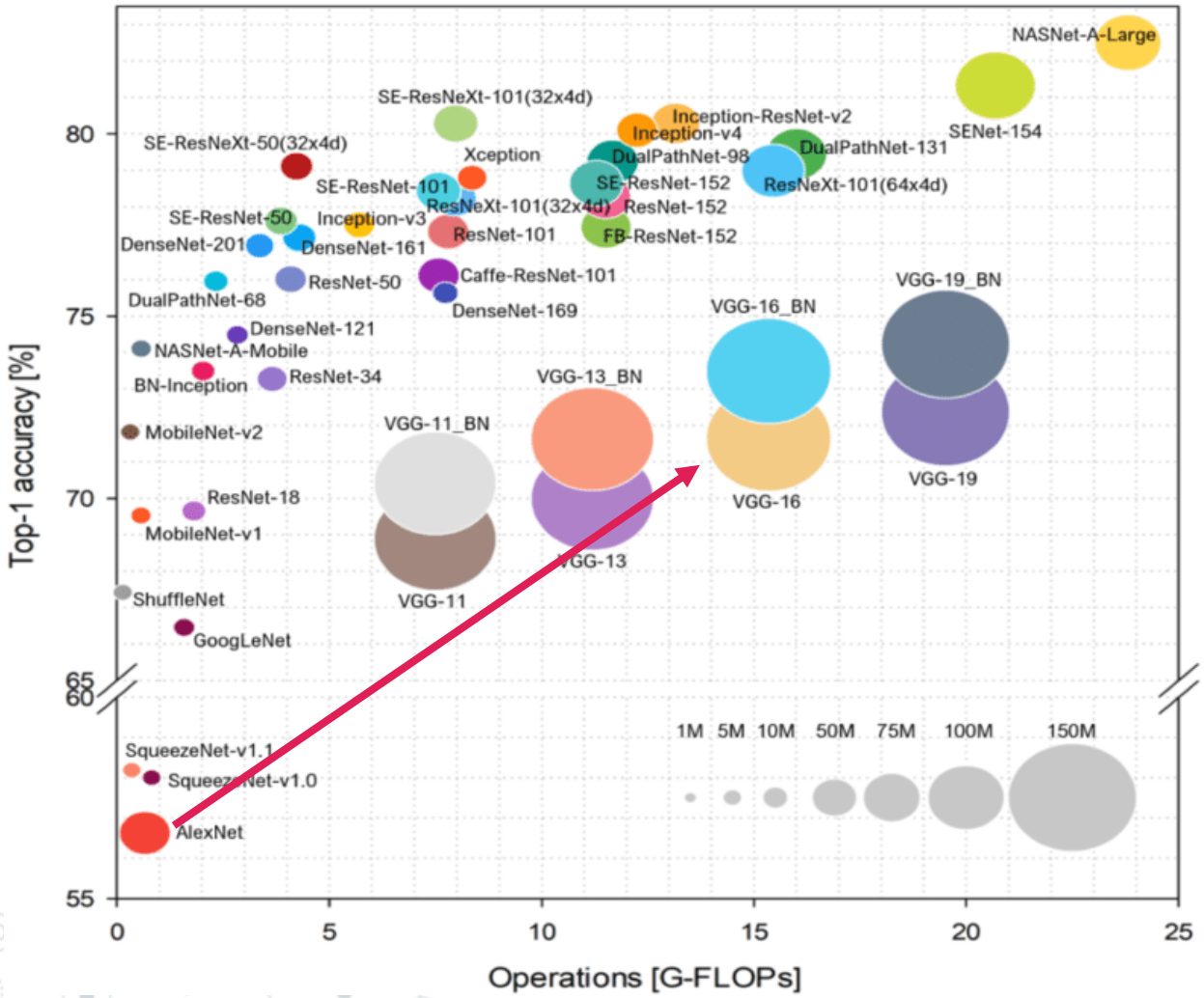
→ **Camadas:** 13 convolucionais e 3 densas.

→ Em dois anos, saímos de um modelo pequeno, requerendo poucos *flops* (baixo consumo), mas com baixa acurácia para modelos mais precisos, porém mais de 8 vezes maiores e requerendo 15 vezes mais *flops* (alto consumo).

FLOPs: operações em ponto flutuante necessárias para inferência.

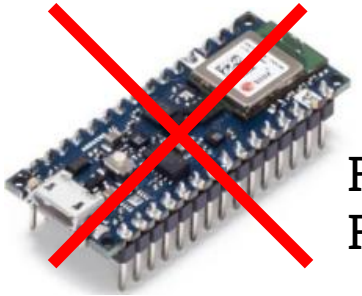
Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



VGG-16 (Universidade de Oxford, Reino Unido - 2014)

- **Acurácia:** 71.5%
- **Tamanho:** 528 MB



RAM: 256 KB
Flash: 1 MB

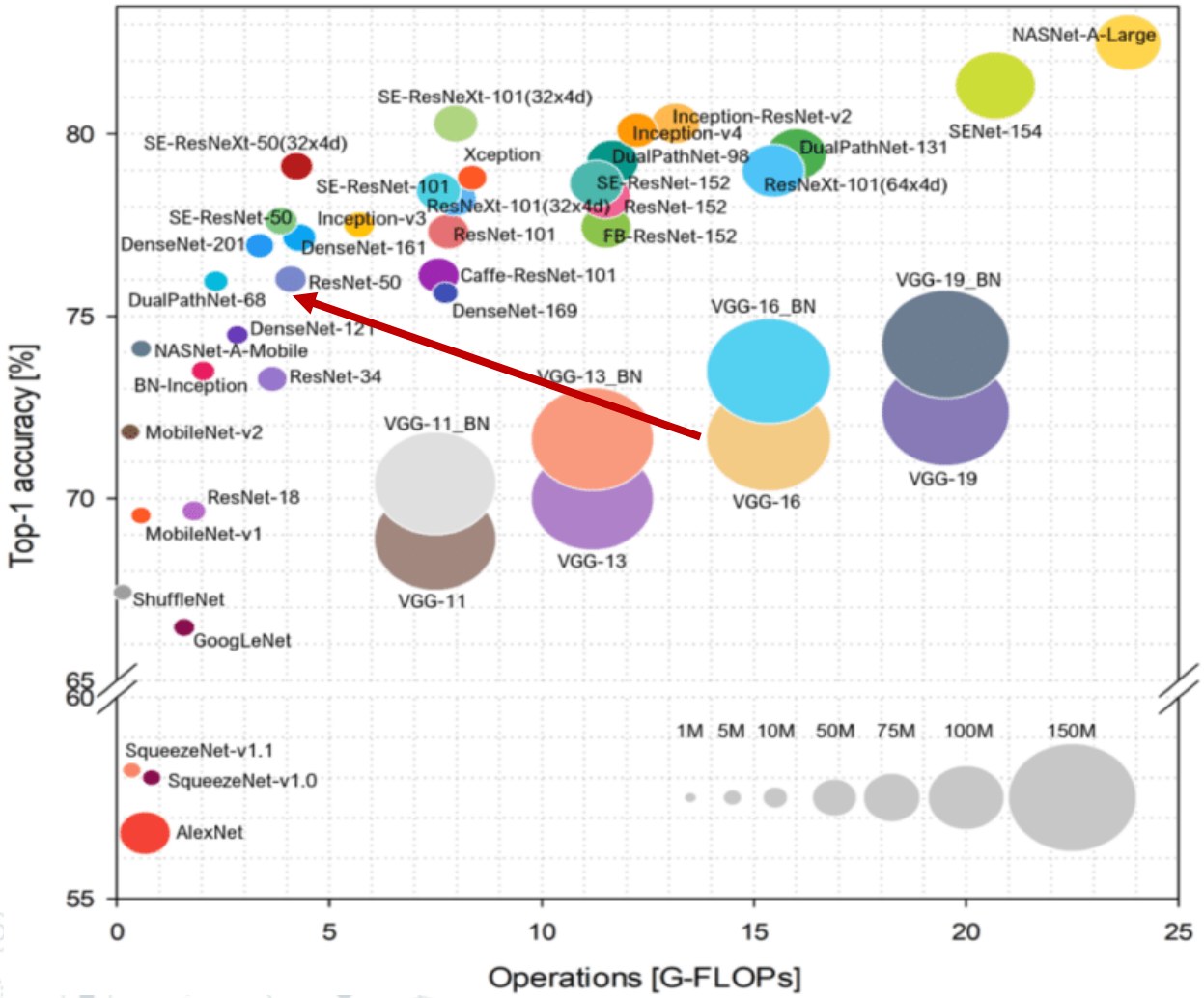


RAM: até 16 GB
Flash: 64 a 256 GB

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



ResNet-50 (Microsoft - 2015)

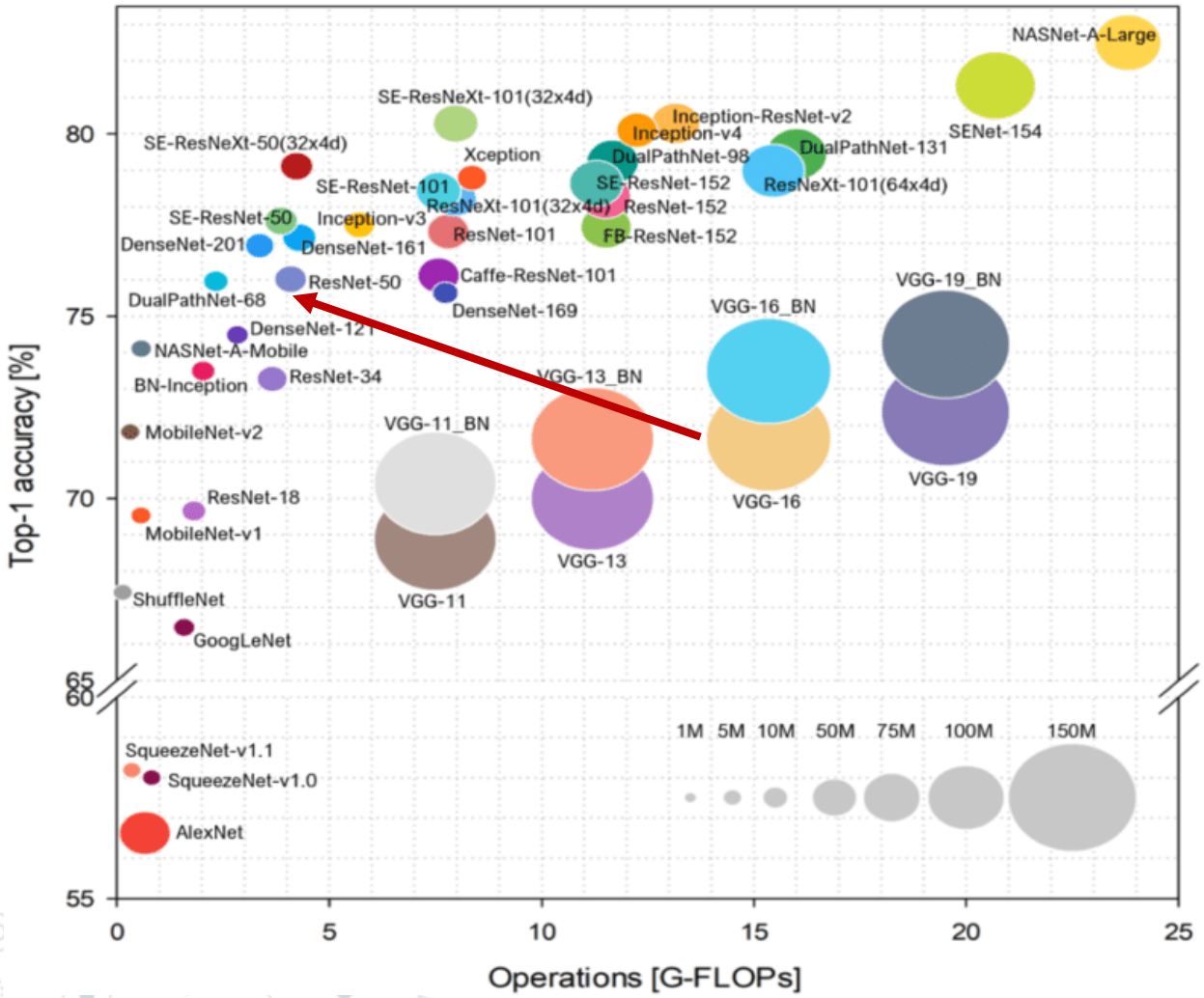
- **Acurácia:** 75.8%
- **Tamanho:** 22.7 MB

→ **Camadas:** 48 convolucionais, 2 de *pooling* (agrupamento) e 1 densa.

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



ResNet-50 (Microsoft - 2015)

- **Acurácia:** 75.8%
- **Tamanho:** 22.7 MB

→ Arquitetura muito mais eficiente do que a VGGNet pois introduz

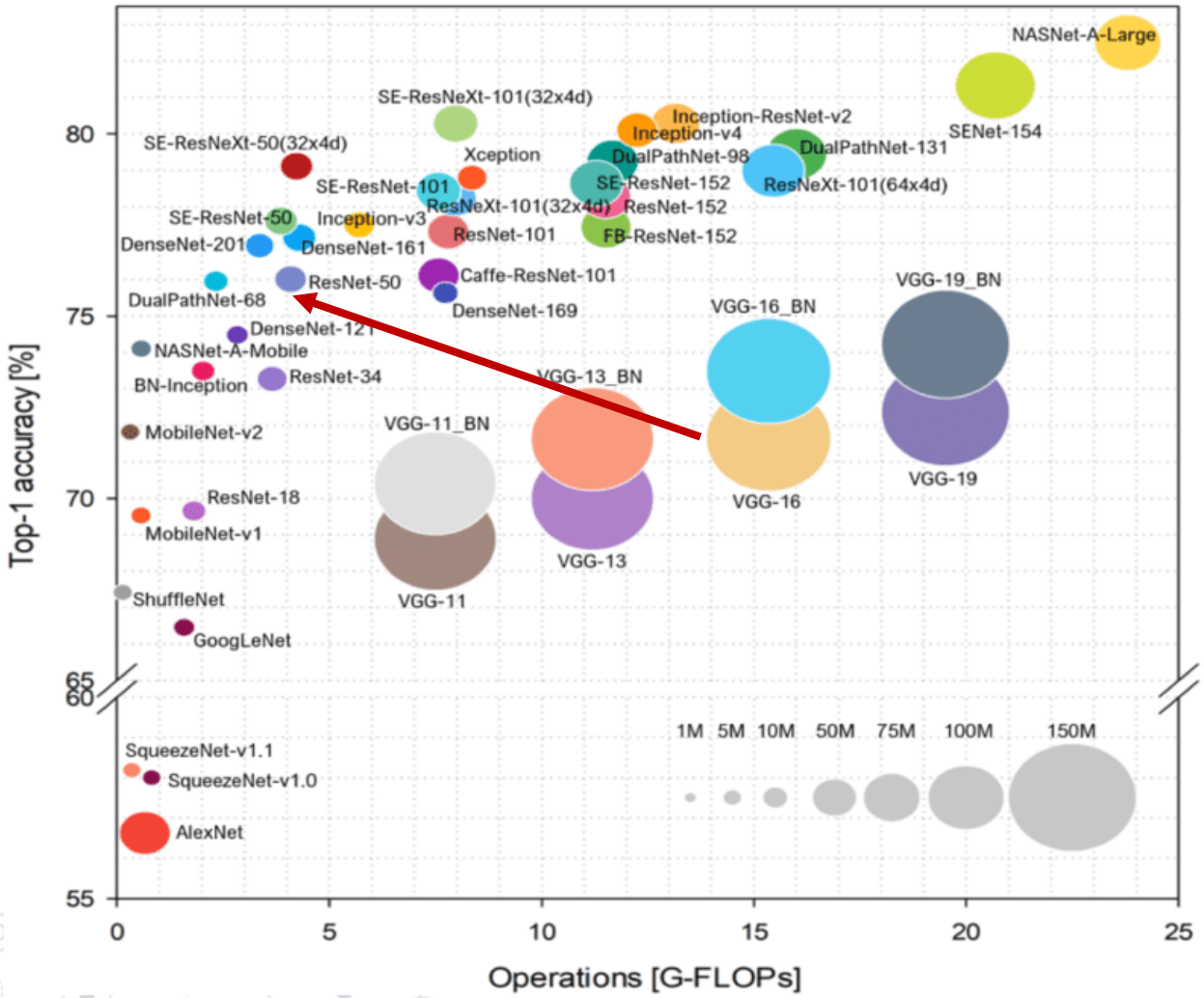
- **Dropout**: desliga aleatoriamente algumas das conexões da rede durante o treinamento para **evitar o sobreajuste**.
- **Blocos residuais**: são atalhos entre camadas convolucionais que **mitigam o problema do desaparecimento do gradiente**.

→Em **um ano**, saímos de um modelo **extremamente grande** e que **requer muitos flops**, mas razoavelmente preciso para um modelo **23 vezes menor**, **mais preciso** e necessitando de **3 vezes menos flops**.

FLOPs: operações em ponto flutuante necessárias para inferência.

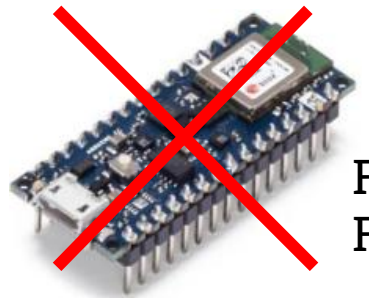
Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



ResNet-50 (Microsoft - 2015)

- **Acurácia:** 75.8%
- **Tamanho:** 22.7 MB



RAM: 256 KB
Flash: 1 MB

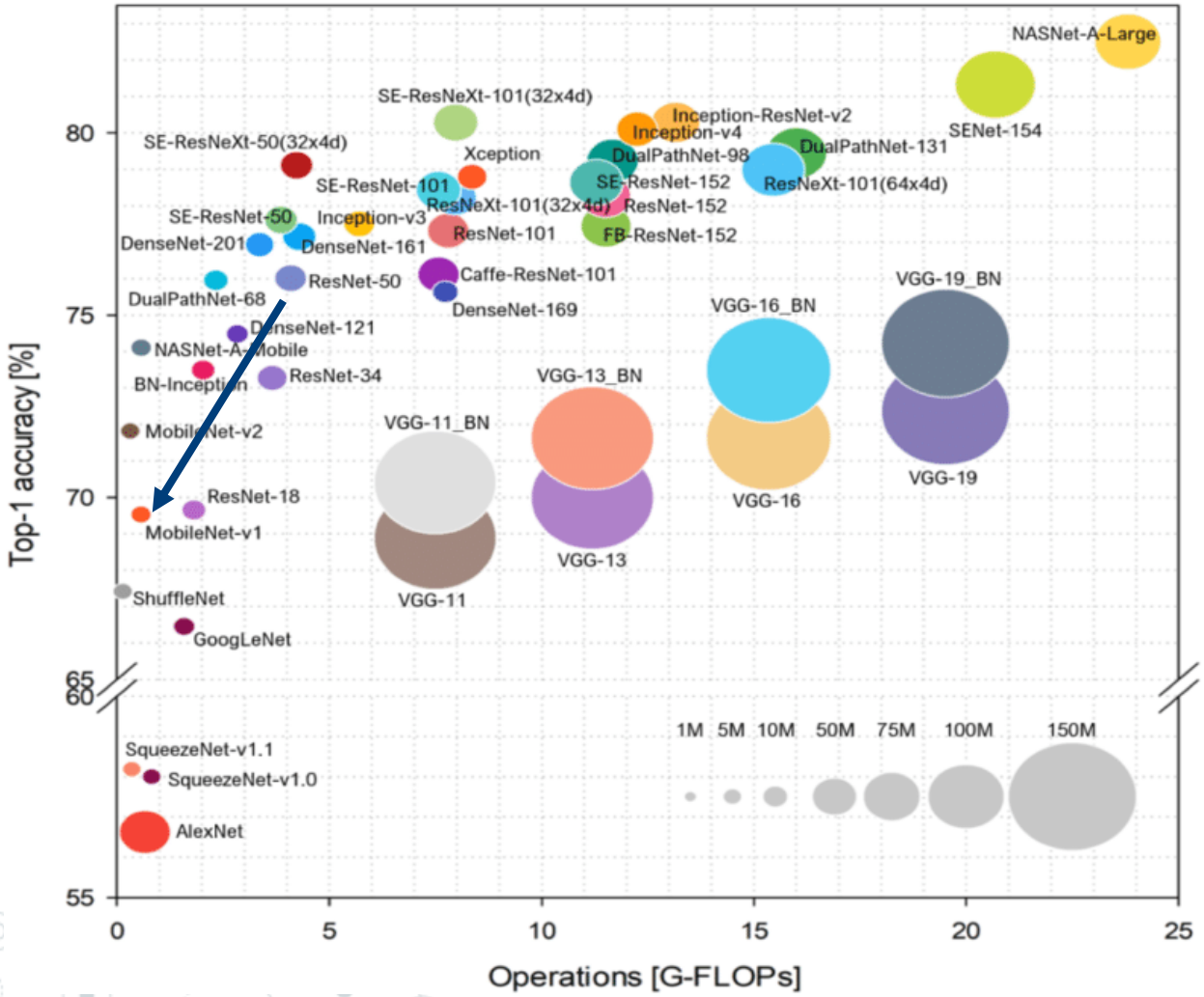


RAM: até 16 GB
Flash: 64 a 256 GB

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



MobileNetv1 (Google - 2015)

- **Acurácia:** 70.6%
- **Tamanho: 16.9 MB**

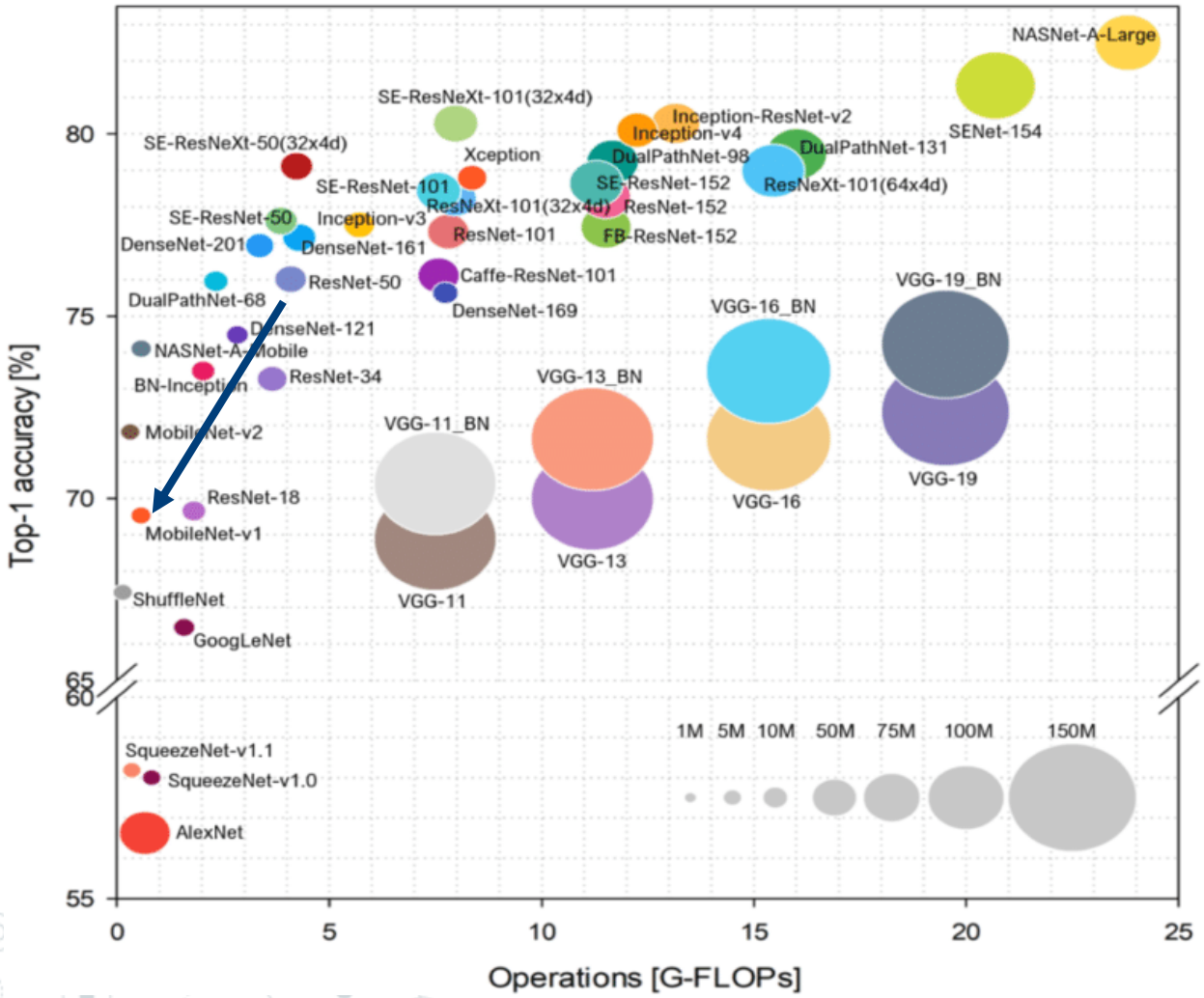
→ **Camadas:** 27 convolucionais e 2 densas.

→ Adequada para aplicações de visão computacional em **dispositivos móveis e embarcados**.

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



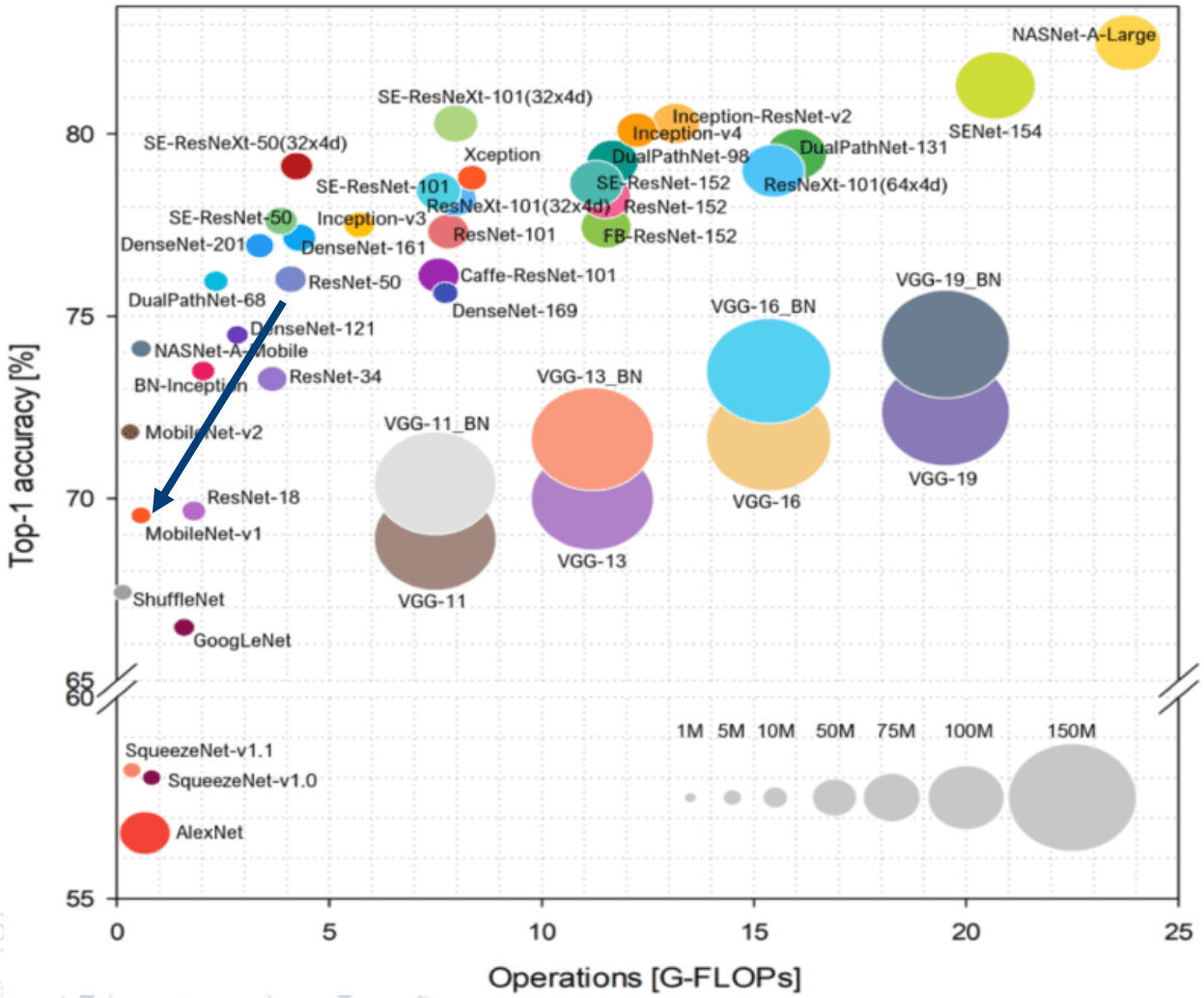
MobileNetv1 (Google - 2015)

- Usa camadas de **convolução separável em profundidade** em vez de convolução tradicional.
- **Divide** a convolução tradicional em **duas etapas**: uma **convolução em profundidade** (que lida com canais) seguida de uma **convolução em ponto** (que lida com pixels individuais).
- Isso **reduz** drasticamente o **tamanho do modelo**, a **quantidade de cálculos** e **melhora sua eficiência computacional**.

FLOPs: operações em ponto flutuante necessárias para inferência.

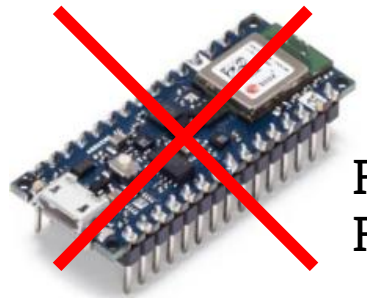
Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



MobileNetv1 (Google - 2015)

- **Acurácia:** 70.6%
- **Tamanho: 16.9 MB**



RAM: 256 KB
Flash: 1 MB

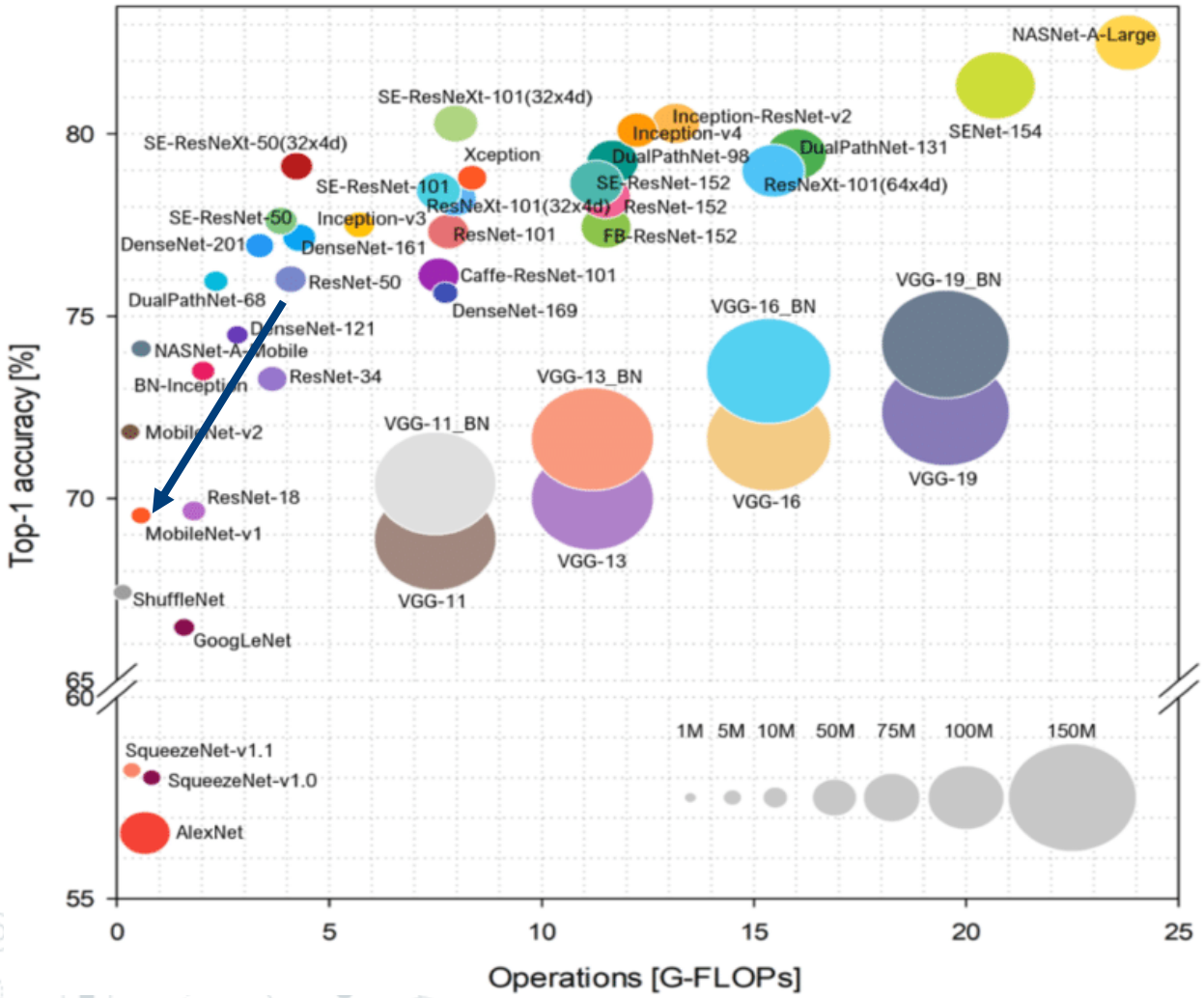


RAM: até 16 GB
Flash: 64 a 256 GB

FLOPs: operações em ponto flutuante necessárias para inferência.

Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Evolução dos Modelos de ML



→ **Os modelos até este ponto ainda eram muito grandes para dispositivos com recursos limitados.**

FLOPs: operações em ponto flutuante necessárias para inferência.

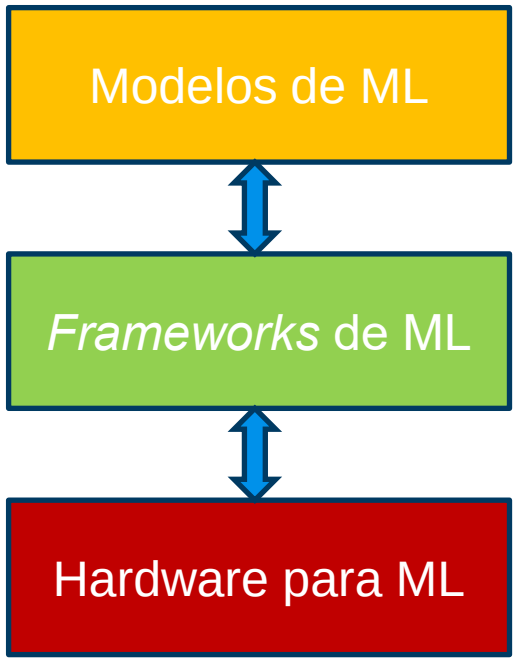
Fonte: Bianco, Simone, et al. "*Benchmark analysis of representative deep neural network architectures*." *IEEE access* 6 (2018): 64270-64277.

Como Executar Modelos de ML em Dispositivos EdgeML?

- Nitidamente, mesmo os menores modelos, **não cabem e/ou não conseguem ser executados de forma eficiente em dispositivos EdgeML,** principalmente os TinyML.
- Portanto, surge uma pergunta.

→ Como podemos **Reduzir** esses Modelos de ML para que **caibam na memória** destes dispositivos, **possam ser executados** por eles e **consumam pouca energia**?

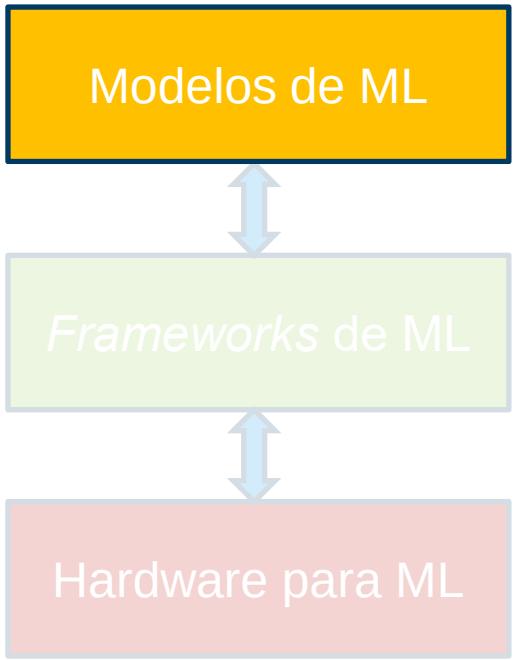
Como Executar Modelos de ML em Dispositivos EdgeML?



→ Podemos lançar mão de três abordagens:

- Compressão dos modelos.
- *Frameworks* mais enxutos e eficientes.
- Hardwares específicos/customizados para ML.

Técnicas de Compressão de Modelos de ML

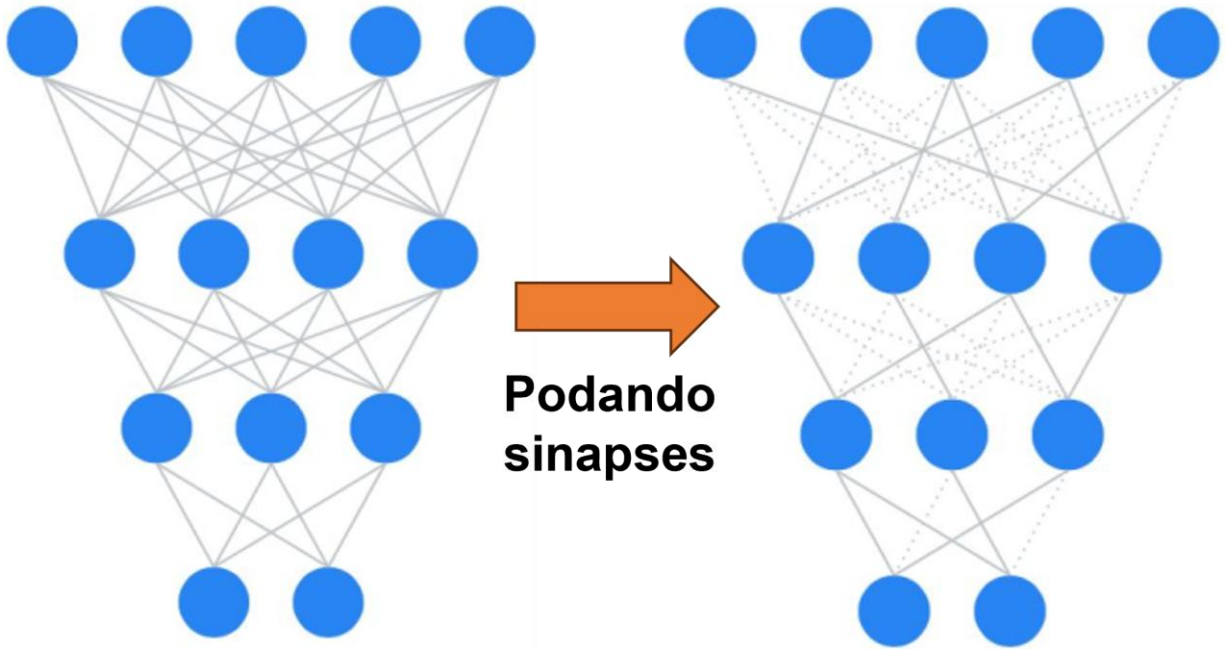


→ Em geral, podemos utilizar quatro técnicas para reduzir o tamanho dos modelos sem perder muito de seu desempenho:

- Pruning (ou poda);
- Quantização;
- Clustering;
- Knowledge Distillation (ou destilação de conhecimento).

Técnicas de Compressão de Modelos de ML

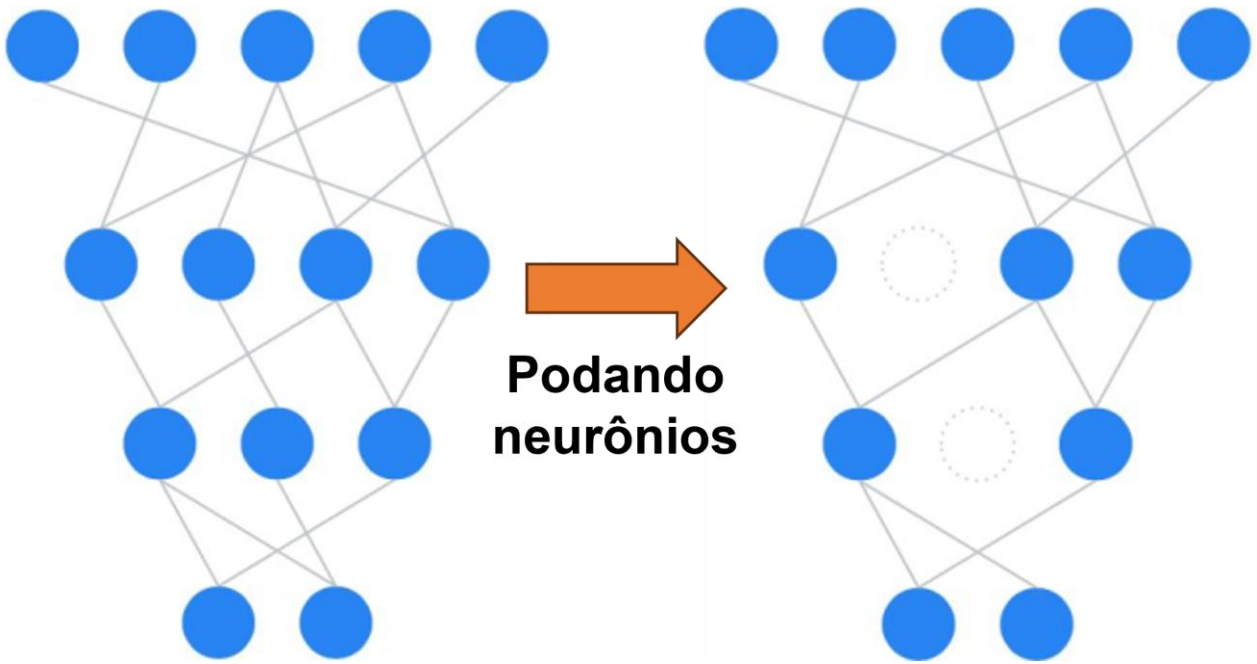
Pruning



- **Remove conexões** para **reduzir** a quantidade de **cálculos** necessários e diminuir o **tamanho do modelo**.
  - Reduz a demanda computacional.
- Essa remoção, em geral, se baseia na **magnitude dos pesos**.
  - **Pesos com magnitudes próximas de zero são removidos**, pois podem não influenciar no desempenho do modelo.
- **Técnica aplicada após o treinamento.**

Técnicas de Compressão de Modelos de ML

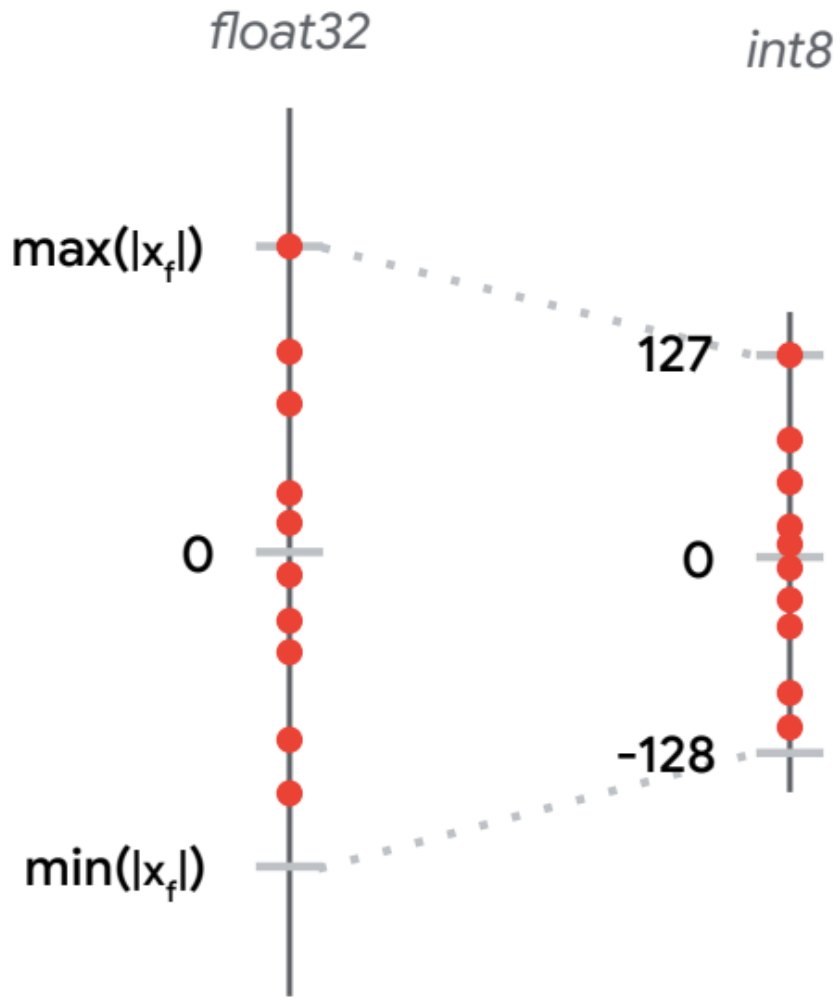
Pruning



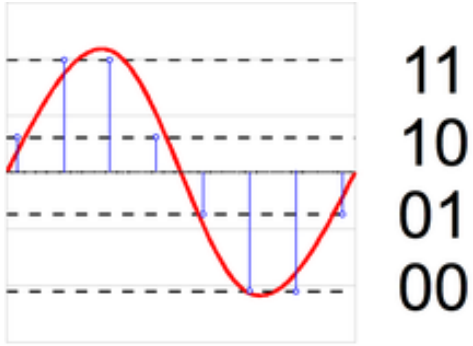
- A poda pode, também, acabar **removendo neurônios completamente**.
- O modelo final **pode** continuar com o **mesmo desempenho ou perder parte dele**.
- Portanto, é preciso **testar e encontrar um balanço** entre redução do modelo e seu desempenho.
- Ajuda a **mitigar problemas de sobreajuste**, melhorando a generalização do modelo.
  - Modelos complexos tendem a sobreajustar.

Técnicas de Compressão de Modelos de ML

Quantização

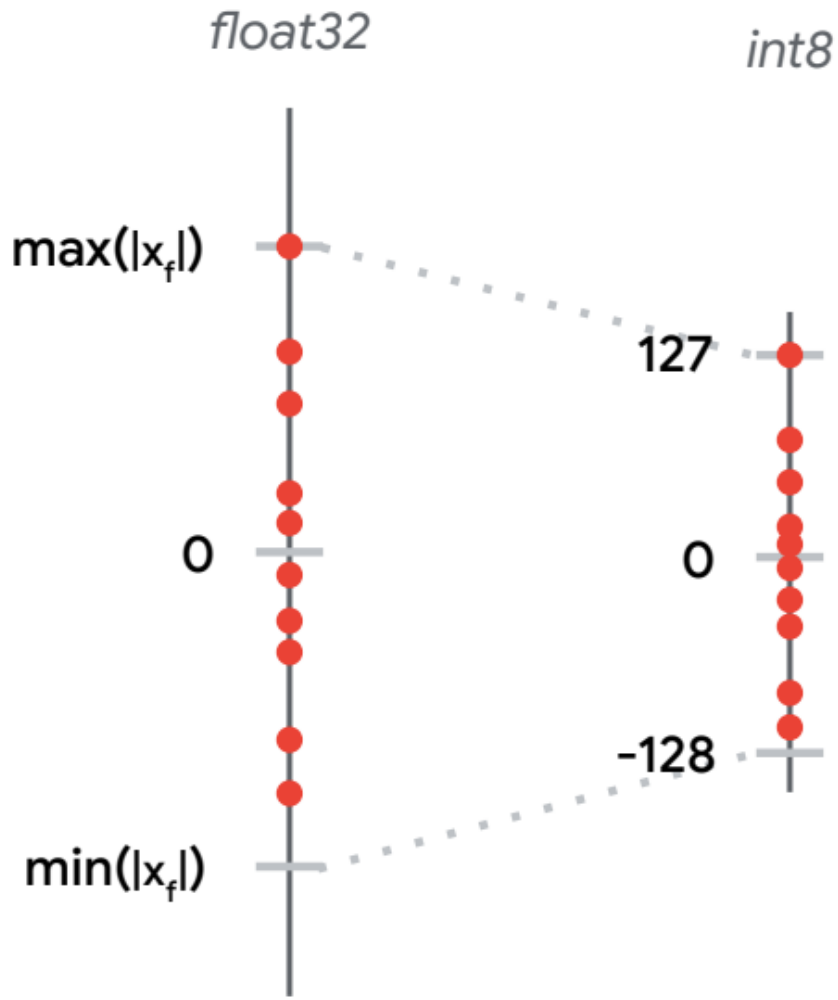


- Outra técnica de compressão bastante usada é chamada de **quantização**.
- Ela **reduz a precisão numérica dos dados** usados para representar os parâmetros (pesos, ativações, etc.) dos modelos.
- A ideia é **mapear** o intervalo dos valores **float** para um **intervalo de inteiros**.



Técnicas de Compressão de Modelos de ML

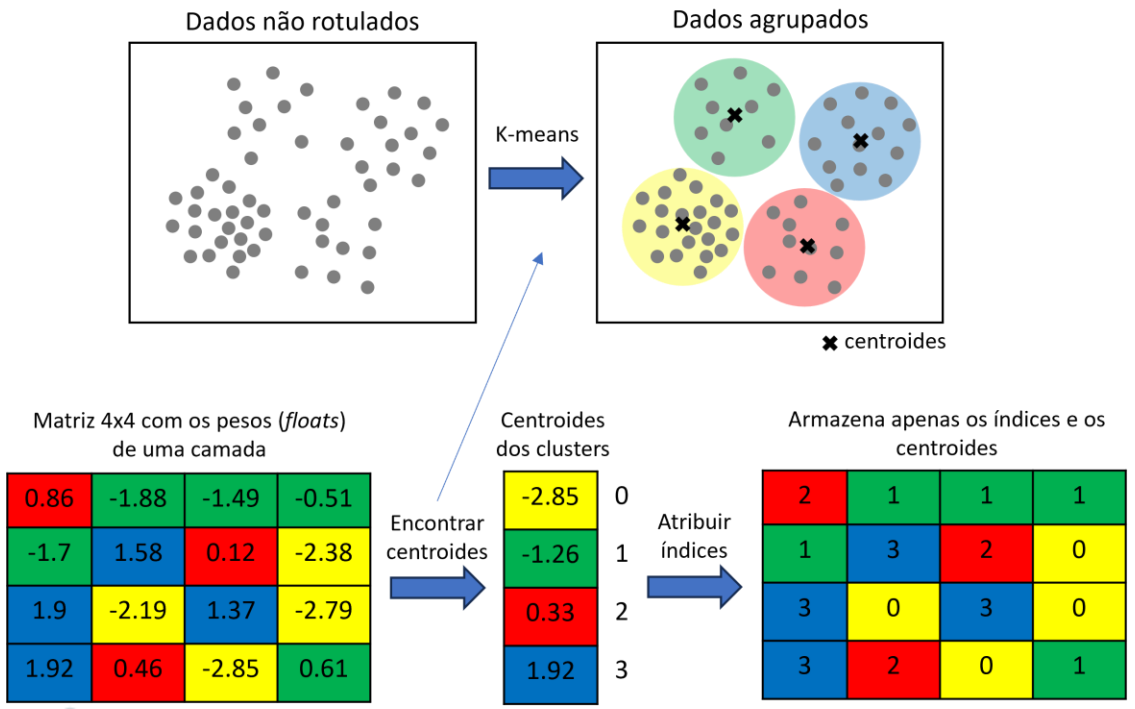
Quantização



- A **quantização reduz a quantidade de valores** que podem ser assumidos por uma variável.
- A quantização **aumenta a granularidade** da representação numérica.
- Com a quantização de float32 para int8, temos uma redução de 4 vezes.
- A quantização **pode reduzir o desempenho do modelo**, mas, em muitos casos, ela **não é significativa** e **pode ser tolerada** quando desejamos utilizar dispositivos EdgeML.
- Aplicada após o treinamento.
- Entretanto, existem técnicas onde o modelo é treinado ciente da quantização, resultando em nenhuma ou redução mínima do desempenho.

Técnicas de Compressão de Modelos de ML

Clustering ou Compartilhamento de Pesos



- O clustering de pesos reduz o **tamanho de armazenamento e de transferência através da rede** de modelos de ML.
- O clustering reduz o tamanho do modelo substituindo **pesos semelhantes em uma camada por um mesmo valor** (i.e., o centroide mais próximo).
- Assim, podemos **armazenar os índices e os centroides**:
  - Neste caso, reduzimos o tamanho da matriz de 16 floats diferentes para 4 floats distintos e 16 índices de 2 bits.
- No caso de armazenamento ou transferência do modelo, ferramentas de compactação, como o *zip*, aproveitam a redundância nos dados para obter maior compactação.
- **Aplicado após o treinamento.**

Técnicas de Compressão de Modelos de ML

Clustering ou Compartilhamento de Pesos

Image classification

Model	Original		Clustered			
	Top-1 accuracy (%)	Size of compressed .tflite (MB)	Configuration	# of clusters	Top-1 accuracy (%)	Size of compressed .tflite (MB)
MobileNetV1	70.976	14.97	Selective (last 3 Conv2D layers)	16, 16, 16	70.294	7.69
			Selective (last 3 Conv2D layers)	32, 32, 32	70.69	8.22
			Full (all Conv2D layers)	32	69.4	4.43
MobileNetV2	71.778	12.38	Selective (last 3 Conv2D layers)	16, 16, 16	70.742	6.68
			Selective (last 3 Conv2D layers)	32, 32, 32	70.926	7.03
			Full (all Conv2D layers)	32	69.744	4.05

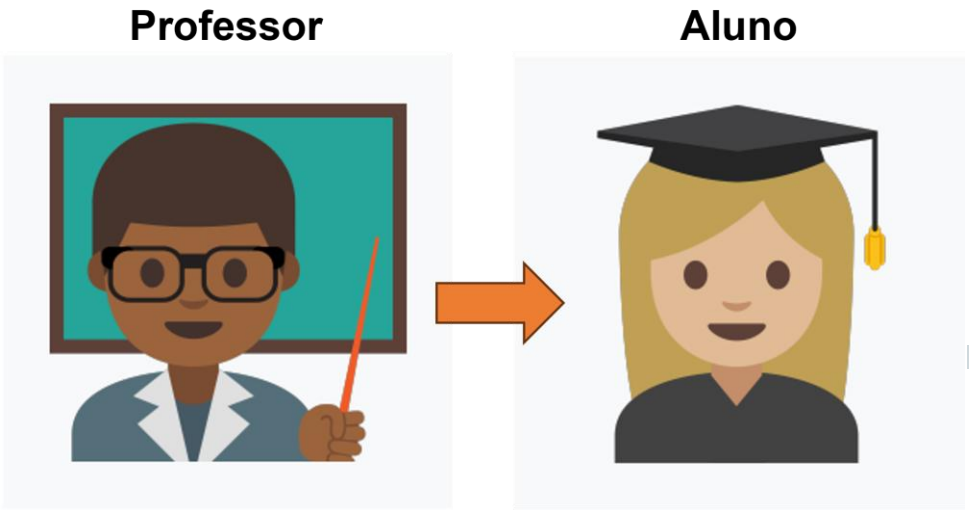
The models were trained and tested on ImageNet.

- A tabela ao lado mostra alguns resultados de *clustering*.

→ Notem que os tamanhos são reduzidos, mas que há uma redução na acurácia.

Técnicas de Compressão de Modelos de ML

Knowledge Distillation (Destilação de Conhecimento)



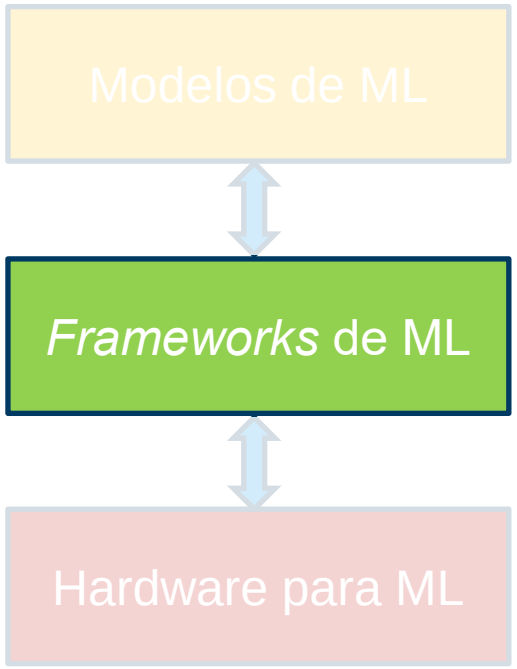
- É a técnica de treinamento de modelos de ML em que um **modelo maior e mais complexo** (i.e., de maior capacidade), conhecido como "**professor**", é usado para **ensinar** (treinar) um **modelo menor e mais simples**, conhecido como "**aluno**".

- A ideia principal é **transferir o conhecimento** do modelo grande para o menor, **mantendo o desempenho**, mas com
  - **menos parâmetros,**
  - **menor consumo de memória e**
  - **maior velocidade de inferência**.

- Em outras palavras, queremos que o aluno aprenda a **generalizar** como o professor, mas de **forma mais eficiente**.

destilar: reduzir algo, mas mantendo sua parte mais importante.

FRAMEWORKS DE ML



- Um **framework** de ML é uma biblioteca ou um conjunto de ferramentas que **fornece funcionalidades e abstrações que facilitam a criação, treinamento e implantação** (i.e., inferência) de modelos de ML.
- Alguns exemplos populares de **frameworks** de ML incluem TensorFlow, PyTorch, Scikit-learn e MXNet, etc.
- Usaremos o **Tensorflow (TF)**, pois é focado no desenvolvimento de redes neurais (profundas), além de apresentar versões direcionadas ao desenvolvimento de aplicações EdgeML.

FRAMEWORKS DE ML



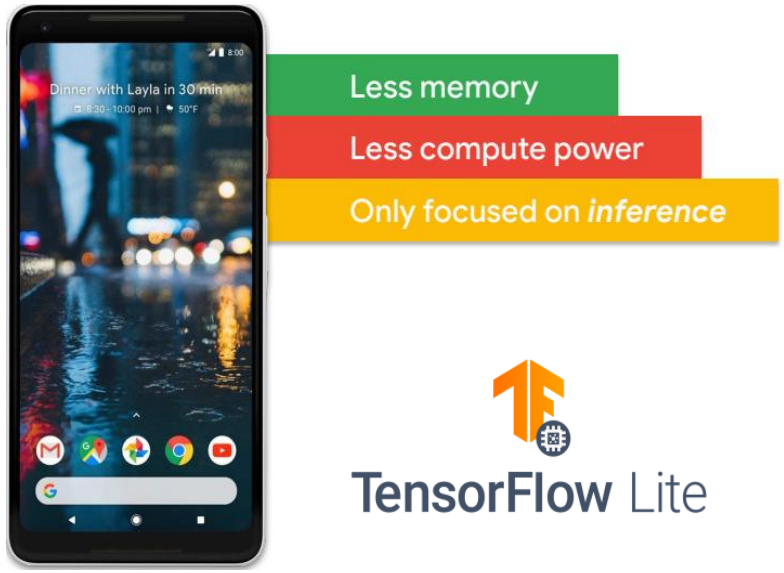
- Quando trabalhamos com aplicações envolvendo modelos de ML em **computadores de uso geral**, além da aplicação, SO e HW, nós usamos a biblioteca **TensorFlow** para a **criação, treinamento e implantação dos modelos**.
- O **TensorFlow** foi **projetado** para tarefas de **alto desempenho e uso intensivo dos recursos disponíveis** (como CPUs e/ou TPUs/GPUs), além de usar operações em **ponto flutuante**.
- Ou seja, não há preocupação com as eficiências **energética** e **computacional**.
- Além disso, o **TensorFlow** é **muito grande**, **não foi pensado para execução em dispositivos embarcados** e **realiza mais tarefas do que precisamos**.
- Por exemplo, em geral, por não conseguirmos fazer o treinamento nos dispositivos, não é necessário a parte da biblioteca destinada ao treinamento.

FRAMEWORKS DE ML



- O **objetivo** com dispositivos embarcados é apenas **fazer predições** (i.e., inferências).
- Portanto, quando queremos **executar modelos de ML em dispositivos com recursos limitados**, precisamos de um **framework mais enxuto e eficiente**.

TensorFlow (TF) Lite



- Agora conhecido como Lite Runtime (LiteRT).
- Usado em smartphones, tablets e sistemas embarcados.
- **Subconjunto do TF**, portanto, é **mais restrito e enxuto.**
- Projetado para ser **leve e eficiente em termos de uso de CPU e memória**.
- **Aproveita periféricos (GPUs, TPUs, NPUs, etc) e instruções específicas de CPUs (SIMD, MAC, etc.) para acelerar os modelos.**
- Suporta técnicas de otimização, como **quantização**, **pruning** e **clustering**.
- Realiza **apenas inferência (apenas executa o modelo treinado)**.
  - O modelo deve ser treinados em uma máquina com mais recursos.
- **Compatível com modelos treinados no TF**, **permitindo sua conversão**.
- Exemplos: https://github.com/tensorflow/examples/tree/master/lite/examples

TensorFlow (TF) Lite

Sequência de Trabalho com TF Lite



- **É possível gerar um modelo com extensão TF Lite de 3 formas:**

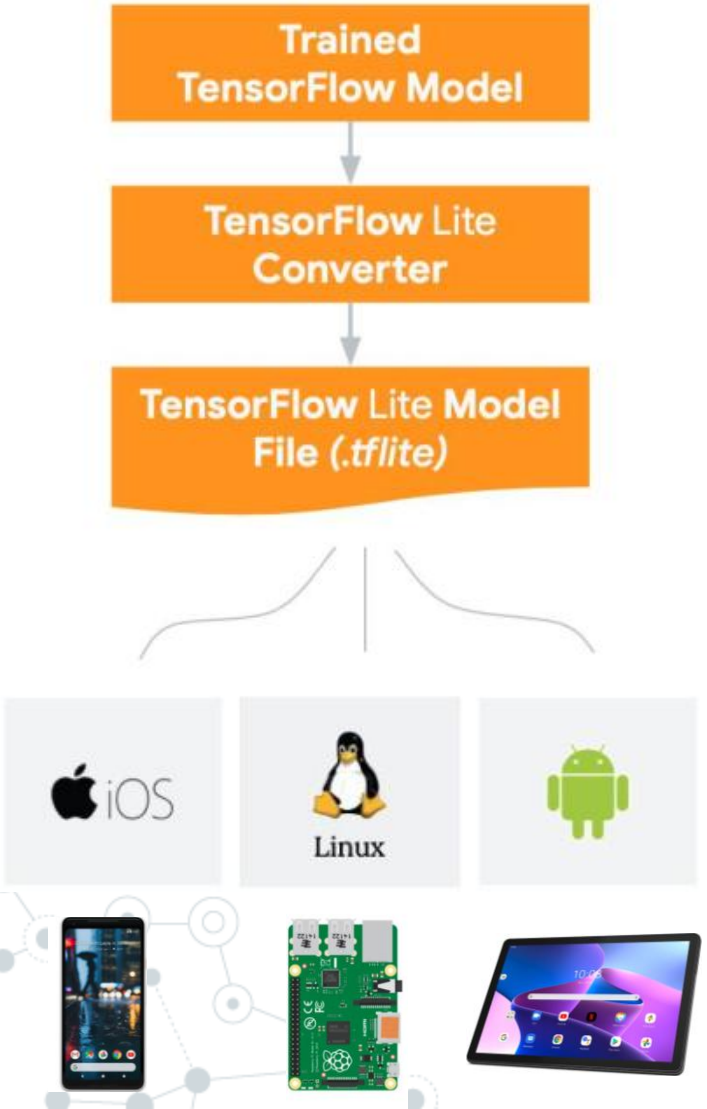
→ Usando um modelo pré-treinado do TF Lite.
- https://github.com/tensorflow/examples/tree/master/lite/examples

→ Criando um modelo do TensorFlow Lite.
- Para isso, podemos usar o TensorFlow Lite Model Maker

→ Convertendo um modelo do TF em um do TF Lite.

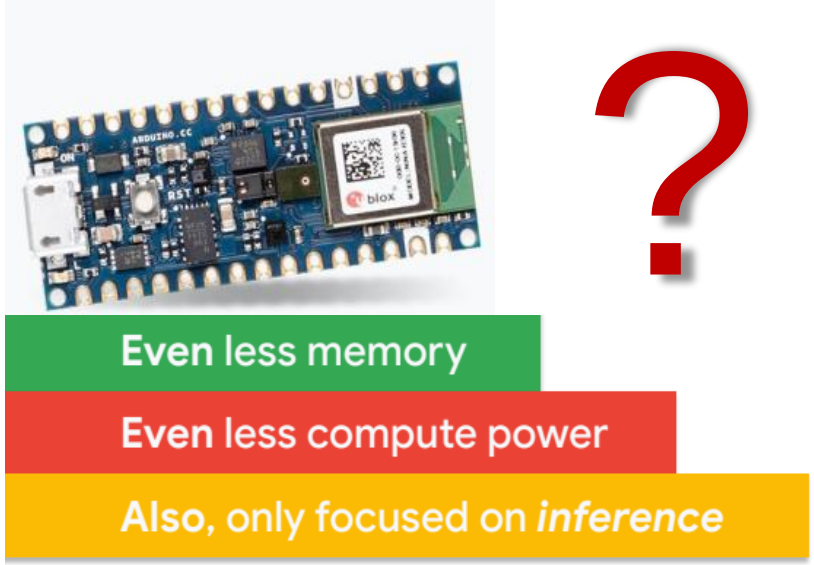
TensorFlow (TF) Lite

Sequência de Trabalho com TF e TF Lite



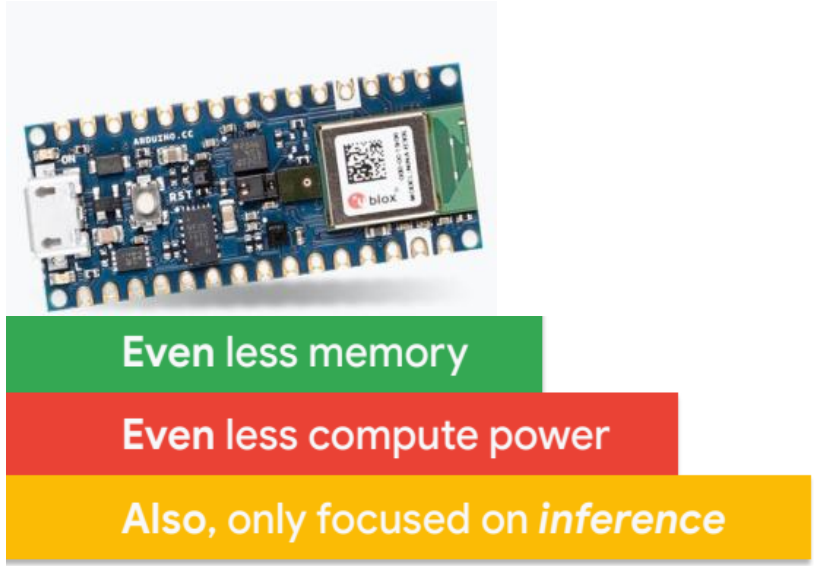
- O primeiro passo é **treinar um modelo criado com o TF** em um computador com muitos recursos computacionais.
- Após, usamos o **TF Lite converter para converter** esse modelo grande em um modelo pequeno.
  - Durante a conversão, podemos aplicar otimizações ao modelo.
- Esse processo gera um **arquivo com extensão .tflite**, o qual contém o modelo.
- Na sequência, **usamos este arquivo para implantar o modelo** em smartphones, tablets ou sistemas embarcados.
  - Basta carregar o arquivo com APIs específicas do TF Lite e realizar inferências.

TF Lite Micro



- Agora conhecido como LiteRT para microcontroladores.
- O TF Lite é a solução ideal para dispositivos edgeML.
  - **EdgeML:** dispositivos com capacidade computacional maior.
- Mas e quando falamos do uso de ML em **dispositivos com recursos extremamente limitados** e que requerem **baixíssimo consumo de energia**, como microcontroladores?
- Como habilitamos a inferência nesses dispositivos?
- Eles têm **capacidades computacionais muito menores** em comparação com smartphones e tablets.
- Portanto, **eles requerem uma implementação ainda mais enxuta e eficiente**.

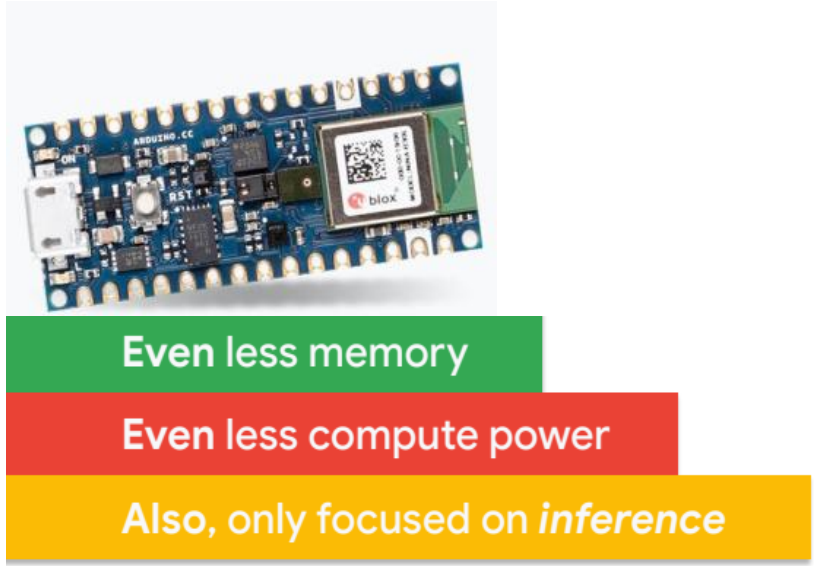
TF Lite Micro



TensorFlow Lite Micro

- O TF Lite Micro é uma versão ainda **mais leve e otimizada do TF Lite**.
- Projetado para **dispositivos embarcados com recursos computacionais extremamente limitados**, como microcontroladores.
- Escrito em C++ 17 e **roda apenas em plataformas de 32 bits**.
- Testado em sistemas com CPUs baseadas na série ARM Cortex-M e ESP32.
- Precisa apenas de **16 KB de memória**.

TF Lite Micro



TensorFlow Lite Micro

- **NÃO REQUER:**
  - suporte de **sistema operacional**,
  - **bibliotecas** C ou C++ padrão ou
  - **alocação dinâmica de memória**.

- Após a conversão do modelo TF em TF Lite, ele é convertido para uma **array de bytes** (chamado de **flatbuffer**), que é **compilada junto ao programa**.

TF Lite Micro

Pipeline de Desenvolvimento

TensorFlow

TensorFlow Lite/ Micro

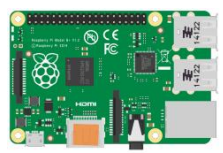
Train a model → Convert model → Optimize model → Deploy model at Edge → Make inferences at Edge

Treino com TF

Converter para TF Lite

Quantização, pruning, etc.

No caso do TF Lite Micro, converte o modelo em uma *array* de bytes.



Linux

iOS

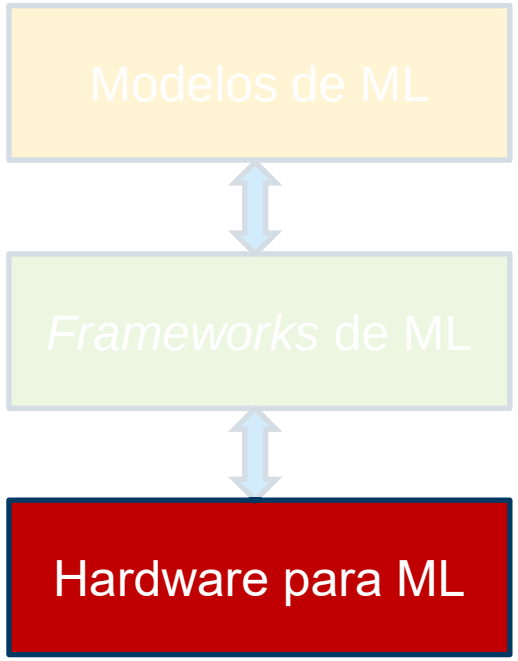








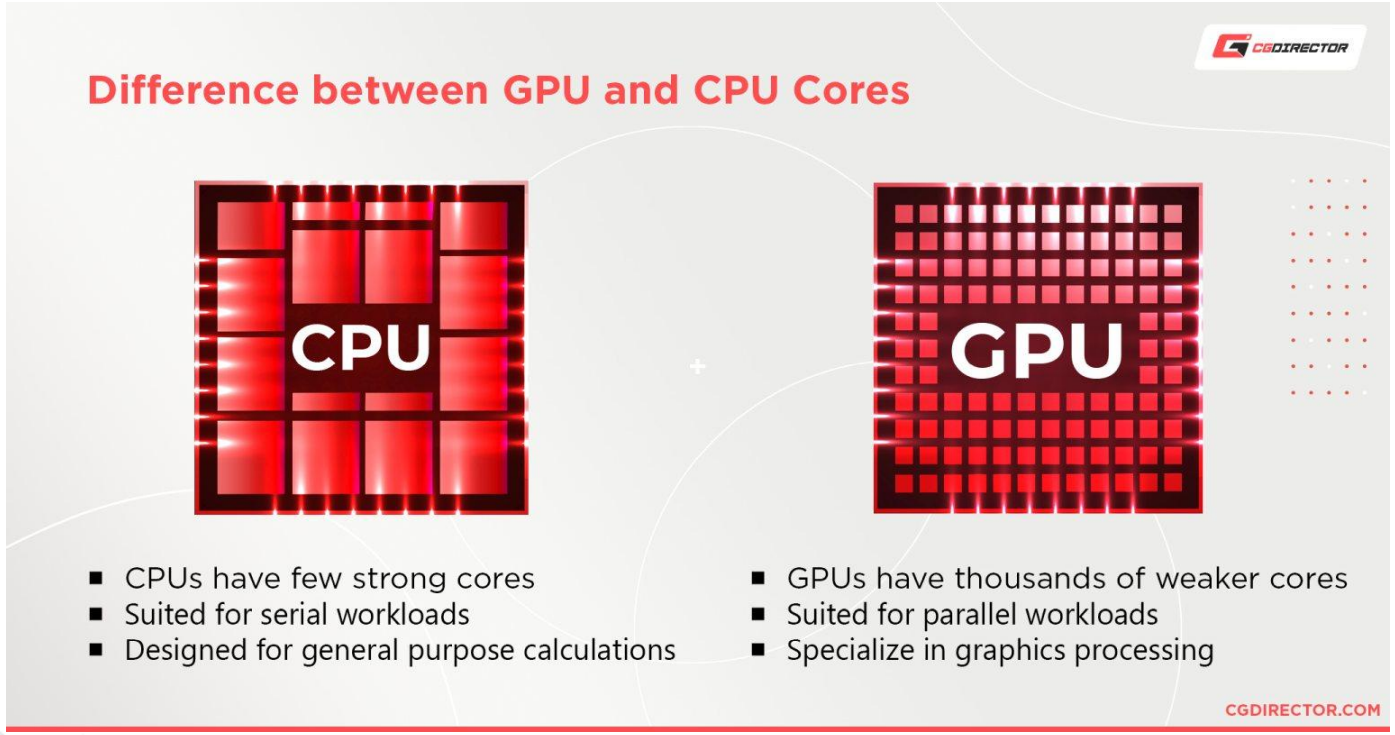
Hardware Especializado



- Outra forma para habilitar o uso de ML em dispositivos embarcados é através de **hardwares otimizados** para a **execução eficiente dos modelos**.
- Vários fabricantes têm lançados Systems-on-a-Chip (SoCs) e microcontroladores equipados com **GPUs** e/ou **aceleradores de ML**, também chamados de **neural processing units** (NPUs) ou **neural engines** (NEs).

Hardware Especializado

Graphics Processing Unit (GPU)

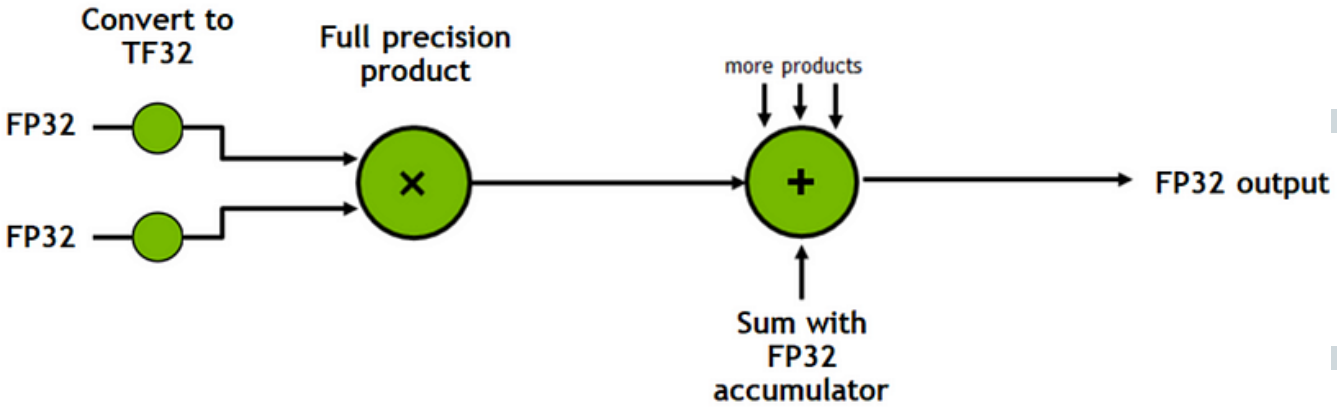


- Componente de HW especialmente **projetado para lidar com tarefas paralelas de computação** envolvendo grandes quantidades de dados.
  - Executam a mesma operação em paralelo em uma grande quantidade de dados.

- Isso as torna ideal para trabalhar com **aplicações com processamento intensivo que envolvam cálculos matriciais**, como
  - processamento e renderização de vídeo,
  - simulações físicas,
  - modelagem 3D,
  - treinamento de modelos de DL,
  - etc.

Hardware Especializado

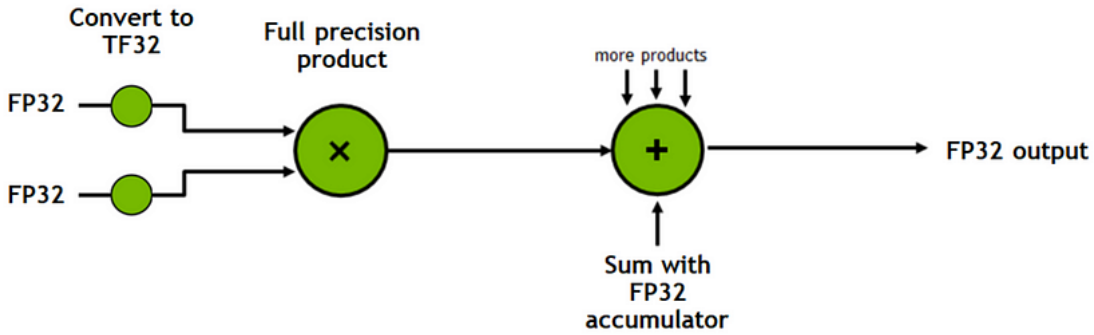
Graphics Processing Unit (GPU)

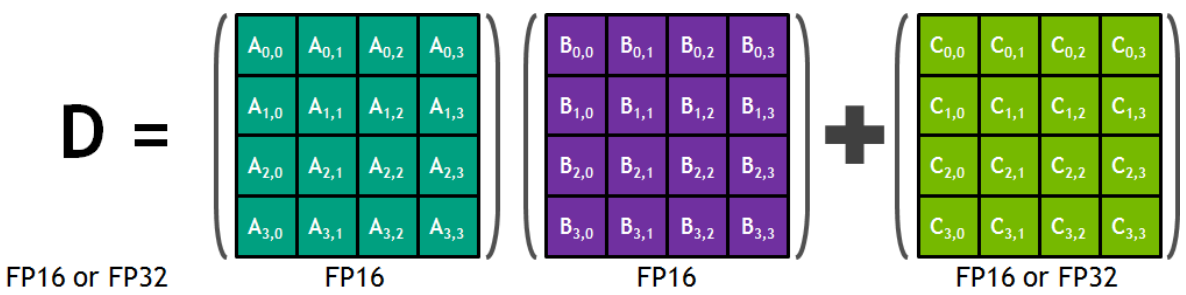


- Possuem vários pequenos **(GPU) cores especializados em operações de multiplicação e acumulação** (multiply and accumulate - **MAC**).
- Isso torna as GPUs muito boas em **operações paralelas** de multiplicação e adição de matrizes.
- Essas operações são **essenciais para o processamento de redes neurais**.

Hardware Especializado

Graphics Processing Unit (GPU)

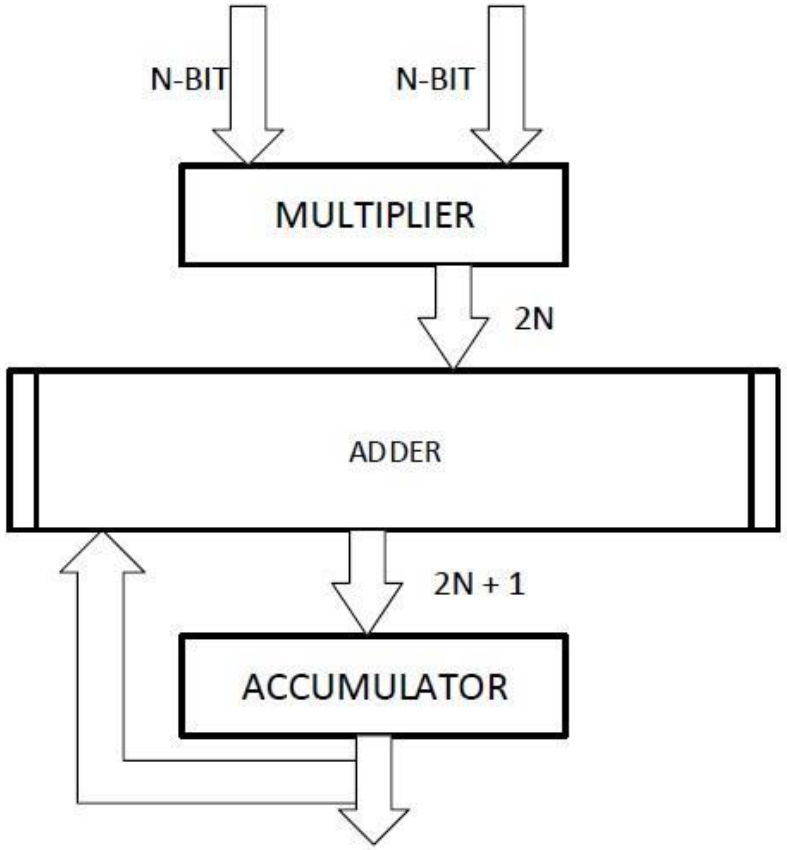




- Alguns modelos de GPUs têm cores **mais especializados ainda** para operações envolvendo matrizes.
- No caso da NVIDIA, esses cores são chamados de **Tensor Cores**.
- Enquanto os **GPU cores** executam **apenas uma operação MAC por ciclo de clock**, os **Tensor cores executam várias operações**.
  - Por exemplo, um Tensor core da NVIDIA executa uma operação MAC matricial (matrizes 4x4) por ciclo de clock.
- Assim, eles aceleram ainda mais o treinamento de redes neurais.

Hardware Especializado

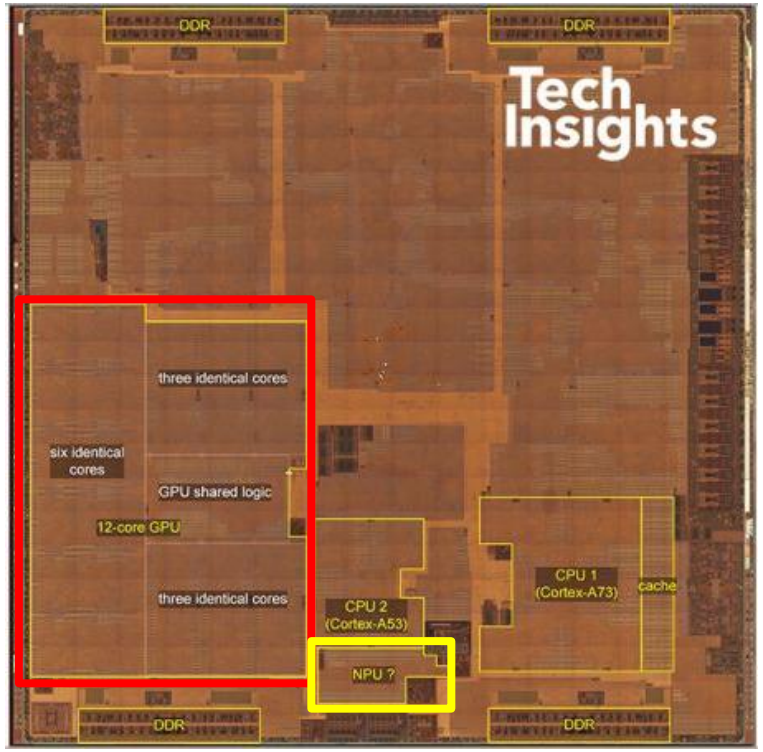
Neural processing units (NPUs)



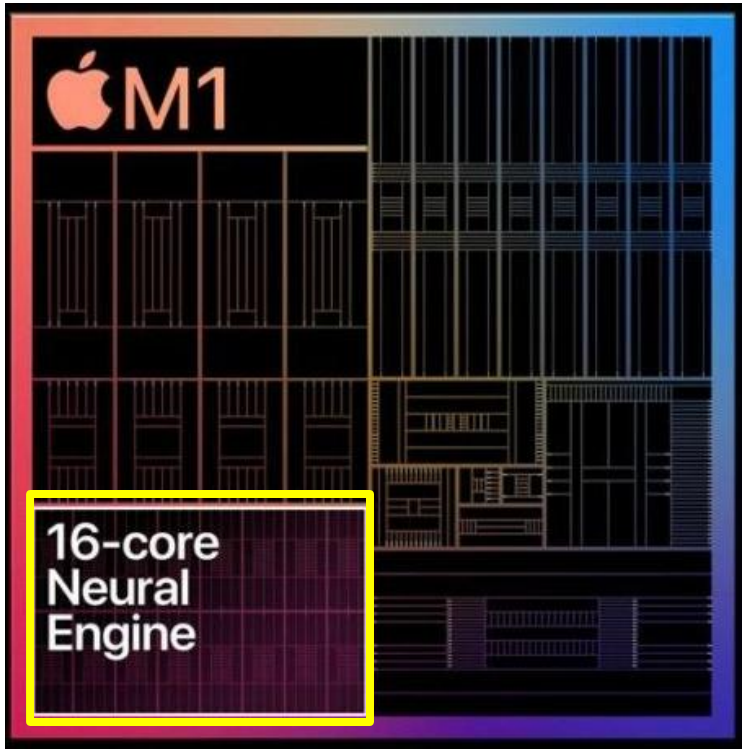
- NPUs ou NEs são **hardwares altamente especializados projetados para acelerar aplicações de ML**.
- Por serem altamente especializadas, oferecem uma execução muito mais rápida do que as CPUs e GPUs (sem Tensor cores).
- NPUs realizam **operações paralelas de MAC** e algumas suportam **compactação e descompactação de pesos**, ajudando a **minimizar o uso da memória** do sistema.

Hardware Especializado

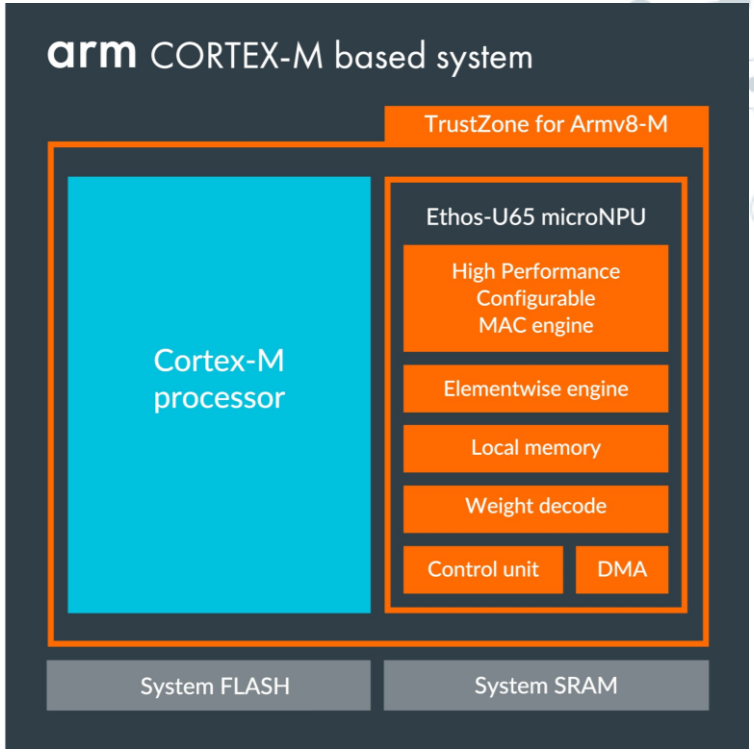
Exemplos:



Huawei's Kirin 970



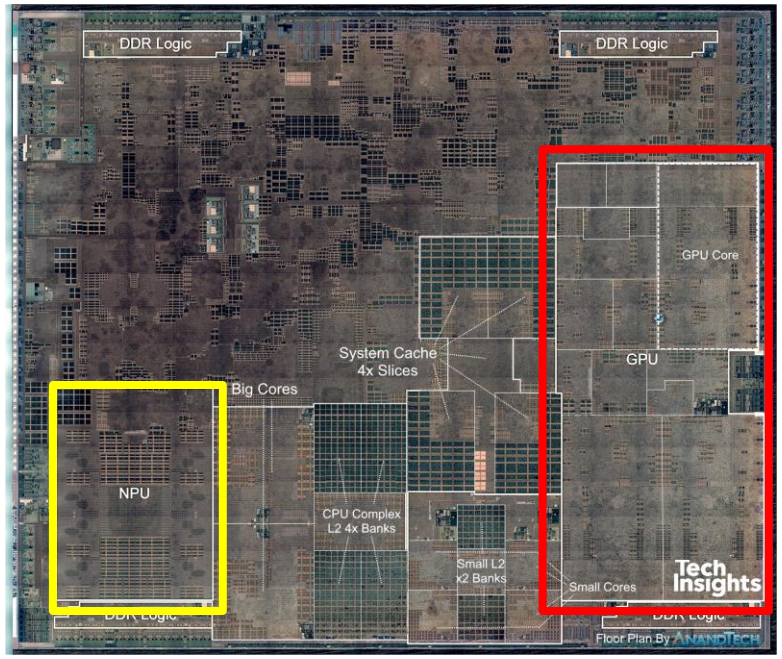
Apple's M1



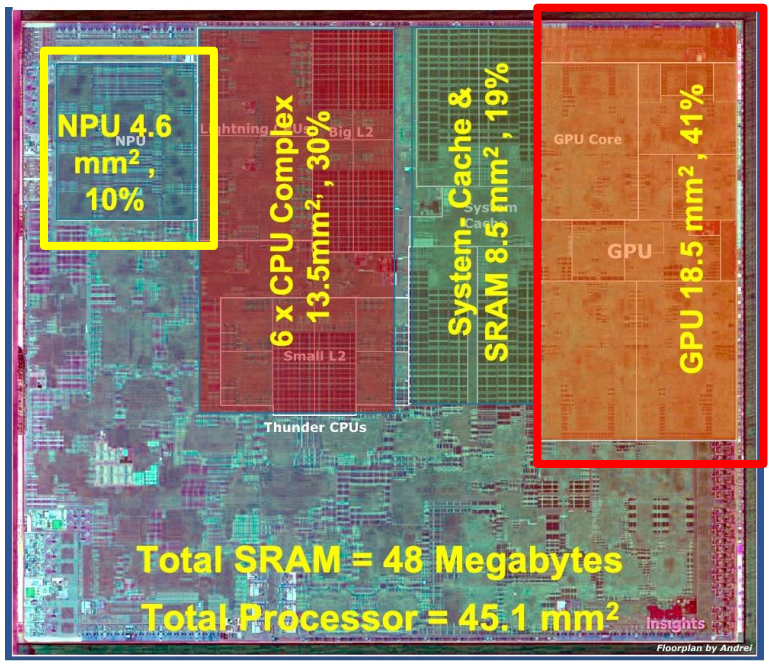
ARM's Cortex-M55, Ethos-U55, U65 e N78

Hardware Especializado

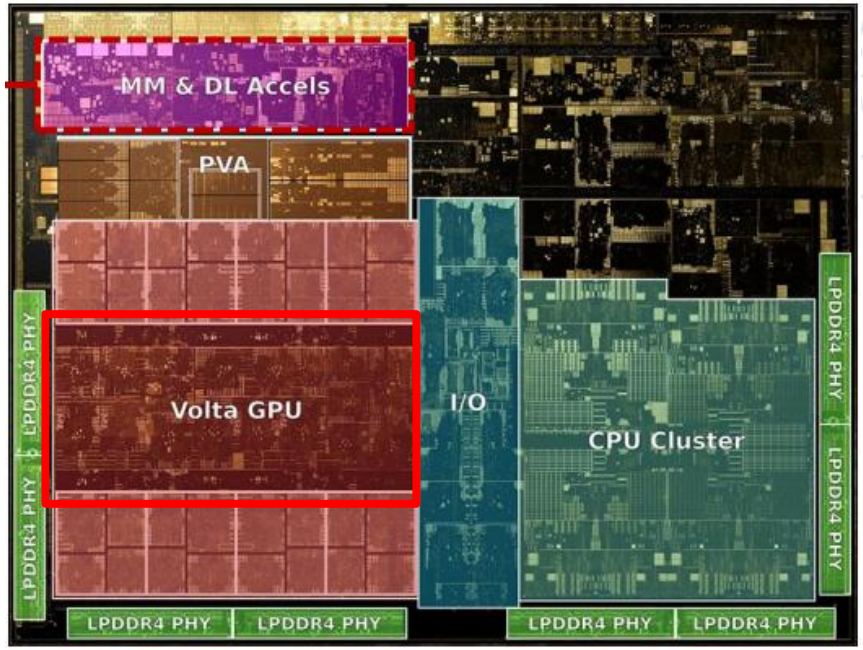
Exemplos:



Apple's A12



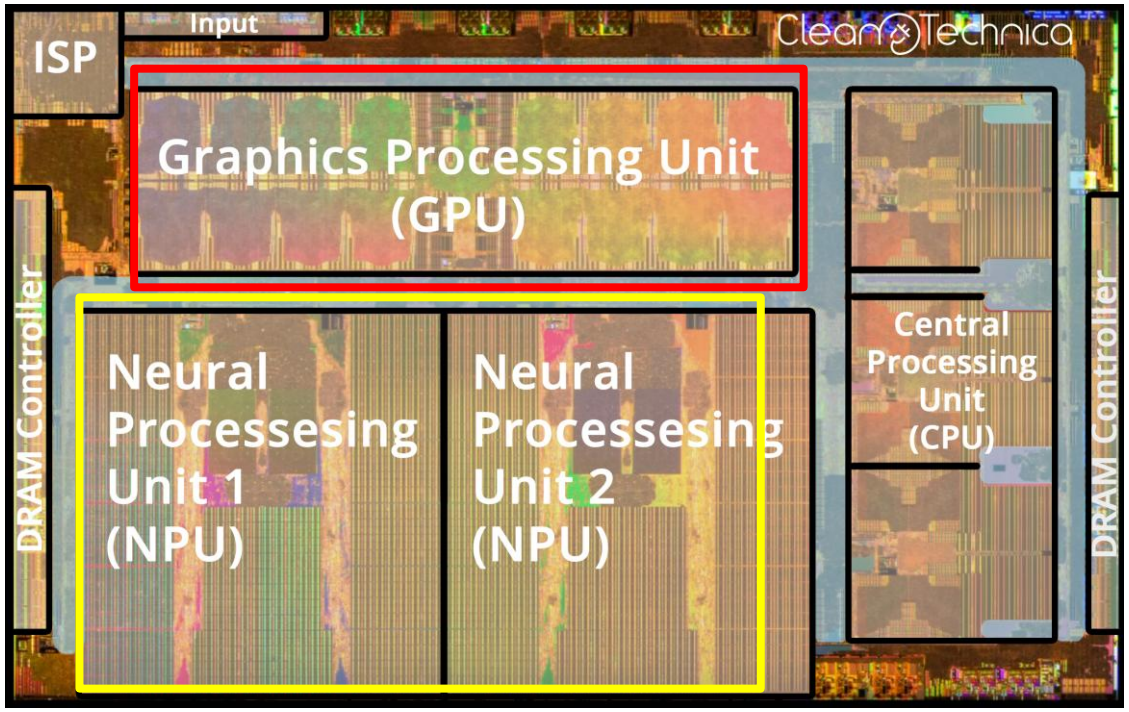
Apple's A13



NVIDIA's Xavier SoC (CUDA cores + Tensor cores)

Hardware Especializado

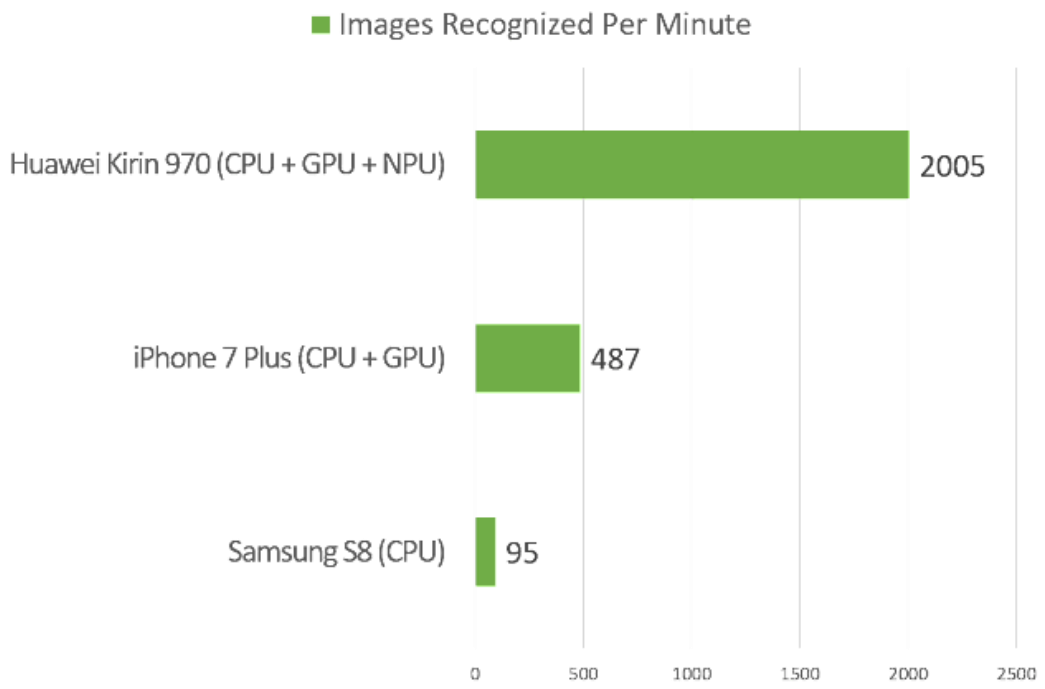
Exemplos:



Tesla's FSD

Hardware Especializado

NPU versus GPU



- Esses **HWs especializados habilitam aplicações mais complexas** como
  - Detecção, segmentação e rastreio de objetos,
  - Reconhecimento de fala em tempo real.
- A Figura ao lado mostra que a união de CPU+GPU+NPU no Kirin 970 da Huawei faz com que ele supere de longe a CPU+GPU do iPhone 7 plus na tarefa de **classificação de imagens**.

Fonte: https://thetechrevolutionist.com/2017/09/huaweis-neural-processing-unit-npu-and.html

Atividades

- Assistir aos vídeos do Prof. Marcelo Rovai sobre Google Colab e Python.
  - **Google Colab intro**: https://www.youtube.com/watch?v=m_Ueb88yd88&ab_channel=MarceloRovai
  - **Python review**: https://www.youtube.com/watch?v=07tGfteD4_s&ab_channel=MarceloRovai
  - **Notebooks e documentos usados nos vídeos**: https://github.com/Mjrovai/UNIFEI-IESTI01-TinyML-2022.1/tree/main/00_Curse_Folder/1_Fundamentals/Class_04a



Inatel

Obrigada pela Atenção!

Dúvidas?

victoria.souto@Inatel.br